

TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA TOÁN - TIN HỌC



**TIỂU LUẬN CUỐI KỲ HỌC PHẦN
LẬP TRÌNH SONG SONG VÀ PHÂN TÁN**

**TÌM HIỂU APACHE SPARK VÀ ỨNG DỤNG
THUẬT TOÁN RANDOM FOREST**

Giảng viên hướng dẫn: **PGS - TS. Dương Văn Hải**

Sinh viên thực hiện – Mã số sinh viên:

- 1. Phùng Ngọc Bảo - 2212318**
- 2. Nguyễn Công Đạt - 2214025**

Lớp: **KLK46**

Lâm Đồng, tháng 12/2025

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN	4
BẢNG PHÂN CÔNG CÔNG VIỆC VÀ NHẬN XÉT	5
BẢNG CÁC THUẬT NGỮ TIẾNG ANH	6
DANH MỤC HÌNH ẢNH	8
DANH MỤC BẢNG BIỂU	10
LỜI NÓI ĐẦU	11
1. Giới thiệu Apache Spark:	12
1.1. Các thành phần chính của Apache Spark:	14
1.1.1. Spark Core - nền tảng của Spark:	14
1.1.2. Spark SQL - truy vấn tương tác:	15
1.1.3. Spark Streaming – xử lý thời gian thực:	16
1.1.4. Spark MLlib - thư viện học máy:	17
1.1.5. Spark GraphX – xử lý đồ thị:	18
1.2. Kiến trúc của Apache Spark:	19
1.2.1. Các thành phần chính:	19
1.2.2. Cách thức hoạt động của Spark:	20
1.2.3. Các trừu tượng chính (Main Abstractions):	20
1.2.4. Trình quản lý Cụm và Chế độ Thực thi:	20
1.3. Sự khác biệt của Spark so với Hadoop:	21
1.4. Ưu điểm và nhược điểm của Apache Spark:	22
1.4.1. Ưu điểm của Apache Spark:	22
1.4.2. Nhược điểm của Apache Spark:	23
1.5. Ứng dụng của Apache Spark:	23
2. Cài đặt Apache Spark và triển khai cụm máy tính toán:	26
2.1. Cài đặt Apache Spark trên hệ điều hành Windows:	26
2.1.1. Chuẩn bị các phần mềm nền tảng:	26
2.1.2. Tải và giải nén Apache Spark:	27
2.1.3. Cấu hình Hadoop Utils cho Windows:	28
2.1.4. Thiết lập biến môi trường (Environment Variables):	30

2.1.5. Kiểm thử cài đặt:.....	31
2.2. Kết nối cụm máy Spark:	32
2.2.1. Thiết lập mạng kết nối (Network Configuration):	32
2.2.2. Khởi động máy Master (Master Node):	35
2.2.3. Khởi động máy Worker (Worker Node):.....	36
2.2.4. Kiến trúc cụm máy Apache Spark:	37
3. Đếm từ (Word count) với Apache Spark:	39
3.1. Word Count với Spark:	40
3.2. So sánh Word Count khi sử dụng Spark với không sử dụng:.....	41
3.3. So sánh Word Count với độ lớn dữ liệu tăng dần:	42
4. Ứng dụng thuật toán Random Forest trong Apache Spark:	43
4.1. Giới thiệu thuật toán phân loại Random Forest và khả năng tính toán song song:.....	43
4.1.1. Thuật toán Random Forest:.....	43
4.1.2. Kiến trúc thuật toán phân loại Random Forest song song:	46
4.2. Triển khai Random Forest trong Spark và so sánh hiệu năng:	47
5. Phương pháp cải thiện tốc độ xử lý:	51
5.1. Data mirroring thủ công:	52
Thử nghiệm lại Word count với Data Mirroring thủ công:	56
5.2. Quản lý dữ liệu với HDFS:	57
5.2.1. Cài đặt và cấu hình HDFS:	57
5.2.2. Tạo dữ liệu và chuyển sang các máy với HDFS:.....	64
6. Nhận xét tổng thể:	71
7. Nhận xét về quá trình thực hiện tiểu luận cuối kỳ và hướng phát triển:	73
8. Tài liệu tham khảo:	75

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ NHẬN XÉT

Thành viên	Công việc	Nhận xét hoàn thành
Phùng Ngọc Bảo - 2212318	• Tìm hiểu lý thuyết về hệ thống phân tán, HDFS, kiến trúc Spark, mô hình MapReduce. • Cài đặt môi trường Hadoop, cấu hình HDFS, thiết lập Spark cluster và kiểm tra trạng thái các DataNode. • Xây dựng, chạy và phân tích thí nghiệm Word Count (so sánh tuần tự – Spark, mirroring thủ công – HDFS). • Thiết kế bộ thí nghiệm thay đổi kích thước dữ liệu và số core, thu thập thời gian chạy và độ chính xác. Xử lý số liệu, lập bảng, vẽ biểu đồ so sánh hiệu năng. • Viết và chỉnh sửa các nội dung bài tiểu luận.	100%
Nguyễn Công Đạt - 2214025	• Tìm hiểu lý thuyết về hệ thống phân tán, HDFS, kiến trúc Spark, mô hình MapReduce. • Cài đặt môi trường Hadoop, cấu hình HDFS, thiết lập Spark cluster và kiểm tra trạng thái các DataNode. • Tìm hiểu và cài đặt thuật toán Random Forest trên Sklearn và Spark MLlib. Xử lý số liệu, lập bảng, vẽ biểu đồ so sánh hiệu năng. • Viết và chỉnh sửa các nội dung mục 1, 2, 3.	100%

Trong quá trình làm việc nhóm, mỗi thành viên đều đã hoàn thành đúng với nhiệm vụ và thời hạn mà nhóm đã đưa ra. Tuy quá trình làm việc còn có một vài những khó khăn nhưng các bạn cũng đã hoàn thành tốt nhiệm vụ của mình, hỗ trợ đánh giá chéo, hiệu chỉnh phần bài của nhau, góp phần hoàn thành bài tiểu luận cuối kì.

BẢNG CÁC THUẬT NGỮ TIẾNG ANH

Vì là phần thực hiện tiểu luận học phần Lập trình song song và phân tán, nên sẽ có những thuật ngữ dùng trong lập trình và phân tán dữ liệu, khoa học dữ liệu hay học máy được sử dụng nhiều trong bài để đảm bảo tính rành mạch. Dưới đây là bảng giải thích các thuật ngữ được dùng nhiều trong bài tiểu luận.

Tiếng Anh	Tiếng Việt	Giải thích
Parallel Programming	Lập trình Song song	Phương pháp thực thi nhiều lệnh cùng lúc trên các lõi khác nhau của CPU.
Distributed Computing	Tính toán Phân tán	Sử dụng nhiều máy tính (node) để giải quyết một bài toán lớn.
Cluster	Cụm máy chủ	Tập hợp các máy tính (node) hoạt động cùng nhau.
Node	Nút/Máy chủ	Một máy tính riêng lẻ trong cụm.
RDD (Resilient Distributed Dataset)	Tập dữ liệu phân tán Chịu lỗi	Cấu trúc dữ liệu bất biến, phân tán và chịu lỗi của Spark.
DataFrame/Dataset	Khung dữ liệu/Tập dữ liệu	Cấu trúc dữ liệu có cấu trúc và có khả năng tối ưu hóa.
SparkContext	Ngữ cảnh Spark	Điểm vào chính để kết nối với cụm.
Driver Program	Chương trình Điều khiển	Tiến trình chạy trên Master, quản lý và lập lịch công việc.
Executor	Bộ Thực thi	Tiến trình chạy trên Worker Node, thực hiện các Task.
Task	Tác vụ/Nhiệm vụ	Đơn vị công việc nhỏ nhất được gửi đến Executor.

Transformation	Phép Biến đổi	Các thao tác tạo RDD/DataFrame mới (thực thi lười - Lazy).
Lazy Evaluation	Đánh giá Lười	Spark trì hoãn tính toán cho đến khi gặp Action.
Stage	Giai đoạn	Tập hợp các Task thực hiện cùng một phép biến đổi, được phân chia bởi Shuffle.
Shuffle	Trộn/Phân phối lại	Giai đoạn trao đổi dữ liệu qua mạng giữa các Executor.
Data Locality	Vị trí Dữ liệu	Nguyên tắc chạy tính toán gần nơi dữ liệu được lưu trữ.
Partition	Phân vùng	Đơn vị logic của RDD/DataFrame, được xử lý song song.
In-Memory Processing	Xử lý trong Bộ nhớ	Xử lý dữ liệu trực tiếp trong RAM (ưu điểm tốc độ của Spark).
Fault Tolerance	Khả năng Chịu lỗi	Khả năng phục hồi của RDD khi các node bị lỗi.
Scalability	Khả năng Mở rộng	Khả năng xử lý khi quy mô dữ liệu và tài nguyên tăng lên.
MapReduce	MapReduce	Mô hình lập trình phân tán (Spark được xây dựng dựa trên ý tưởng này).
Master Node	Nút Chủ/Master	Máy điều phối chung của cụm (chạy Driver/NameNode).
Worker Node	Nút Thợ/Worker	Máy thực hiện các tác vụ tính toán (chạy Executor/DataNode).

DANH MỤC HÌNH ẢNH

Hình 1: Các đặc trưng của Apache Spark.....	12
Hình 2: Spark tương thích với nhiều hệ thống lưu trữ	13
Hình 3: Các thành phần của Apache Spark.....	13
Hình 4: Nhiệm vụ của Spark Core	14
Hình 5: Kiến trúc của Spark SQL.....	15
Hình 6: Cơ chế xử lý dữ liệu của Spark Streaming.....	16
Hình 7: Xử lý dữ liệu với Spark Streaming và các thư viện của Spark.....	17
Hình 8: Các thuật toán học máy được tích hợp sẵn trong Spark MLlib.....	17
Hình 9: GraphX cung cấp nhiều tính năng xử lý đồ thị.....	18
Hình 10: Kiến trúc hoạt động của Apache Spark	19
Hình 11: Hơn 62000 công ty đang sử dụng Apache Spark toàn thế giới	25
Hình 12: Thư mục Java 8 sau khi cài đặt	26
Hình 13: Cài đặt Python	27
Hình 14: Chọn phiên bản Spark và tải xuống.....	27
Hình 15: Thư mục Spark sau khi giải nén.....	28
Hình 16: Đặt winutils.exe và hadoop.dll vào thư mục	29
Hình 17: Đặt hadoop.dll vào thư mục System32	29
Hình 18: Cấu hình biến môi trường.....	30
Hình 19: Cấu hình thư mục đường dẫn Path	31
Hình 20: Apache Spark được cài đặt thành công	32
Hình 21: Cấu hình mạng LAN	33
Hình 22: Kiểm tra kết nối từ Master đến các Worker	34
Hình 23: Màn hình command kích hoạt Master Node	35
Hình 24: Giao diện quản trị của máy Master.....	35
Hình 25: Máy Worker (26.11.67.195) kết nối thành công đến Master	36
Hình 26: Máy Master xuất thông báo có máy Worker kết nối thành công	37
Hình 27: Sơ đồ cụm máy.....	37
Hình 28: Giao diện quản lý kết nối	38
Hình 29: Word Count với mô hình tuần tự.....	39
Hình 30: Kiến trúc Word Count với Spark.....	39

Hình 31: Bảng kết quả trả về wordcount trong PyCharm	40
Hình 32: Các kết quả chạy.....	41
Hình 33: So sánh thời gian chạy với các mức độ lớn của file	43
Hình 34: Minh họa thuật toán Random Forest	44
Hình 35: Kỹ thuật Bootstrap Aggregating.....	44
Hình 36: Kiến trúc tuần tự của Random Forest.....	45
Hình 37: Kiến trúc song song của Random Forest.....	46
Hình 38: Log hiển thị quá trình chạy Random Forest	48
Hình 39: Quá trình cảnh báo dữ liệu bị lạc mất do quá trình truyền tải đến nút worker	48
Hình 40: Giao diện quản lý tài nguyên.....	49
Hình 41: So sánh thời gian và độ chính xác (1 worker node)	50
Hình 42: So sánh thời gian và độ chính xác (2 workers node).....	51
Hình 43: Các file dữ liệu được đặt sẵn ở từng máy	53
Hình 44: Log trạng thái sinh dữ liệu mẫu.....	53
Hình 45: Dữ liệu sinh ra được đặt cố định ở C.....	53
Hình 46: Kết quả chạy Random Forest với Data Mirroring.....	54
Hình 47: So sánh thời gian chạy.....	56
Hình 48: Thư mục Hadoop sau khi giải nén.....	58
Hình 49: Kết quả điền file core-site.xml	59
Hình 50: Kết quả điền file hdfs-site.xml	60
Hình 51: kết quả điền file workers	60
Hình 52: Cấu hình đường dẫn Java	61
Hình 53: Overview của hệ thống HDFS.....	61
Hình 54: Thống kê của hệ thống HDFS	62
Hình 55: Thông tin về lưu trữ của các Datanode.....	63
Hình 56: Hoạt động phân chia dữ liệu của HDFS (16 tăng lên 20 blocks)	64
Hình 57: Kết quả trả về sau khi sinh dữ liệu thành công.....	64
Hình 58: Kết quả cuối sau khi HDFS phân tán thành công.....	65
Hình 59: Kết quả kiểm tra các file dữ liệu được HDFS phân tán.....	65
Hình 60: Kiểm tra dữ liệu thông qua giao diện HDFS	66
Hình 61: Kiến trúc phân tán 2 Worker với HDFS.....	66

Hình 62: So sánh kết quả chạy thuật toán và độ chính xác (HDFS)	67
Hình 63: Kiến trúc phân tán với 3 Workers	68
Hình 64: Giao diện quản lý cụm gồm 3 workers.....	68
Hình 65: So sánh thời gian chạy theo số nhân (Cores).....	69
Hình 66: Warehouse-scale computer.....	71
Hình 67: Amazon AWS có 428 server ở khắp nơi. Khắc phục điểm yếu về địa lý.	72

DANH MỤC BẢNG BIỂU

Bảng 1: So sánh Hadoop và Spark	22
Bảng 2: Danh sách tài nguyên của cụm máy.....	38
Bảng 3: So sánh thời gian chạy giữa các yếu tố (giây).....	41
Bảng 4: Bảng so sánh thời gian chạy với mỗi độ lớn khác nhau	43
Bảng 5: Dữ liệu mô phỏng	47
Bảng 6: So sánh các kết quả chạy (giây).....	49
Bảng 7: So sánh độ chính xác (%).....	50
Bảng 8: So sánh tốc độ chạy và độ chính xác	54
Bảng 9: So sánh thời gian đếm sau khi đặt dữ liệu sẵn ở các máy	56
Bảng 10: So sánh kết quả chạy thuật toán và độ chính xác.....	67
Bảng 11: Danh sách tài nguyên cụm máy (3 workers).....	68
Bảng 12: So sánh thời gian chạy với số nhân.....	69

LỜI NÓI ĐẦU

Trong bối cảnh cuộc Cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, dữ liệu đã trở thành một trong những tài nguyên quan trọng nhất, quyết định khả năng cạnh tranh và đổi mới của các tổ chức. Khối lượng dữ liệu khổng lồ được tạo ra mỗi ngày từ các hệ thống giao dịch tài chính, thiết bị IoT, mạng xã hội, thương mại điện tử hay các ứng dụng trí tuệ nhân tạo,... đã đặt ra yêu cầu cấp thiết về những công cụ xử lý dữ liệu hiệu quả, nhanh chóng và có khả năng mở rộng.

Apache Spark nổi lên như một nền tảng xử lý dữ liệu phân tán mạnh mẽ, cho phép thực thi các tác vụ tính toán ở tốc độ cao thông qua khả năng xử lý song song, tối ưu hóa bộ nhớ và phân tán dữ liệu trên nhiều nút trong cụm máy (cluster). Nhờ đó, Spark đã trở thành giải pháp quan trọng trong nhiều hệ thống Big Data hiện đại, đặc biệt trong các bài toán phân tích dữ liệu, học máy và khai thác tri thức từ dữ liệu.

Trong số các thuật toán học máy được triển khai trên Spark, Random Forest là phương pháp mạnh mẽ, ổn định và hiệu quả trong phân lớp và dự đoán nhờ khả năng kết hợp nhiều cây quyết định, giảm thiểu hiện tượng quá khớp và cải thiện độ chính xác. Việc tích hợp Random Forest trong môi trường phân tán của Spark càng làm tăng sức mạnh của thuật toán, cho phép xử lý dữ liệu lớn với tốc độ cao và độ tin cậy cao.

Xuất phát từ những yêu cầu thực tiễn và ý nghĩa quan trọng của việc xử lý dữ liệu phân tán, tiểu luận này tập trung nghiên cứu mục tiêu:

- Trình bày tổng quan về nền tảng Apache Spark và kiến trúc xử lý song song phân tán.
- Phân tích nguyên lý hoạt động của thuật toán Random Forest trong bài toán học máy tuần tự cũng như song song.
- Ứng dụng Random Forest trên Apache Spark để mô phỏng quá trình xử lý dữ liệu lớn và đánh giá hiệu suất, đặc biệt là về thời gian xử lý với dữ liệu lớn.

Thông qua nghiên cứu, nhóm thực hiện mong muốn làm rõ tiềm năng của Spark trong việc triển khai các thuật toán học máy quy mô lớn, đồng thời minh chứng hiệu quả của Random Forest khi được vận hành trong môi trường phân tán. Kết quả thu được không chỉ góp phần củng cố kiến thức của học phần Lập trình song song và phân tán mà còn mở ra hướng ứng dụng thực tiễn trong các hệ thống phân tích dữ liệu lớn hiện nay.

1. Giới thiệu Apache Spark:

Apache Spark ra đời vào năm 2009 tại UC Berkeley như một dự án nghiên cứu với mục tiêu cải thiện những hạn chế của mô hình MapReduce, đặc biệt trong các tác vụ lặp lại (iterative jobs) và tác vụ tương tác (interactive queries). Nhờ kiến trúc thực thi trong bộ nhớ (in-memory computing) và cơ chế khôi phục lỗi hiệu quả, Spark ban đầu đạt tốc độ xử lý nhanh hơn MapReduce từ 10 đến 20 lần và nhanh chóng được ứng dụng trong các dự án thực tế như Mobile Millennium, là một hệ thống dự đoán ùn tắc giao thông theo thời gian thực.

Từ một dự án nội bộ, Spark thu hút sự quan tâm rộng rãi của cộng đồng nghiên cứu và doanh nghiệp. Đến năm 2010, Spark được công bố mã nguồn mở, và vào năm 2013 chính thức trở thành dự án của Apache Software Foundation. Phiên bản ổn định đầu tiên, Spark 1.0, ra mắt vào tháng 5/2014, đánh dấu bước trưởng thành quan trọng của nền tảng này. Sự đóng góp từ hơn 50 tổ chức lớn như Databricks, Yahoo! và Intel đã thúc đẩy Spark phát triển thành một hệ sinh thái mạnh mẽ và ngày càng phổ biến trong xử lý dữ liệu lớn.

Điểm nổi bật của Spark nằm ở khả năng xử lý tính toán phân tán tốc độ cao, nhờ thực thi song song trên nhiều máy khác nhau trong cụm (cluster). Spark hỗ trợ lưu trữ dữ liệu và tính toán trực tiếp trong RAM, giúp giảm đáng kể độ trễ I/O so với các hệ thống truyền thống dựa nhiều vào việc đọc ghi đĩa. Không chỉ tối ưu cho xử lý trong bộ nhớ, Spark còn cải thiện hiệu năng đáng kể ngay cả khi xử lý dữ liệu trên đĩa, giúp nó vượt trội hơn MapReduce trong hầu hết các ứng dụng phức tạp.

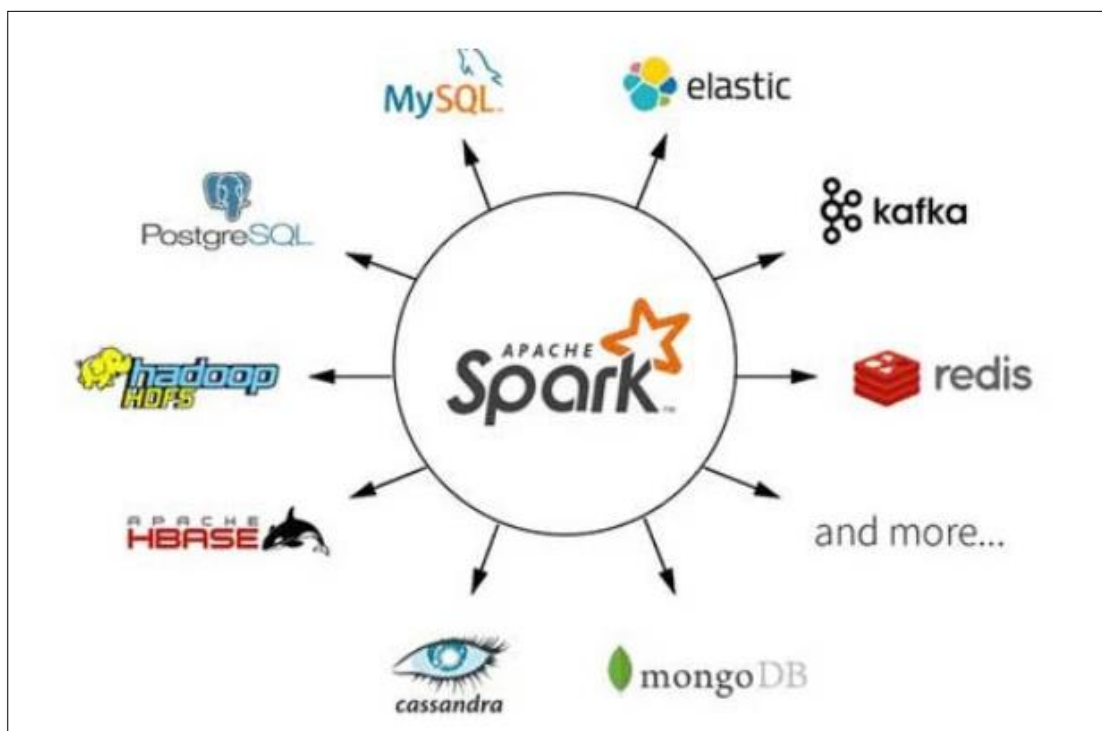
Bên cạnh tốc độ, Spark còn là một nền tảng đa năng. Nó hỗ trợ nhiều loại tác vụ xử lý dữ liệu khác nhau như xử lý hàng loạt (batch processing), phân tích tương tác (interactive analysis), xử lý dữ liệu thời gian thực (Spark Streaming) và các thuật toán học máy lặp (iterative machine learning). Tất cả đều được tích hợp trong cùng một hệ sinh thái với các thư viện mạnh mẽ như Spark SQL, MLlib, GraphX, giúp đơn giản hóa việc kết hợp các mô hình xử lý mà trước đây phải dùng nhiều hệ thống độc lập. Spark có khả năng xử lý lỗi và khả năng mở rộng tốt, cho phép tăng cường khả năng chịu tải khi cần thiết.



Hình 1: Các đặc trưng của Apache Spark

Spark tương thích với nhiều hệ thống lưu trữ thông qua API của Hadoop, bao gồm HDFS, Hive, HBase, Cassandra, Amazon S3 và nhiều định dạng dữ liệu như text, CSV, JSON, Avro, Parquet. Điều quan trọng là Spark hoàn toàn không phụ thuộc vào bất kỳ hệ thống file nào, cho phép triển khai linh hoạt từ máy cá nhân đến cụm hàng nghìn nút. Về khả năng sử dụng, Spark cung cấp API thân thiện cho các ngôn ngữ phổ biến như Python, Scala, Java và R. Điều này giúp lập trình viên dễ dàng xây dựng, kiểm thử và mở rộng ứng dụng từ quy mô

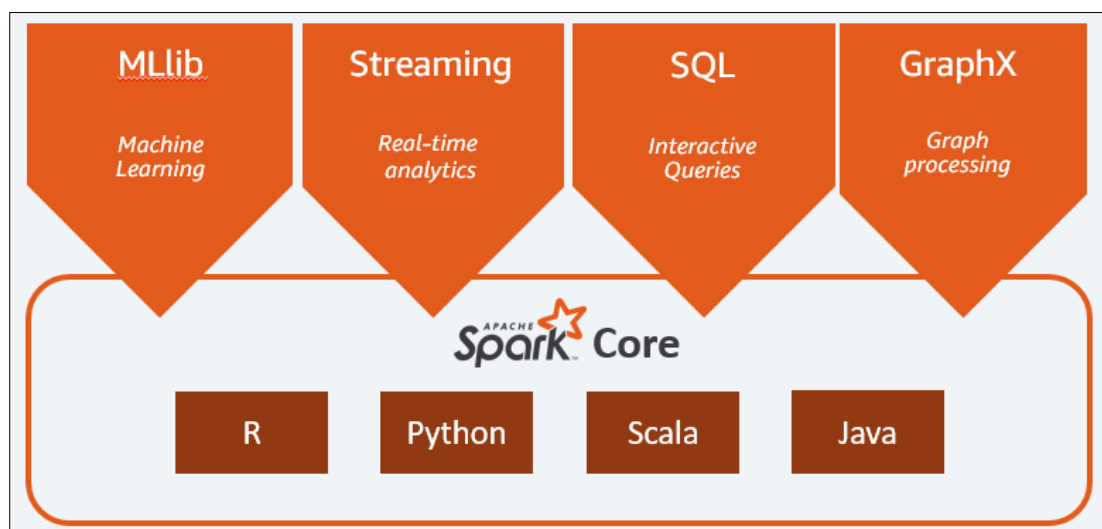
nhỏ đến rất lớn. Spark MLlib – thư viện học máy tích hợp sẵn – hỗ trợ nhiều thuật toán quan trọng như Decision Tree, Random Forest, KMeans..., tạo điều kiện thuận lợi cho việc triển khai các mô hình học máy trên dữ liệu lớn (Big Data) trong môi trường phân tán.



Hình 2: Spark tương thích với nhiều hệ thống lưu trữ

Tổng thể, Apache Spark là một nền tảng xử lý dữ liệu lớn mạnh mẽ, hiệu quả và linh hoạt, được thiết kế để đáp ứng nhu cầu tính toán hiện đại, từ xử lý thời gian thực đến các thuật toán học máy phân tán, hỗ trợ doanh nghiệp và tổ chức khai thác tối đa giá trị của dữ liệu.

Apache Spark gồm có 5 thành phần chính:

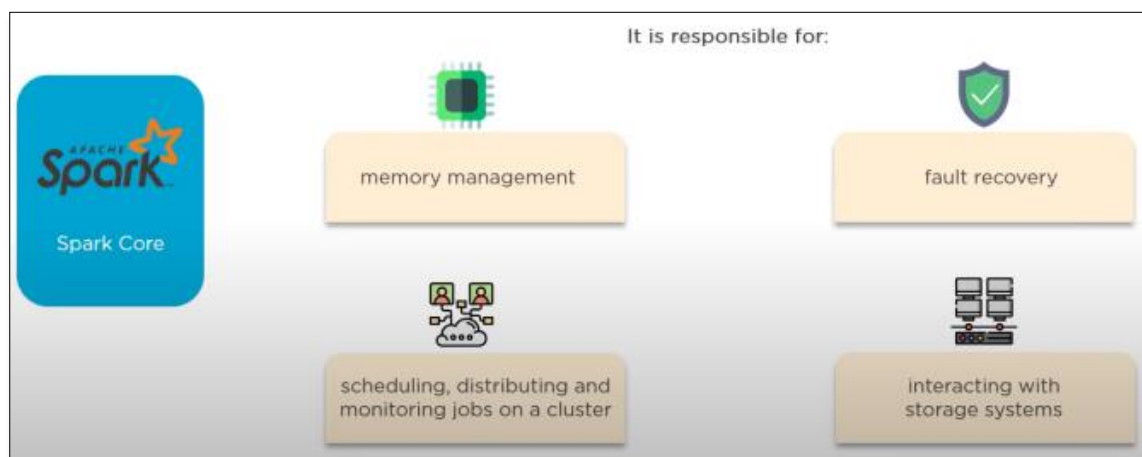


Hình 3: Các thành phần của Apache Spark

1.1. Các thành phần chính của Apache Spark:

1.1.1. Spark Core - nền tảng của Spark:

Spark Core là yếu tố nền tảng giúp các thành phần còn lại có thể vận hành trong Apache Spark. Spark Core đảm nhận việc thực hiện các tính toán, xử lý trong bộ nhớ (Inmemory computing). Đồng thời, thành phần này còn tham chiếu các dữ liệu lưu trữ ở bộ lưu trữ bên ngoài, phục hồi sau lỗi, lên lịch, phân phối và giám sát các công việc và tương tác với các hệ thống lưu trữ. Spark Core giới thiệu RDD (Resilient Distributed Datasets) là khái niệm cốt lõi cho phép xử lý dữ liệu song song trên nhiều máy.



Hình 4: Nhiệm vụ của Spark Core

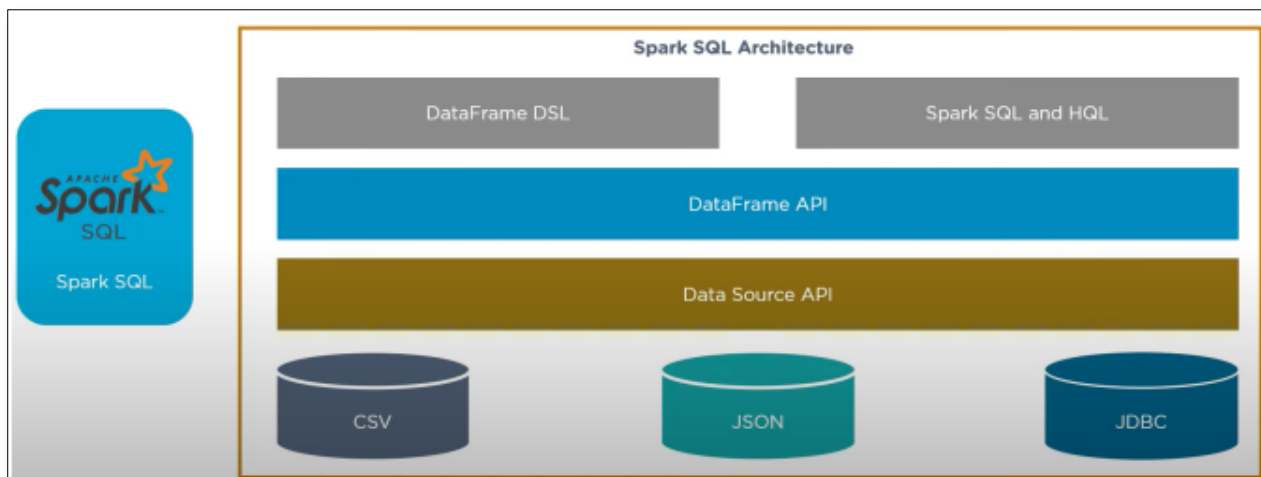
Resilient Distributed Datasets – RDD (Bộ dữ liệu phân tán có khả năng phục hồi) là một khái niệm quan trọng trong Spark Core được sử dụng để lưu trữ và xử lý dữ liệu trên một cụm máy tính phân tán. RDD cung cấp khả năng chịu lỗi thông qua thông tin dòng, cho phép tính toán lại dữ liệu bị mất từ dữ liệu nguồn, đồng thời có thể chứa bất kỳ kiểu dữ liệu nào của Python, Java, hoặc đối tượng Scala, bao gồm các kiểu dữ liệu do người dùng định nghĩa. Thông thường, RDD chỉ cho phép đọc, phân mục tập hợp của các bản ghi. RDD có thể được tạo ra qua điều khiển xác định trên dữ liệu trong bộ nhớ hoặc RDD. RDD là một tập hợp có khả năng chịu lỗi, mỗi thành phần có thể được tính toán song song.

Các đặc trưng của RDD gồm:

- **Phân phối dữ liệu:** RDD chia dữ liệu thành các phần nhỏ và phân tán chúng trên các nút trong cụm. Mỗi phần dữ liệu được lưu trữ trên một nút và có thể xử lý một cách độc lập.
- **Khả năng chịu lỗi và phục hồi:** RDD có khả năng tự phục hồi khi có lỗi xảy ra trên một phần dữ liệu. Điều này đạt được bằng cách lưu trạng thái gốc và sử dụng dữ liệu gốc để tái tính toán dữ liệu bị mất. Ngoài ra, RDD là không thay đổi, nghĩa là sau khi tạo ra một RDD, người dùng không thể thay đổi nó. Thay vào đó, người dùng tạo các RDD mới từ RDD gốc bằng cách sử dụng các phép biến đổi.
- **Lazy evaluation:** RDD sử dụng cơ chế Lazy evaluation, cho phép xây dựng và tối ưu các phân tích dữ liệu phức tạp.
- **In-memory caching:** RDD có khả năng lưu trữ dữ liệu trong bộ nhớ, giúp tối ưu hiệu suất xử lý dữ liệu. Khi dữ liệu đã được lưu trữ trong bộ nhớ, các phép tính toán sau này trên RDD có thể được thực hiện nhanh hơn vì không cần đọc dữ liệu từ đĩa.

1.1.2. Spark SQL - truy vấn tương tác:

Spark SQL là một module trong hệ sinh thái Apache Spark cung cấp giao diện lập trình để xử lý dữ liệu có cấu trúc và bán cấu trúc bằng SQL (Ngôn ngữ truy vấn có cấu trúc). Nó cho phép người dùng tích hợp liền mạch các truy vấn SQL với các ứng dụng PySpark của họ, giúp phân tích và thao tác dữ liệu có cấu trúc trong khung điện toán phân tán dễ dàng hơn. PySpark SQL là một công cụ có giá trị dành cho các kỹ sư dữ liệu và nhà khoa học làm việc với các bộ dữ liệu có cấu trúc, quy mô lớn và được sử dụng rộng rãi để khám phá dữ liệu, truy vấn và thực hiện các tác vụ ETL.



Hình 5: Kiến trúc của Spark SQL

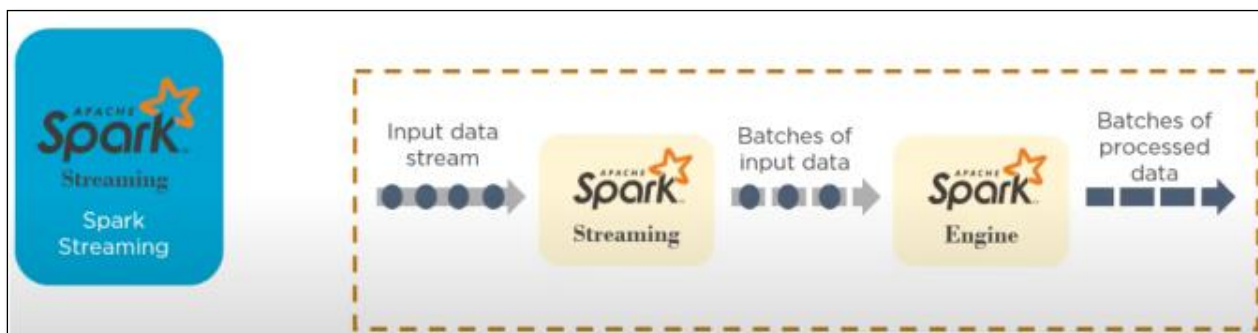
Các đặc trưng của Spark SQL gồm:

- **Hỗ trợ SQL truyền thống:** Spark SQL cho phép thực hiện truy vấn SQL truyền thống trên dữ liệu phân tán. Có thể sử dụng các truy vấn SELECT, JOIN, GROUP BY, và nhiều lệnh SQL khác để truy vấn và xử lý dữ liệu.
- **Tích hợp dữ liệu cấu trúc và bán cấu trúc:** Spark SQL cho phép làm việc với dữ liệu cấu trúc (Structured Data) thông qua các DataFrame, cũng như dữ liệu bán cấu trúc (Semi-structured Data) bằng cách sử dụng các RDD.
- **Tương thích với nhiều nguồn dữ liệu:** Spark SQL có khả năng làm việc với nhiều nguồn dữ liệu khác nhau, bao gồm Parquet, JSON, CSV và cơ sở dữ liệu HDFS, Hive. Spark SQL cung cấp một cơ chế để tích hợp với nhiều hệ thống lưu trữ dữ liệu khác như MySQL, Oracle hoặc PostgreSQL và xử lý dữ liệu bằng cách sử dụng các tính năng phân tán của Spark.
- **Catalyst Optimizer:** Spark SQL sử dụng Catalyst Optimizer để tối ưu hóa các truy vấn SQL. Catalyst là một bộ tối ưu hóa dựa trên quy tắc (rule-based optimizer) giúp cải thiện hiệu suất truy vấn.
- **Hỗ trợ cho UDFs (User-Defined Functions):** Có thể định nghĩa và sử dụng các hàm người dùng (UDFs) trong truy vấn SQL để thực hiện tính toán tùy chỉnh trên dữ liệu.
- **Lập trình bằng nhiều ngôn ngữ:** Spark SQL hỗ trợ nhiều ngôn ngữ lập trình, bao gồm Scala, Java, Python và R.
- **Hỗ trợ cho BI Tools:** Spark SQL có thể tích hợp với các công cụ Business Intelligence (BI) như Tableau, QlikView, và Power BI để trực quan hóa và trình bày dữ liệu.

- Hỗ trợ cho MLlib và GraphX: Dữ liệu có thể được trực tiếp chuyển đổi thành các đối tượng sử dụng trong thư viện MLlib và GraphX của Spark để thực hiện học máy và xử lý đồ thị.

1.1.3. Spark Streaming – xử lý thời gian thực:

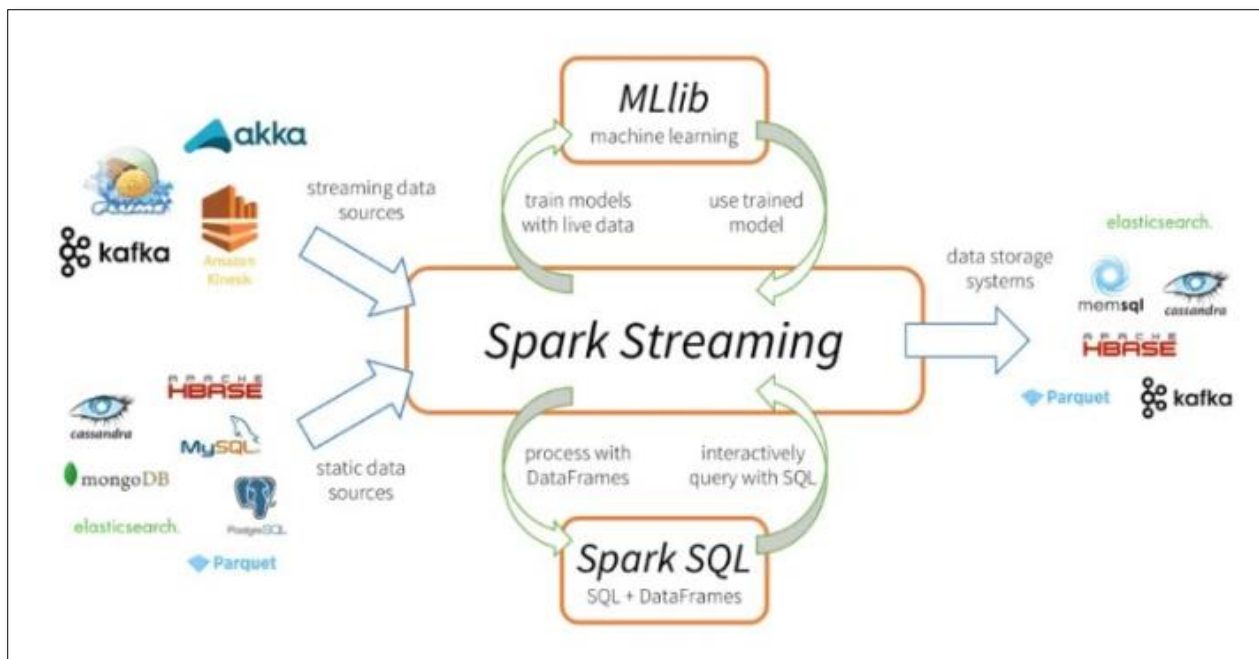
Spark Streaming là một module cho phép xử lý dữ liệu trực tiếp trong thời gian thực (real-time). Spark Streaming cung cấp một cơ chế xử lý dữ liệu liên tục bằng cách chia nhỏ dữ liệu thành một chuỗi các microbatch nhỏ hơn và xử lý chúng thông qua API Apache Spark. Spark Streaming được ứng dụng trong phân tích dữ liệu trực tuyến, phát hiện sự cố hệ thống và phân tích dữ liệu giám sát trong thời gian thực với tốc độ và khả năng mở rộng cao.



Hình 6: Cơ chế xử lý dữ liệu của Spark Streaming

Các đặc trưng của Spark Streaming gồm:

- Hỗ trợ cho các nguồn dữ liệu đa dạng: Xử lý dữ liệu trực tiếp từ nhiều nguồn như streaming data (logs, sự kiện, dữ liệu sensor), batch data, và hệ thống hàng đợi tin nhắn (message queues) như Apache Kafka.
- API đồng nhất với Spark: Tích hợp chặt chẽ với Apache Spark, giúp người dùng sử dụng cùng một API.
- Khả năng mở rộng: Có khả năng mở rộng ngang (horizontal scaling), cho phép xử lý dữ liệu để đối phó với tải công việc cao và dữ liệu đầu vào lớn.
- Near Real-Time: Mặc dù Spark Streaming không đạt đến thời gian thực (real-time) chính xác, nó cung cấp khả năng xử lý dữ liệu gần thời gian thực (near real-time) với độ trễ thấp.
- Tích hợp với thư viện Spark: Có thể sử dụng các thư viện khác của Spark như Spark SQL, MLlib, và GraphX để xử lý dữ liệu trực tiếp.
- Windowed Processing: Spark Streaming cho phép thực hiện xử lý dữ liệu trong cửa sổ thời gian, giúp tính toán tổng hợp và thống kê trên dữ liệu trong khoảng thời gian nhất định.



Hình 7: Xử lý dữ liệu với Spark Streaming và các thư viện của Spark

1.1.4. Spark MLlib - thư viện học máy:

Spark MLlib là thư viện Machine Learning mã nguồn mở được tích hợp sẵn trong Apache Spark, cung cấp các thuật toán và công cụ phổ biến để phát triển mô hình học máy trên dữ liệu phân tán, giúp ích trong việc xử lý dữ liệu lớn. MLlib cung cấp các thuật toán phổ biến như: Hồi quy, phân loại, phân cụm, lọc cộng tác, giảm chiều dữ liệu, trích xuất đặc trưng. MLlib được thiết kế để xử lý dữ liệu phân tán và song song, khiến nó trở nên lý tưởng để xử lý dữ liệu lớn. Đây là một công cụ có giá trị dành cho các nhà khoa học và kỹ sư dữ liệu làm việc trong các dự án dữ liệu lớn vì nó kết hợp sức mạnh của Apache Spark với khả năng học máy, cho phép xử lý dữ liệu hiệu quả và đào tạo mô hình trên quy mô lớn.



Hình 8: Các thuật toán học máy được tích hợp sẵn trong Spark MLlib

Các đặc trưng của Spark MLlib gồm:

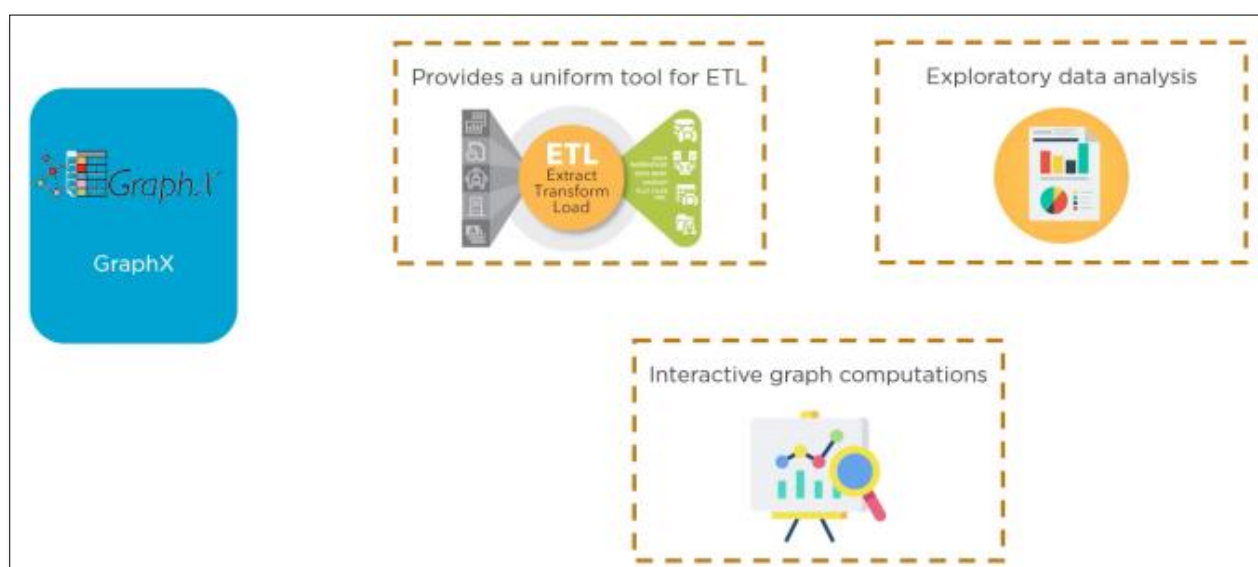
- Tích hợp với Apache Spark: Tích hợp chặt chẽ với Apache Spark, cho phép sử dụng ngôn ngữ Scala, Java, Python hoặc R để làm việc với dữ liệu và thực hiện huấn luyện

học máy trên cụm Spark. Điều này mang lại sự mạnh mẽ và khả năng mở rộng của Spark cho việc xử lý dữ liệu lớn.

- Mô hình Học máy: Cung cấp một loạt các mô hình học máy khác nhau: phân loại (classification), hồi quy (regression), phân cụm (clustering). Các mô hình này đã được tối ưu hóa để hoạt động trên cụm và xử lý dữ liệu phân tán.
- Các công cụ và tính năng liên quan đến học máy: Cung cấp các công cụ và tính năng liên quan đến học máy như đánh giá mô hình (model evaluation), data pipeline, model pipeline.
- Khả năng mở rộng: Có khả năng mở rộng và có thể xử lý cả dữ liệu lớn và cụm máy tính phân tán, cho phép đào tạo mô hình trên dữ liệu lớn và tận dụng các tài nguyên cụm.
- Hỗ trợ cho Spark Streaming và Spark SQL: Tích hợp với Spark Streaming và Spark SQL, cho phép xây dựng ứng dụng thực hiện huấn luyện học máy trên dữ liệu trực tiếp và dữ liệu cấu trúc.
- Sự phát triển và cộng đồng mạnh mẽ: Là một dự án mã nguồn mở phát triển mạnh mẽ và có sự đóng góp từ cộng đồng rộng rãi. Điều này đảm bảo tính cập nhật, sửa lỗi và bổ sung mới liên tục.

1.1.5. Spark GraphX – xử lý đồ thị:

GraphX là một thư viện mã nguồn mở trong hệ sinh thái Apache Spark, được thiết kế để xử lý và phân tích dữ liệu đồ thị. Thư viện này cung cấp các cấu trúc dữ liệu và các thuật toán đồ thị để giúp người dùng nghiên cứu, phân tích mạng xã hội, xử lý dữ liệu đồ thị trong lĩnh vực khai thác dữ liệu và học máy. GraphX còn cung cấp ETL, phân tích thăm dò và điện toán đồ thị lặp lại để giúp người dùng xây dựng một cách tương tác và chuyển đổi cấu trúc dữ liệu đồ thị ở quy mô lớn.



Hình 9: GraphX cung cấp nhiều tính năng xử lý đồ thị

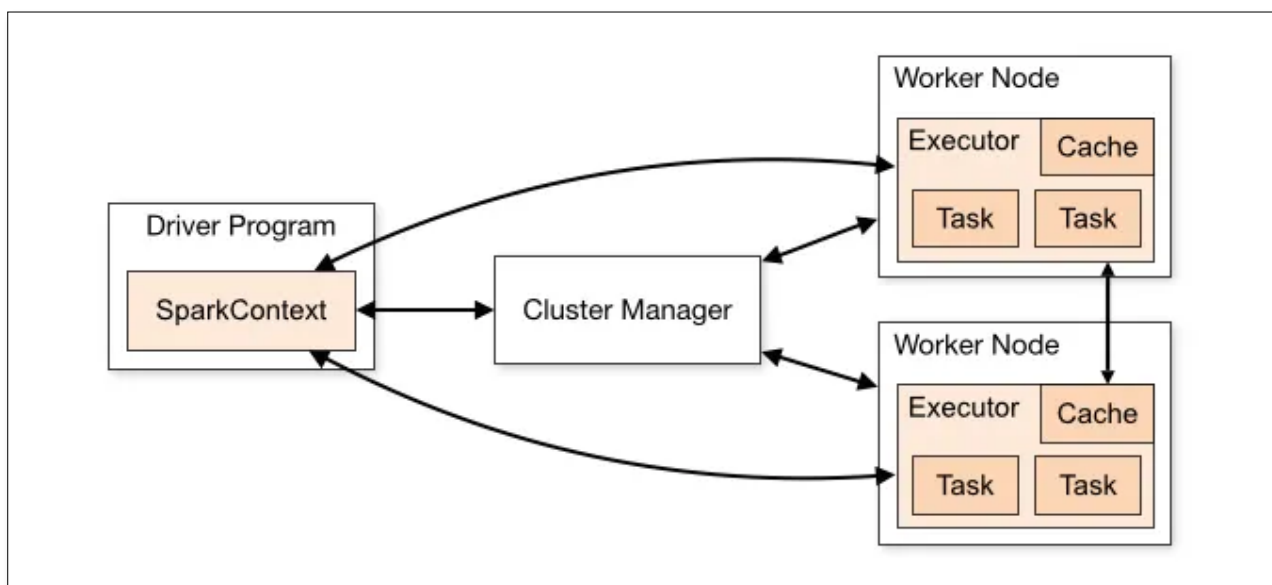
Các đặc trưng của Spark GraphX gồm:

- Xử lý dữ liệu đồ thị: Xử lý dữ liệu đồ thị, trong đó các đỉnh và cạnh của đồ thị có thể đại diện cho các thực thể và mối quan hệ giữa chúng. Điều này phù hợp cho việc nghiên cứu các mạng xã hội, mạng lưới, mạng thông tin, và nhiều ứng dụng khác.

- API đồng nhất với Spark: Tích hợp chặt chẽ với Apache Spark, giúp sử dụng cùng một API để xử lý dữ liệu đồ thị và dữ liệu lớn. Điều này giúp tận dụng sức mạnh của Spark trong việc xử lý phân tán.
- Cấu trúc dữ liệu linh hoạt: Cung cấp cấu trúc dữ liệu linh hoạt để biểu diễn dữ liệu đồ thị, bao gồm đồ thị hướng và đồ thị không hướng. Điều này cho phép biểu diễn một loạt các dữ liệu đồ thị phức tạp.
- Các thuật toán đồ thị tích hợp: Cung cấp một loạt các thuật toán đồ thị như duyệt đồ thị (graph traversal), tính toán đường đi ngắn nhất (shortest path), tìm kiếm đồ thị (graph search), tính toán bậc và thuật toán phân cụm (clustering algorithms) để phân tích và khám phá thông tin từ dữ liệu đồ thị.
- Khả năng mở rộng: Có khả năng mở rộng và tối ưu hóa hiệu suất, cho phép xử lý các đồ thị lớn trên cụm máy tính phân tán.
- Hỗ trợ cho tích hợp với MLlib: GraphX có thể tích hợp với Spark MLlib, cho phép sử dụng học máy để phân tích dữ liệu đồ thị và xây dựng các mô hình học máy dựa trên thông tin từ đồ thị.
- Hỗ trợ cho Spark Streaming và Spark SQL: GraphX có thể được tích hợp với Spark Streaming và Spark SQL để xử lý và phân tích dữ liệu đồ thị trong các ứng dụng thời gian thực và các truy vấn SQL.

1.2. Kiến trúc của Apache Spark:

Apache Spark hoạt động dựa trên kiến trúc master-slave (chủ-tớ). Kiến trúc này bao gồm một Driver (đóng vai trò là nút chủ - master node) và nhiều Executors (đóng vai trò là các nút thợ - worker nodes) được phân tán trên một cụm. Khung làm việc này hỗ trợ cả xử lý hàng loạt và xử lý thời gian thực.



Hình 10: Kiến trúc hoạt động của Apache Spark

1.2.1. Các thành phần chính:

Kiến trúc này dựa vào một số thành phần riêng biệt để phối hợp và thực thi các tác vụ:

- **Spark Driver:** Đây là quy trình hoặc chương trình chịu trách nhiệm điều phối việc thực thi ứng dụng. Nó chạy hàm main và tạo SparkContext. Bên trong, driver chứa các thành phần như DAG Scheduler (Bộ lập lịch DAG), Task Scheduler (Bộ lập lịch tác vụ), Backend Scheduler và Block Manager để chuyển mã người dùng thành các công việc có thể thực thi.
- **SparkContext:** Đóng vai trò là điểm đầu vào cho các chức năng của Spark, thành phần này kết nối với cụm và được sử dụng để tạo RDD, biến tích lũy (accumulators) và biến quảng bá (broadcast variables). Nó phối hợp với Spark Driver để giám sát việc thực thi công việc.
- **Spark Executors:** Đây là các quy trình thợ (worker processes) được khởi chạy trên các nút thợ. Chúng chịu trách nhiệm thực thi các tác vụ đồng thời và giao tiếp với driver cũng như trình quản lý cụm. Chúng cũng xử lý việc lưu trữ dữ liệu trong bộ nhớ hoặc trên đĩa để lưu bộ nhớ đệm (caching).
- **Cluster Manager (Trình quản lý cụm):** Thành phần này quản lý cụm và chịu trách nhiệm phân bổ tài nguyên cho ứng dụng.
- **Task (Tác vụ):** Đơn vị công việc nhỏ nhất, đại diện cho một đơn vị tính toán được thực hiện trên một phân vùng dữ liệu duy nhất.

1.2.2. Cách thức hoạt động của Spark:

Luồng thực thi bao gồm sự phối hợp giữa Driver, Cluster Manager và các Worker:

1. Khởi tạo: Chương trình Driver chạy chương trình thực của ứng dụng và tạo SparkContext.
2. Phân bổ tài nguyên: Spark Driver làm việc với Cluster Manager để phân bổ các tài nguyên cần thiết cho công việc.
3. Chia nhỏ công việc: Công việc được chia thành các công việc nhỏ hơn và các tác vụ. SparkContext nhận thông tin tác vụ từ Cluster Manager.
4. Thực thi: Các tác vụ được đưa vào hàng đợi trên các nút thợ. Các Executor thực hiện các nhiệm vụ này, xử lý RDD và có thể lưu kết quả vào bộ nhớ đệm. Vòng đời của một executor khớp với vòng đời của Ứng dụng Spark.

1.2.3. Các trừu tượng chính (Main Abstractions):

Kiến trúc Spark được thiết kế xoay quanh hai sự trừu tượng hóa chính:

- **Resilient Distributed Dataset (RDD):** RDD là một tập hợp các phần tử cơ bản, bất biến (chỉ đọc), có thể được xử lý song song trên nhiều thiết bị. Mỗi tập dữ liệu được chia thành các phần logic được thực thi trên các nút khác nhau của cụm.
- **Directed Acyclic Graph (DAG):** Đây là lớp lập lịch thực hiện việc lập lịch theo định hướng giai đoạn. Không giống như MapReduce bị giới hạn trong hai giai đoạn (Map và Reduce), DAG của Spark có thể chứa nhiều giai đoạn.

1.2.4. Trình quản lý Cụm và Chế độ Thực thi:

Spark hỗ trợ nhiều trình quản lý cụm khác nhau, bao gồm Standalone (thiết lập đơn giản), Apache Mesos (trình quản lý chung), Hadoop YARN (trình quản lý tài nguyên Hadoop 2) và Kubernetes (quản lý container).

Các ứng dụng có thể được chạy ở ba chế độ khác nhau:

- **Cluster Mode (Chế độ cụm):** Chế độ phổ biến nhất, trong đó trình quản lý cụm khởi chạy quy trình driver trên một nút thợ bên trong cụm, duy trì tất cả các quy trình liên quan đến ứng dụng.
- **Client Mode (Chế độ máy khách):** Tương tự như chế độ cụm, nhưng driver Spark vẫn chạy trên máy khách đã gửi ứng dụng, trong khi trình quản lý cụm duy trì các executor.
- **Local Mode (Chế độ cục bộ):** Toàn bộ ứng dụng chạy trên một máy duy nhất, đạt được sự song song thông qua các luồng (threads). Chế độ này chủ yếu dùng để học tập, kiểm tra và phát triển lặp lại.

1.3. Sự khác biệt của Spark so với Hadoop:

Hadoop MapReduce là một mô hình lập trình để xử lý các tập dữ liệu lớn bằng thuật toán phân tán song song. Các nhà phát triển có thể viết toán tử song song hàng loạt mà không phải lo lắng về phân phối công việc và khả năng chịu lỗi. Tuy nhiên, một thách thức đối với MapReduce là quy trình nhiều bước tuần tự cần thiết để chạy một công việc. Với mỗi bước, MapReduce đọc dữ liệu từ cụm, thực hiện các thao tác và ghi lại kết quả vào HDFS. Bởi vì mỗi bước đều yêu cầu đọc đĩa và ghi, MapReduce hoàn thành công việc chậm hơn do độ trễ của I/O đĩa.

Spark được tạo ra để giải quyết các hạn chế của MapReduce bằng cách thực hiện xử lý trong bộ nhớ, giảm số bước trong một công việc và bằng cách sử dụng lại dữ liệu trên nhiều thao tác song song. Với Spark, chỉ cần thực hiện một bước là dữ liệu được đọc vào bộ nhớ, các thao tác được thực hiện và kết quả được ghi lại, từ đó giúp việc thực thi nhanh hơn nhiều. Spark cũng sử dụng lại dữ liệu bằng cách sử dụng bộ nhớ đệm nằm trong bộ nhớ để tăng tốc đáng kể các thuật toán máy học liên tục gọi một hàm trên cùng một tập dữ liệu. Việc tái sử dụng dữ liệu được thực hiện thông qua việc tạo DataFrames, một khung trừu tượng trên tập dữ liệu phân tán linh hoạt (RDD), là một tập hợp các đối tượng được lưu trong bộ nhớ đệm và được tái sử dụng trong nhiều hoạt động của Spark. Điều này làm giảm đáng kể độ trễ khiến Spark có tốc độ nhanh hơn nhiều lần so với MapReduce, đặc biệt là khi thực hiện máy học và phân tích tương tác.

Đặc trưng	Hadoop	Spark
Tốc độ xử lý	Chậm hơn. Xử lý dựa trên đĩa cứng (Disk-based).	Nhanh hơn rất nhiều. Nhanh hơn 100 lần khi chạy trên RAM và 9 lần khi chạy trên ổ cứng.
Mô hình xử lý	Xử lý theo lô (Batch processing).	Hỗ trợ cả xử lý theo lô (Batch) và xử lý luồng thời gian thực (Real-time Streaming).
Lưu trữ dữ liệu	Có hệ thống tệp gốc riêng là HDFS (Hadoop Distributed File System), chia dữ liệu thành nhiều khối.	Không có hệ thống tệp gốc riêng. Nó phải dựa vào hệ thống khác (như HDFS, S3, v.v.) để lưu trữ.
Học máy	Không có thư viện tích hợp sẵn, phải dùng công cụ bên ngoài.	Tích hợp sẵn thư viện MLlib mạnh mẽ cho các thuật toán học máy.

Hỗ trợ ngôn ngữ lập trình	Chủ yếu hỗ trợ tốt nhất cho Java.	Cung cấp API đa dạng, dễ sử dụng cho nhiều ngôn ngữ: Java, Scala, Python, R.
Bảo mật	Có tính năng bảo vệ dữ liệu mạnh mẽ hơn (Kerberos, ACLs...).	Mô hình bảo mật mặc định yếu hơn Hadoop, thường dựa vào bảo mật của hệ thống file bên dưới.
Chi phí phần cứng	Thấp hơn. Chạy tốt trên phần cứng thông thường, lưu trữ trên đĩa rẻ tiền.	Cao hơn. Yêu cầu nhiều RAM (bộ nhớ trong) để xử lý in-memory, mà RAM thì đắt hơn ổ cứng.
Khả năng mở rộng (Scale)	Dễ dàng thêm nút (node) mới với chi phí hợp lý.	Có thể mở rộng nhưng chi phí cao hơn do yêu cầu cấu hình RAM lớn cho mỗi nút.

Bảng 1: So sánh Hadoop và Spark

Ngoài sự khác biệt trong thiết kế của Spark và Hadoop, hiện nay nhiều tổ chức và doanh nghiệp đã nhận thấy, ứng dụng các framework dữ liệu lớn này có thể bổ sung cho nhau, sử dụng cùng nhau để giải quyết vấn đề rộng lớn hơn trong kinh doanh.

1.4. Ưu điểm và nhược điểm của Apache Spark:

1.4.1. Ưu điểm của Apache Spark:

Tốc độ và hiệu suất vượt trội: Spark có khả năng xử lý nhanh hơn hẳn so với Hadoop MapReduce (có thể lên tới 100 lần trong một số điều kiện) nhờ vào việc lưu giữ và xử lý dữ liệu ngay trong bộ nhớ (RAM) thay vì phải đọc/ghi liên tục xuống ổ đĩa. Điều này đặc biệt hữu ích cho các thuật toán lặp trong học máy và tính toán đồ thị.

Hỗ trợ đa ngôn ngữ: Mặc dù được viết bằng Scala, Spark hỗ trợ mạnh mẽ Java, Python (thông qua PySpark) và R. Điều này giúp giảm rào cản gia nhập cho các lập trình viên và cho phép các nhóm đa chức năng (ví dụ: Khoa học dữ liệu dùng Python, Kỹ sư dữ liệu dùng Java/Scala) cùng làm việc hiệu quả trên một nền tảng.

Giao diện người dùng Web tích hợp (Spark UI): Spark cung cấp một giao diện web (Spark UI) tiện lợi giúp giám sát trạng thái cụm và mức tiêu thụ tài nguyên. Các tab chi tiết như Jobs (công việc), Stages (giai đoạn), và Executors (bộ thực thi) cho phép người dùng theo dõi tiến độ, xem trực quan hóa DAG (đồ thị không chu trình có hướng) và gỡ lỗi hoặc tối ưu hóa ứng dụng dễ dàng.

Linh hoạt và khả năng tương thích cao: Spark có thể chạy trên nhiều trình quản lý cụm khác nhau (như Kubernetes hoặc chế độ Standalone) và xử lý dữ liệu từ đa dạng nguồn (HDFS, Cassandra, Amazon S3). Nó cũng tích hợp tốt với các thư viện học máy (TensorFlow, PyTorch), các công cụ BI (Tableau, Looker) và nhiều hệ thống cơ sở dữ liệu hiện đại.

Khả năng phân tích nâng cao: Spark tích hợp sẵn một bộ thư viện toàn diện gồm Spark SQL (xử lý dữ liệu có cấu trúc), MLlib (học máy), GraphX (xử lý đồ thị) và Spark Streaming.

Điều này cho phép Kỹ sư và Khoa học dữ liệu thực hiện các tác vụ phức tạp trên một nền tảng thống nhất mà không cần tích hợp nhiều công cụ rời rạc.

Tài liệu chi tiết: Spark sở hữu hệ thống tài liệu hướng dẫn rất đầy đủ và toàn diện, bao gồm kiến trúc, API và thư viện. Các bài hướng dẫn và ví dụ cụ thể giúp lập trình viên ở mọi trình độ dễ dàng tiếp cận, gỡ lỗi và hiểu sâu về công nghệ.

Cộng đồng đa dạng và lớn mạnh: Là một nền tảng mã nguồn mở, Spark được hậu thuẫn bởi một cộng đồng khổng lồ. Với hàng nghìn người đóng góp trên GitHub và hàng chục nghìn câu hỏi trên Stack Overflow, người dùng có thể dễ dàng tìm kiếm sự hỗ trợ, kiến thức chia sẻ và giải pháp khắc phục sự cố.

1.4.2. Nhược điểm của Apache Spark:

Tiêu thụ bộ nhớ cao và chi phí phần cứng gia tăng: Apache Spark là một hệ thống tiêu tốn nhiều tài nguyên, đặc biệt là RAM, do thiết kế tập trung vào tính toán trong bộ nhớ (in-memory) để đạt tốc độ xử lý cao. Việc phụ thuộc nặng nề vào RAM dẫn đến chi phí vận hành cao hơn, giới hạn khả năng mở rộng và đòi hỏi phần cứng đắt tiền hơn (nhiều RAM và CPU nhanh hơn) so với các giải pháp dựa trên đĩa như Hadoop MapReduce, từ đó làm tăng chi phí thiết lập ban đầu.

Hỗ trợ hạn chế cho xử lý thời gian thực (Real-time): Mặc dù được biết đến với khả năng xử lý gần thời gian thực (near-real-time), Apache Spark không có khả năng xử lý thời gian thực thực sự. Nguyên nhân là do Spark Streaming dựa vào các vi lô (micro-batches) để xử lý các nhóm sự kiện nhỏ trong một khoảng thời gian định trước, do đó các công cụ như Apache Flink hoặc Apache Storm sẽ phù hợp hơn cho các nhu cầu phân tích luồng thực tế.

Khó khăn khi mới tập sử dụng: Mặc dù các khái niệm cơ bản như DataFrames khá đơn giản, người dùng thường gặp khó khăn để thực sự hiểu và tận dụng hết khả năng của Spark. Các khái niệm cốt lõi như lưu trữ phân tán, xử lý trong bộ nhớ, chi phí trao đổi dữ liệu và định dạng cột tạo nên rào cản lớn cho người mới bắt đầu, mặc dù việc có kiến thức nền tảng về Python (PySpark) có thể giúp quá trình học dễ dàng hơn.

Vấn đề với tệp nhỏ (Small files problem): Apache Spark gặp khó khăn khi phải xử lý một số lượng lớn các tệp nhỏ. Nhiều tệp trong một khối lượng công việc đồng nghĩa với việc hệ thống phải phân tích nhiều siêu dữ liệu (metadata) hơn và lập lịch cho nhiều tác vụ hơn, điều này có thể làm chậm quá trình xử lý một cách đáng kể.

Phụ thuộc vào hệ thống lưu trữ bên ngoài: Spark không có hệ thống lưu trữ tích hợp sẵn mà được thiết kế để giao tiếp với các giải pháp bên ngoài như HDFS, Amazon S3, hoặc Apache Cassandra. Sự phụ thuộc này làm tăng thêm độ phức tạp khi tích hợp Spark vào đường ống dữ liệu (data pipeline) và có thể gây ra các vấn đề về tương thích hoặc quản lý dữ liệu với các kho lưu trữ của bên thứ ba, đòi hỏi thêm thời gian và công sức để giải quyết.

1.5. Ứng dụng của Apache Spark:

Apache Spark là một framework đa năng trong thế giới Big Data vì nó làm được hầu hết mọi việc liên quan đến dữ liệu lớn. Dưới đây là các ứng dụng cốt lõi của Spark trong thực tế:

1. Xử lý và làm sạch dữ liệu (ETL & Data Engineering): Đây là ứng dụng phổ biến nhất, chiếm khoảng 60-70% công việc của Spark.

Vấn đề: Các công ty có hàng Terabyte dữ liệu "rác" (log máy chủ, file CSV lỗi, dữ liệu chưa chuẩn hóa) nằm rải rác.

Giải pháp Spark: Dùng Spark để đọc hàng tỷ dòng log, lọc bỏ dòng lỗi, chuyển đổi định dạng (ví dụ từ CSV sang Parquet như bạn đã làm), và lưu vào kho dữ liệu sạch (Data Warehouse).

Ví dụ: Facebook dùng Spark để xử lý hàng Petabyte dữ liệu log hành vi người dùng mỗi ngày để chuẩn bị cho việc phân tích.

2. Học máy trên dữ liệu lớn (Machine Learning - MLlib):

Vấn đề: Các thư viện như Scikit-learn (Python) chỉ chạy trên 1 máy. Khi dữ liệu lớn hơn RAM của máy đó (ví dụ 500GB), Scikit-learn sẽ sập nguồn (Memory Error).

Giải pháp Spark: Spark MLlib phân tán dữ liệu ra 10, 20, hay 100 máy để cùng train một mô hình.

Ứng dụng thực tế:

- Hệ thống gợi ý (Recommendation System): Netflix/YouTube dùng Spark để huấn luyện mô hình gợi ý phim/video cho hàng tỷ người dùng.
- Phân loại khách hàng: Ngân hàng dùng Spark để phân nhóm triệu khách hàng nhằm chạy quảng cáo đúng mục tiêu.

3. Xử lý dữ liệu thời gian thực (Real-time Streaming):

Vấn đề: Cần xử lý dữ liệu ngay khi nó vừa sinh ra, độ trễ chỉ tính bằng mili-giây hoặc giây.

Giải pháp Spark (Spark Streaming): Nhận dữ liệu liên tục từ các nguồn như Kafka, xử lý và đẩy ra kết quả ngay lập tức.

Ví dụ:

- Phát hiện gian lận thẻ tín dụng: Khi bạn quẹt thẻ, Spark Streaming phân tích giao dịch đó trong tích tắc để xem có bất thường không (dựa trên địa điểm, số tiền) trước khi ngân hàng duyệt lệnh.
- Grab/Uber: Tính toán giá cước (Surge pricing) theo thời gian thực dựa trên nhu cầu đặt xe và số lượng tài xế hiện có tại khu vực đó.

4. Phân tích đồ thị (Graph Processing - GraphX):

Vấn đề: Phân tích mối quan hệ chằng chịt giữa các đối tượng.

Giải pháp Spark: Spark GraphX giúp tính toán các thuật toán đồ thị trên quy mô lớn.

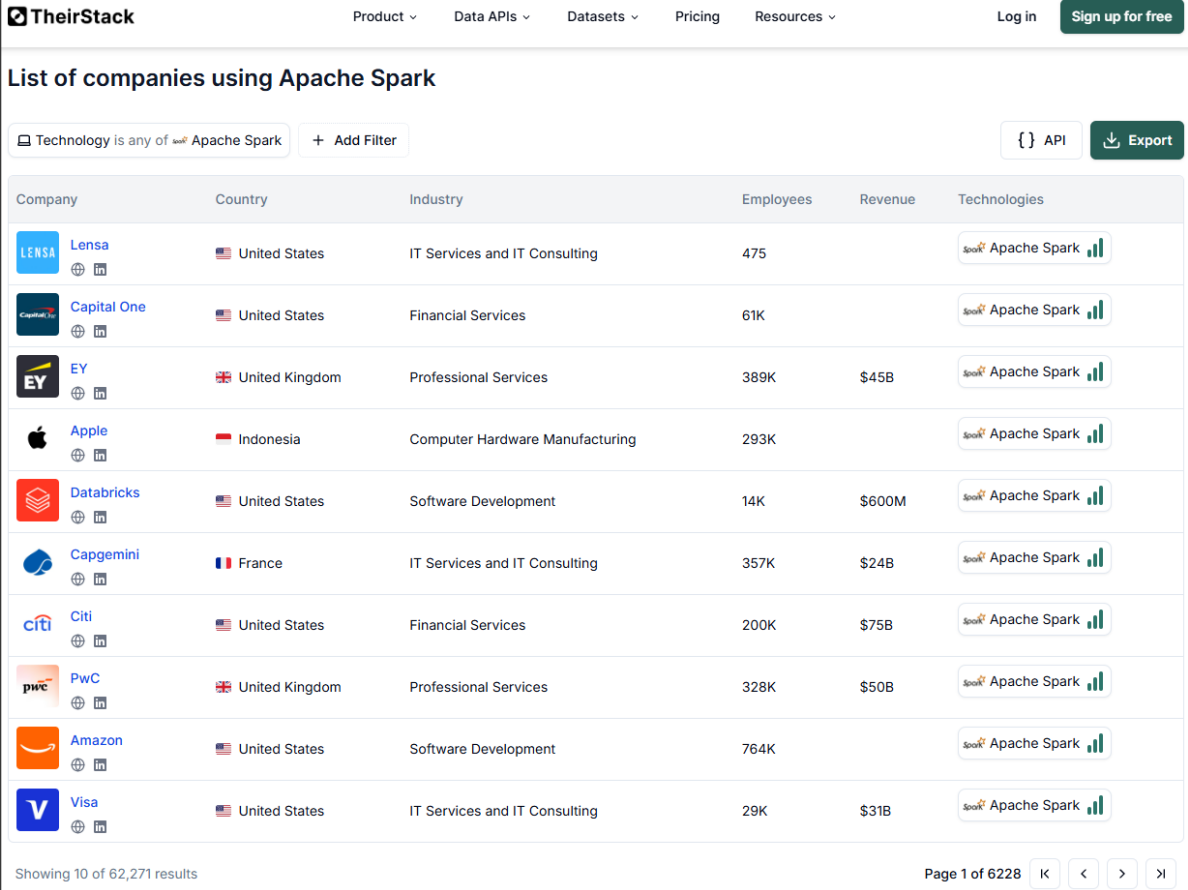
Ví dụ:

- Mạng xã hội: Tìm kiếm "bạn của bạn" (Friend suggestion) hoặc tìm những người có tầm ảnh hưởng (Influencer) trong cộng đồng.
- Google Maps: Tính toán đường đi ngắn nhất giữa các điểm nút giao thông trên quy mô toàn cầu.

5. Truy vấn tương tác (Interactive Analytics - Spark SQL):

Vấn đề: Các bên liên quan hoặc Business Analyst muốn xem báo cáo nhanh từ Big Data mà không biết code Java/Scala/Python.

Giải pháp: Họ dùng các công cụ BI (như Tableau, PowerBI) kết nối với Spark SQL. Spark sẽ chạy ngầm bên dưới để trả về kết quả cực nhanh.



The screenshot shows the 'List of companies using Apache Spark' page on the TheirStack website. The page features a table with columns: Company, Country, Industry, Employees, Revenue, and Technologies. The table lists 10 companies, each with a logo, name, country flag, industry, employee count, revenue, and a 'Spark' icon indicating their use of Apache Spark. The companies listed are Lensa, Capital One, EY, Apple, Databricks, Capgemini, Citi, PwC, Amazon, and Visa. The page also includes a search bar, filters, and pagination controls.

Company	Country	Industry	Employees	Revenue	Technologies
 Lensa	 United States	IT Services and IT Consulting	475		 Apache Spark 
 Capital One	 United States	Financial Services	61K		 Apache Spark 
 EY	 United Kingdom	Professional Services	389K	\$45B	 Apache Spark 
 Apple	 Indonesia	Computer Hardware Manufacturing	293K		 Apache Spark 
 Databricks	 United States	Software Development	14K	\$600M	 Apache Spark 
 Capgemini	 France	IT Services and IT Consulting	357K	\$24B	 Apache Spark 
 Citi	 United States	Financial Services	200K	\$75B	 Apache Spark 
 PwC	 United Kingdom	Professional Services	328K	\$50B	 Apache Spark 
 Amazon	 United States	Software Development	764K		 Apache Spark 
 Visa	 United States	IT Services and IT Consulting	29K	\$31B	 Apache Spark 

Hình 11: Hơn 62000 công ty đang sử dụng Apache Spark toàn thế giới

2. Cài đặt Apache Spark và triển khai cụm máy tính toán:

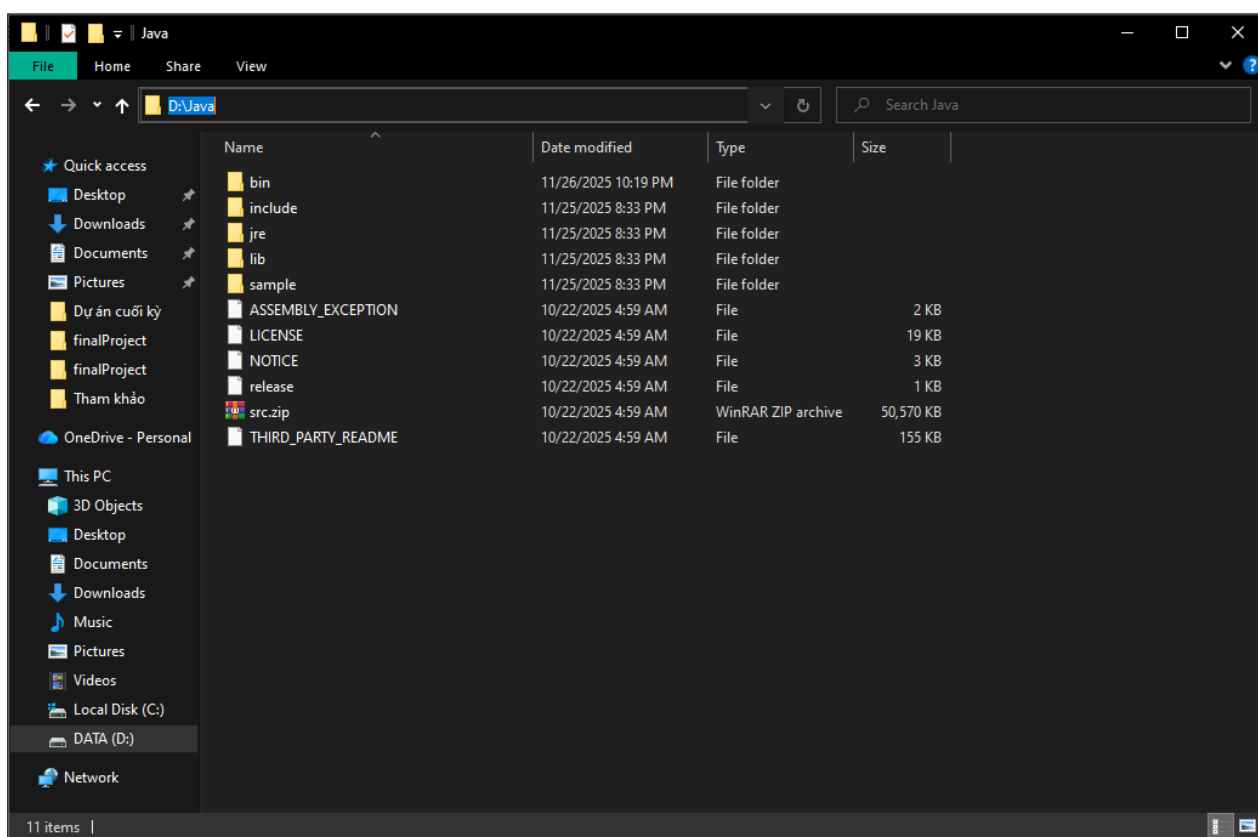
2.1. Cài đặt Apache Spark trên hệ điều hành Windows:

Việc cài đặt Apache Spark trên Windows đòi hỏi một số cấu hình đặc thù để giả lập môi trường file system của Linux (HDFS/POSIX). Quy trình cài đặt được chia thành các bước chi tiết như sau:

2.1.1. Chuẩn bị các phần mềm nền tảng:

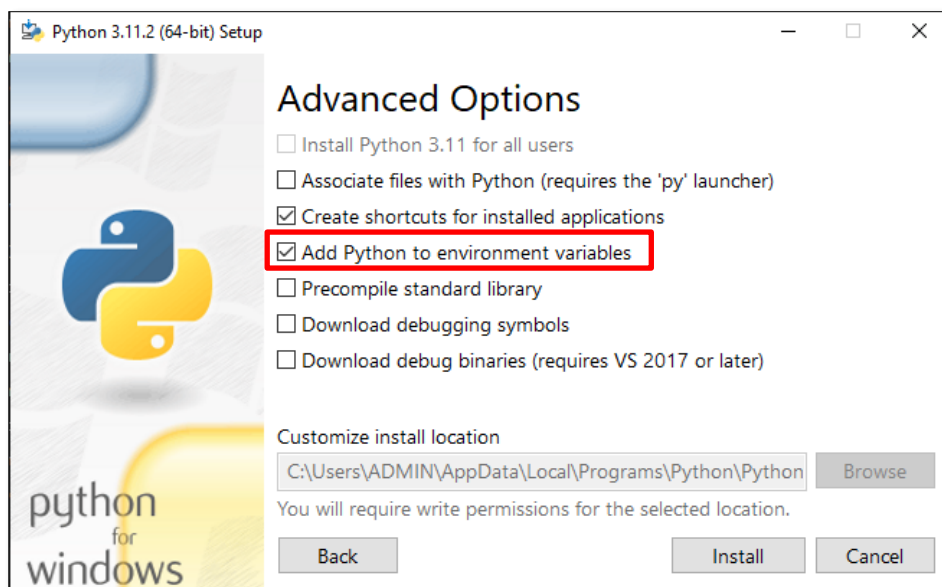
Apache Spark chạy trên nền máy ảo Java (JVM), do đó yêu cầu tiên quyết là cài đặt Java Development Kit (JDK).

- Java JDK: Khuyến nghị sử dụng JDK 8 (phiên bản ổn định nhất với Spark trên Windows). Tải và cài đặt JDK 8 (ví dụ: [Zulu OpenJDK 8](#) hoặc [Oracle JDK 8](#)). Đường dẫn cài đặt gợi ý: `D:\Java` (Tránh thư mục có khoảng trắng như "Program Files").



Hình 12: Thư mục Java 8 sau khi cài đặt

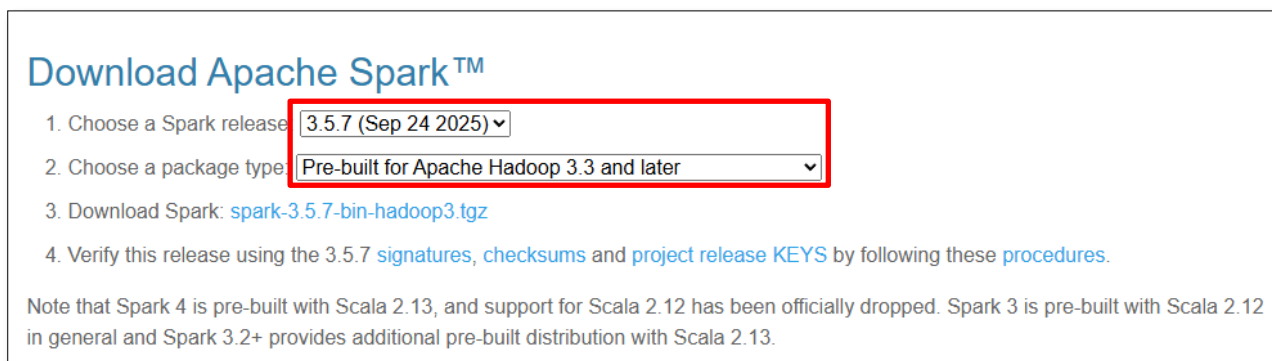
- Python: Cần thiết nếu sử dụng PySpark. Cài đặt Python (phiên bản 3.x) và đảm bảo đã chọn "Add Python to PATH/environment variables" khi cài đặt. Link tải Python: [Download Python | Python.org](#)



Hình 13: Cài đặt Python

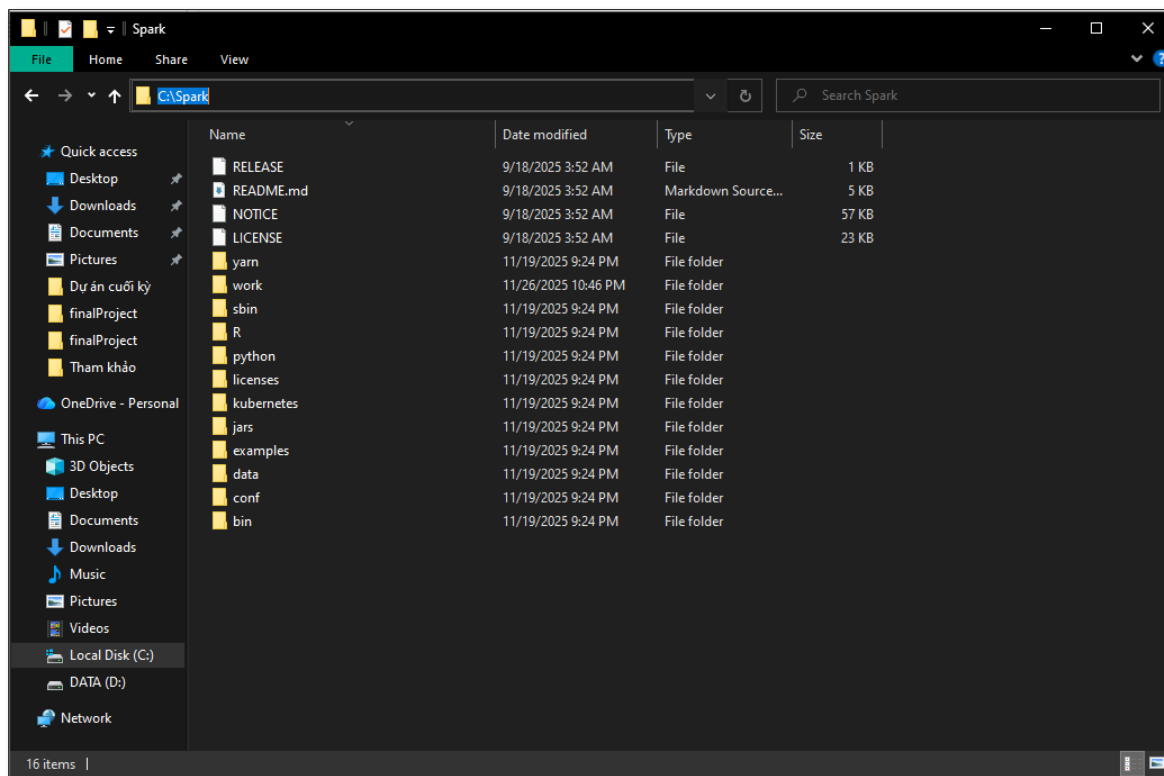
2.1.2. Tải và giải nén Apache Spark:

Truy cập trang web của Apache Spark: [Downloads | Apache Spark](#) và thực hiện chọn phiên bản Spark mới nhất (ví dụ: 3.5.x) và gói "Pre-built for Apache Hadoop". Ở đây sinh viên thực hiện tải phiên bản Spark 3.5.7.



Hình 14: Chọn phiên bản Spark và tải xuống

File tải xuống sẽ có định dạng **spark-3.5.7-bin-hadoop3.tgz**. Sử dụng công cụ giải nén (WinRAR/7-Zip) để giải nén file. Nên đổi tên thư mục giải nén thành Spark và đặt tại thư mục gốc ổ đĩa để đường dẫn ngắn gọn. Đường dẫn cài đặt: *C:\Spark* là thích hợp nhất.



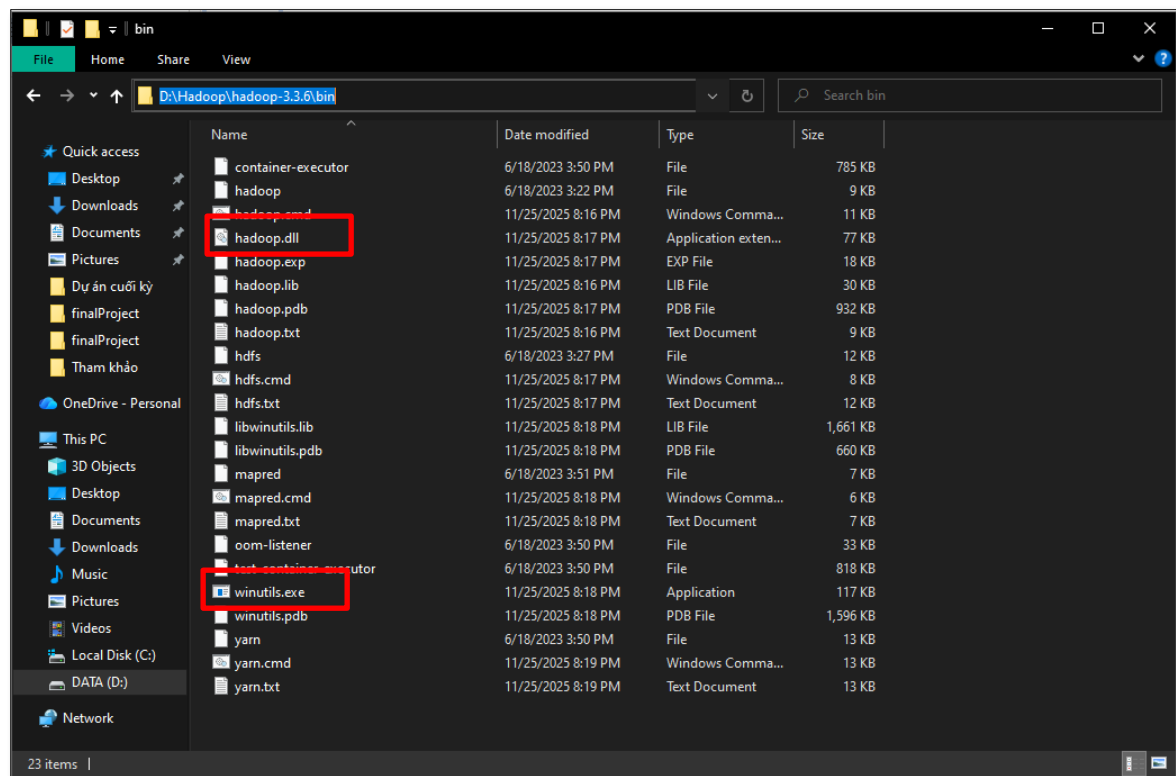
Hình 15: Thư mục Spark sau khi giải nén

2.1.3. Cấu hình Hadoop Utils cho Windows:

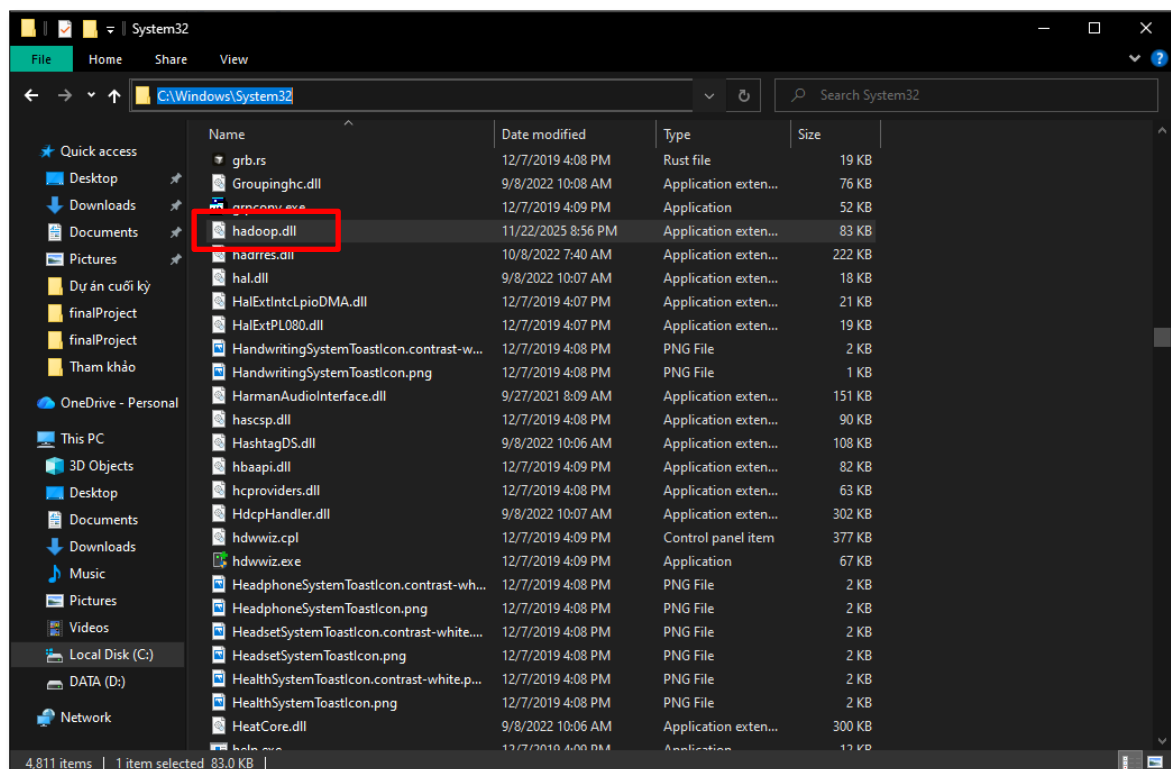
Spark trên Windows yêu cầu thư viện **winutils.exe** và **hadoop.dll** để tương tác với hệ thống tệp tin, vì Windows không hỗ trợ POSIX file permissions như Linux.

Tải bộ binary winutils ([cdarlint/winutils](https://cdarlint.com/winutils/): [winutils.exe](https://cdarlint.com/winutils/) [hadoop.dll](https://cdarlint.com/winutils/) and [hdfs.dll](https://cdarlint.com/winutils/) binaries for [hadoop windows](https://cdarlint.com/winutils/)) tương ứng với phiên bản Hadoop tích hợp trong Spark (thường là Hadoop 3.3.x).

Tiếp đến, tạo thư mục: `D:\Hadoop\bin`. Sao chép file **winutils.exe** và **hadoop.dll** vào thư mục `D:\Hadoop\bin`. Đồng thời, sao chép file **hadoop.dll** vào thư mục hệ thống `C:\Windows\System32` để tránh lỗi thiếu thư viện liên kết động.



Hình 16: Đặt **winutils.exe** và **hadoop.dll** vào thư mục



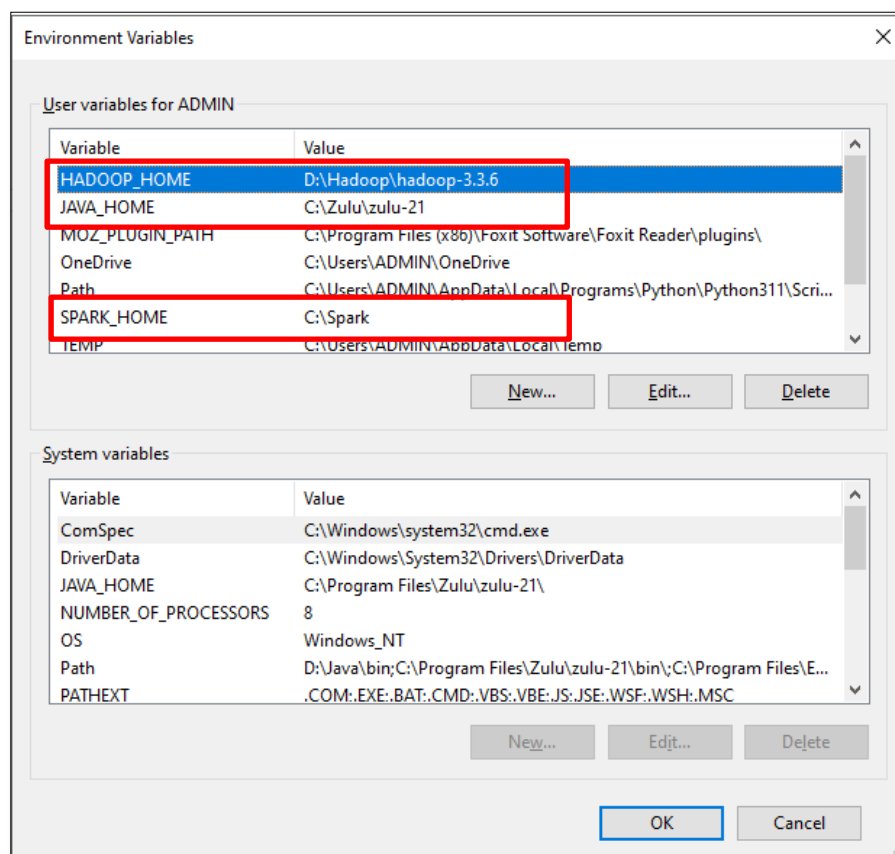
Hình 17: Đặt **hadoop.dll** vào thư mục **System32**

2.1.4. Thiết lập biến môi trường (Environment Variables):

Cần cấu hình các biến môi trường để hệ điều hành nhận diện được Spark và Hadoop.

Mở **System Properties > Environment Variables**. Tạo các biến hệ thống (**System Variables**) mới:

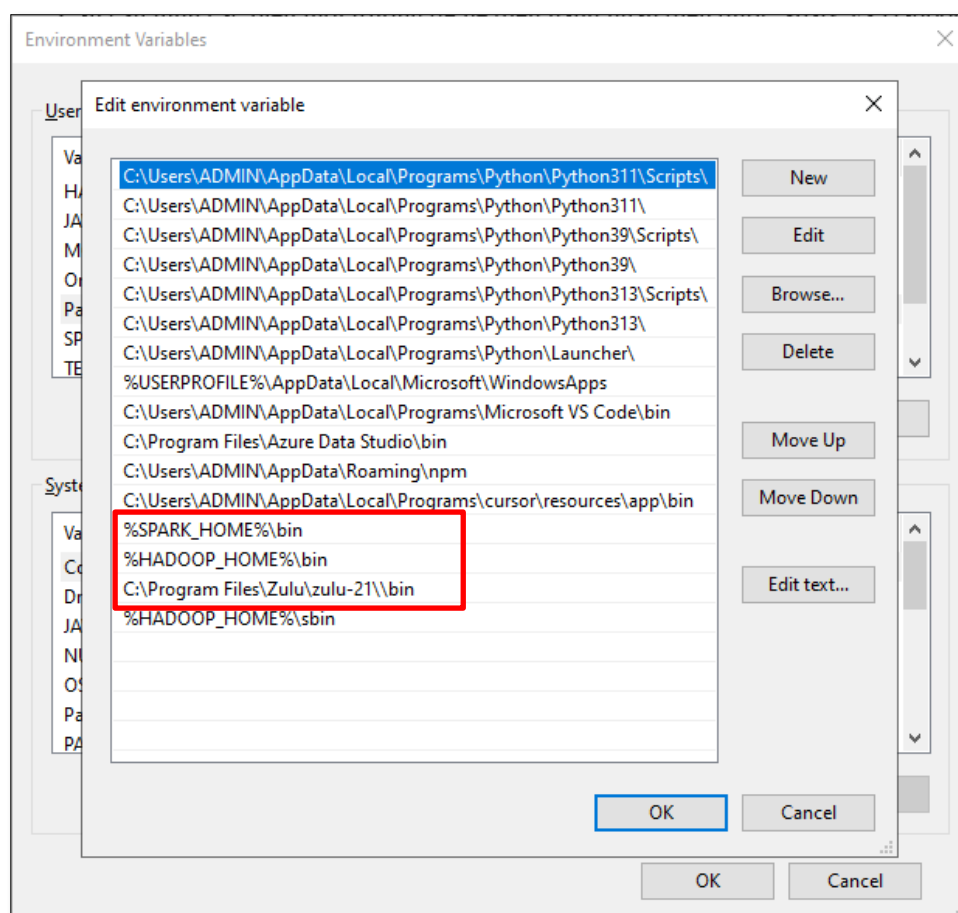
- **JAVA_HOME**: Trỏ đến thư mục cài Java.
- **SPARK_HOME**: Trỏ đến thư mục Spark.
- **HADOOP_HOME**: Trỏ đến thư mục Hadoop.



Hình 18: Cấu hình biến môi trường

Cập nhật biến **Path**: Thêm các đường dẫn sau vào danh sách:

- %JAVA_HOME%\bin
- %SPARK_HOME%\bin
- %HADOOP_HOME%\bin



Hình 19: Cấu hình thư mục đường dẫn Path

2.1.5. Kiểm thử cài đặt:

Để xác nhận quá trình cài đặt thành công trên máy đơn (Standalone mode). Mở Command Prompt (CMD). Tiếp đến, gõ lệnh: **spark-shell** (để vào môi trường Scala) hoặc **pyspark** (để vào môi trường Python). Nếu màn hình hiện logo **SPARK** và dòng lệnh chờ, quá trình cài đặt trên Windows đã hoàn tất.

Khi lệnh kích hoạt **spark-shell** được thực hiện thành công, màn hình sẽ trả về kết quả gồm các thông tin quan trọng:

- *Spark context Web UI available at http://DESKTOP-57U3OLD:4040:*

Spark đã khởi động một trang web quản trị (Dashboard) để bạn theo dõi công việc. Chi tiết:

- *DESKTOP-57U3OLD*: Tên máy tính hiện tại (Hostname).
- *4040*: Cổng (Port) mặc định của giao diện Web UI Spark.

Người dùng có thể mở trình duyệt (Chrome/Edge) và gõ <http://localhost:4040> để xem biểu đồ các job đang chạy, bộ nhớ đang dùng,...

```

C:\Users\ADMIN>spark-shell
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://DESKTOP-57U3OLD:4040
Spark context available as 'sc' (master = local[*], app id = local-1765380461815).
Spark session available as 'spark'.
Welcome to

  ____
 /  _ \
/_/_/  \_/_/  version 3.5.7

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 21.0.8)
Type in expressions to have them evaluated.
Type :help for more information.

scala> 25/12/10 22:27:56 WARN GarbageCollectionMetrics: To enable non-built-in garbage collector(s) List(G1 Concurrent G
C), users should configure it(them) to spark.eventLog.gcMetrics.youngGenerationGarbageCollectors or spark.eventLog.gcMet
rics.oldGenerationGarbageCollectors
1+1
res0: Int = 2

scala> _

```

Hình 20: Apache Spark được cài đặt thành công

- *Spark context available as 'sc' (master = local[*], app id = local-1765380461815):*
Đây là dòng quan trọng nhất, nó cho biết môi trường Spark đang chạy ở chế độ nào:
 - **Spark context available as 'sc':** Biến **sc** đã được tạo sẵn. Dùng biến này để điều khiển các thao tác cấp thấp (RDD). Ví dụ: `sc.textFile(...)`.
 - **master = local[*]: local:** Nghĩa là Spark đang chạy ở chế độ **Cục bộ (Local Mode)** trên chính máy tính này, **KHÔNG PHẢI** chạy trên cụm Cluster (nhiều máy). **[*]:** Nghĩa là Spark được phép sử dụng **Tất cả các nhân CPU (Cores)** có trên máy tính của để tính toán song song. (Ví dụ máy có 8 nhân thì spark được dùng cả 8).
 - **app id = local-...:** Mã định danh duy nhất cho lần chạy này (dùng để tra cứu log sau này).

2.2. Kết nối cụm máy Spark:

Sau khi hoàn tất cài đặt môi trường trên từng máy đơn lẻ, bước tiếp theo là kết nối các máy tính lại thành một hệ thống phân tán thống nhất. Mô hình triển khai được lựa chọn là Spark Standalone Mode với kiến trúc Master-Slave, sử dụng mạng riêng ảo (VPN) để đảm bảo kết nối thông suốt giữa các thiết bị.

2.2.1. Thiết lập mạng kết nối (Network Configuration):

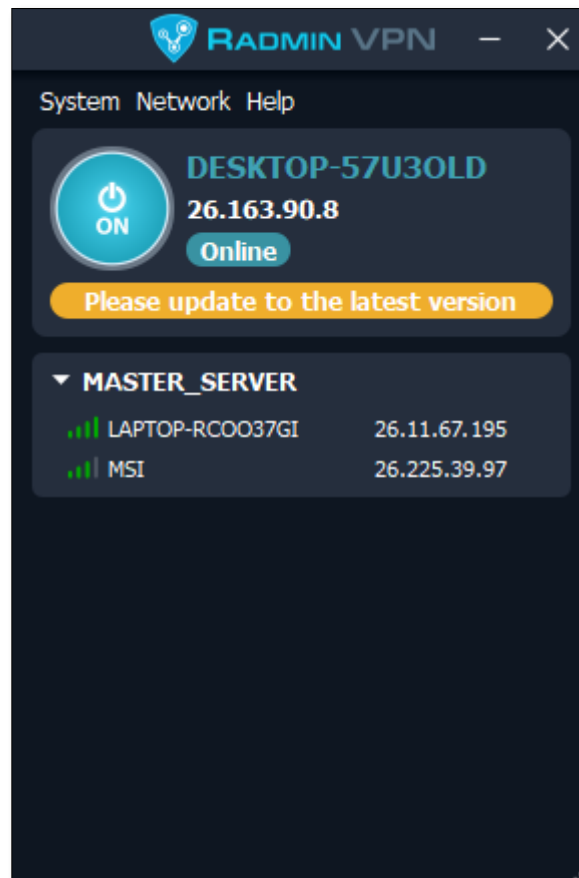
Do các máy tính (Laptop, PC) nằm ở các vị trí địa lý khác nhau hoặc sử dụng IP động trong mạng LAN, việc kết nối trực tiếp thường gặp trở ngại về tường lửa (Firewall) và định tuyến (Routing). Giải pháp được sử dụng là Radmin VPN để tạo một mạng Lan ảo (Virtual LAN).

Cài đặt: Cài đặt **Radmin VPN** trên tất cả các máy (Nút Master và các nút Worker).

Thiết lập mạng: Tạo một mạng mới (Network Name) trên máy Master và cho phép các máy Worker tham gia vào mạng này.

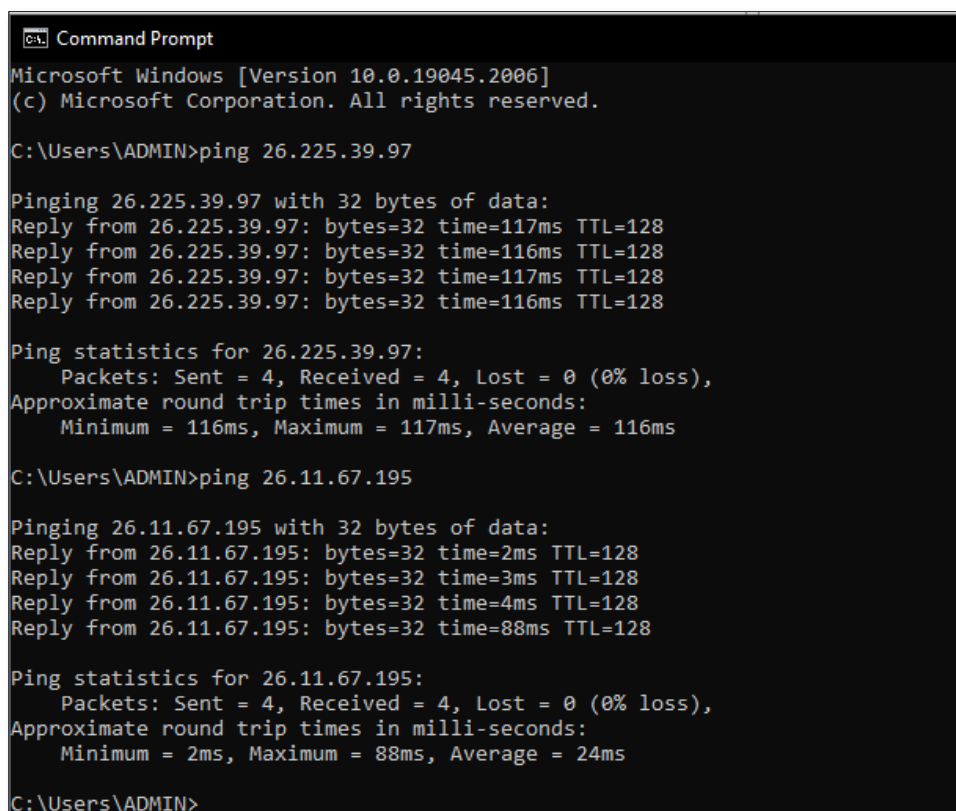
Đồng bộ IP: Sau khi kết nối, mỗi máy sẽ được cấp một địa chỉ IP tĩnh trong dải 26.x.x.x.

- Ví dụ IP Máy Master: 26.163.90.8
- Ví dụ IP Máy Worker 1: 26.11.67.195
- Ví dụ IP Máy Worker 2: 26.225.39.97



Hình 21: Cấu hình mạng LAN

Kiểm tra: Sử dụng lệnh *ping* <IP_Worker> từ máy Master hoặc ngược lại để đảm bảo kết nối mạng đã thông suốt.



```
Command Prompt
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>ping 26.225.39.97

Pinging 26.225.39.97 with 32 bytes of data:
Reply from 26.225.39.97: bytes=32 time=117ms TTL=128
Reply from 26.225.39.97: bytes=32 time=116ms TTL=128
Reply from 26.225.39.97: bytes=32 time=117ms TTL=128
Reply from 26.225.39.97: bytes=32 time=116ms TTL=128

Ping statistics for 26.225.39.97:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 116ms, Maximum = 117ms, Average = 116ms

C:\Users\ADMIN>ping 26.11.67.195

Pinging 26.11.67.195 with 32 bytes of data:
Reply from 26.11.67.195: bytes=32 time=2ms TTL=128
Reply from 26.11.67.195: bytes=32 time=3ms TTL=128
Reply from 26.11.67.195: bytes=32 time=4ms TTL=128
Reply from 26.11.67.195: bytes=32 time=88ms TTL=128

Ping statistics for 26.11.67.195:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 88ms, Average = 24ms

C:\Users\ADMIN>
```

Hình 22: Kiểm tra kết nối từ Master đến các Worker

Việc thiết lập mạng ảo (VPN) thành công để giả lập môi trường Cluster. Dải IP 26.x.x.x xác nhận việc dùng Radmin VPN thành công. Ping 26.225.39.97 (Time ~116ms) cho thấy độ trễ này là kết nối tới một máy ở xa (Máy của Nguyễn Công Đạt) nhưng kết nối ổn định (0% loss). Ping 26.11.67.195 (Time ~3ms), độ trễ cực thấp, gần như tức thời (Máy của Phùng Ngọc Bảo), đây là máy worker chung mạng với máy master. Hạ tầng mạng đã thông suốt, đảm bảo Master và Worker có thể "nhìn thấy" nhau để truyền dữ liệu.

2.2.2. Khởi động máy Master (Master Node):

Nút Master đóng vai trò điều phối tài nguyên và quản lý lịch trình công việc. Trên máy được chọn cho vai trò Master, mở Command Prompt (Admin) và thực thi lệnh khởi động.

```
spark-class org.apache.spark.deploy.master.Master --host 26.163.90.8 --port 7077
```

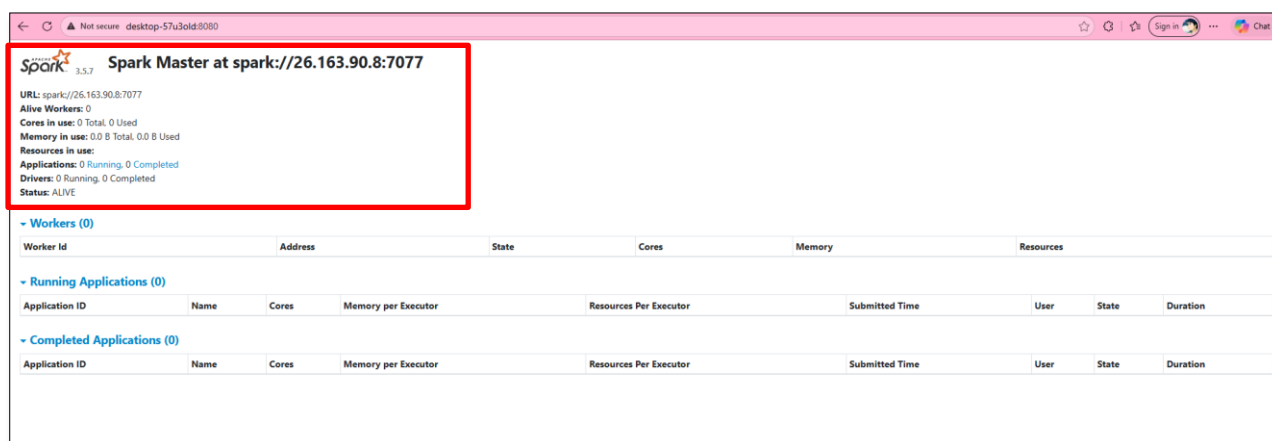
Sau khi khởi động thành công, truy cập giao diện quản trị (Web UI) tại địa chỉ <http://26.163.90.8:8080>. Tại đây, trạng thái *Status: ALIVE* xác nhận Master đã sẵn sàng nhận kết nối.

```

Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>spark-class org.apache.spark.deploy.master.Master --host 26.163.90.8 --port 7077
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/12/10 23:11:31 INFO Master: Started daemon with process name: 8104@DESKTOP-57U3OLD
25/12/10 23:11:31 INFO SecurityManager: Changing view acls to: ADMIN
25/12/10 23:11:31 INFO SecurityManager: Changing modify acls to: ADMIN
25/12/10 23:11:31 INFO SecurityManager: Changing view acls groups to:
25/12/10 23:11:31 INFO SecurityManager: Changing modify acls groups to:
25/12/10 23:11:31 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view perm
issions: ADMIN; groups with view permissions: EMPTY; users with modify permissions: ADMIN; groups with modify permission
s: EMPTY
25/12/10 23:11:31 INFO Utils: Successfully started service 'sparkMaster' on port 7077.
25/12/10 23:11:31 INFO Master: Starting Spark master at spark://26.163.90.8:7077
25/12/10 23:11:31 INFO Master: Running Spark version 3.5.7
25/12/10 23:11:31 INFO JettyUtils: Start Jetty 0.0.0.0:8080 for MasterUI
25/12/10 23:11:31 INFO Utils: Successfully started service 'MasterUI' on port 8080.
25/12/10 23:11:31 INFO MasterWebUI: Bound MasterWebUI to 0.0.0.0, and started at http://DESKTOP-57U3OLD:8080
25/12/10 23:11:31 INFO Master: I have been elected leader! New state: ALIVE
  
```

Hình 23: Màn hình command kích hoạt Master Node



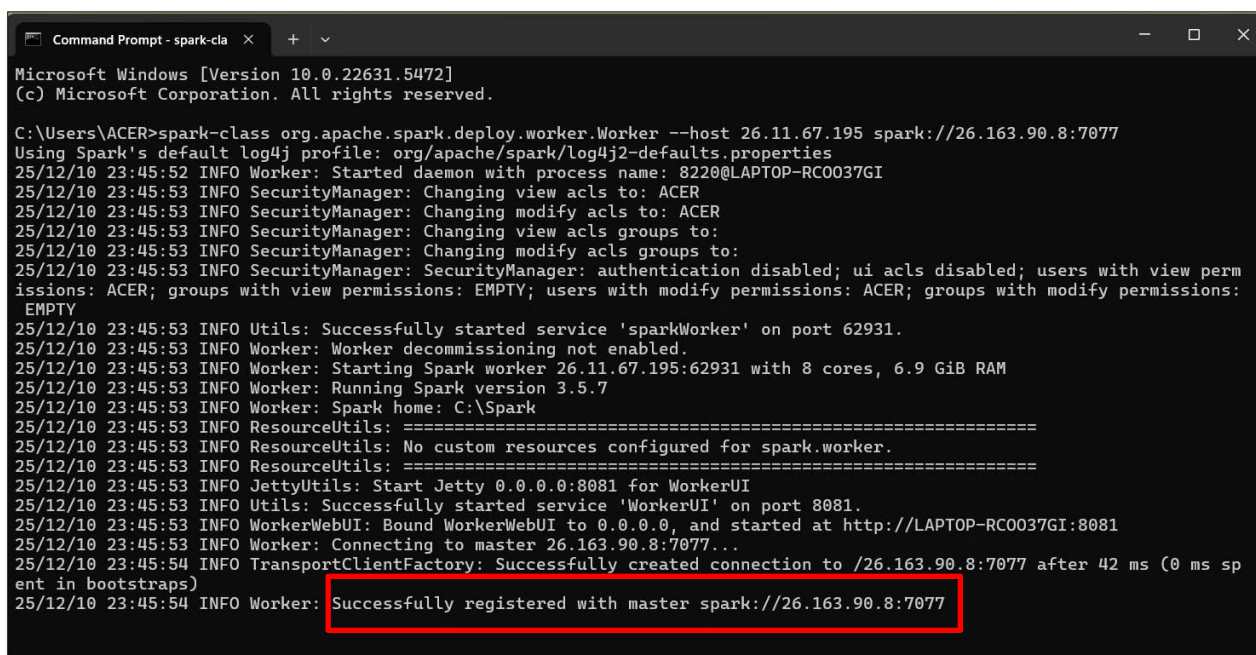
Hình 24: Giao diện quản trị của máy Master

2.2.3. Khởi động máy Worker (Worker Node):

Các nút Worker chịu trách nhiệm thực thi các tác vụ tính toán và lưu trữ dữ liệu. Quá trình kết nối Worker vào Cluster thực hiện như sau (Đảm bảo máy Worker đã cài đặt đầy đủ Java, Spark và Hadoop Utils). Mở Command Prompt trên máy Worker. Chạy lệnh kết nối tới Master thông qua giao thức `spark://`.

```
spark-class org.apache.spark.deploy.worker.Worker --host 26.11.67.195 spark://26.163.90.8:7077
```

```
spark-class org.apache.spark.deploy.worker.Worker --host 26.225.39.97 spark://26.163.90.8:7077
```



```
Microsoft Windows [Version 10.0.22631.5472]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER>spark-class org.apache.spark.deploy.worker.Worker --host 26.11.67.195 spark://26.163.90.8:7077
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/12/10 23:45:52 INFO Worker: Started daemon with process name: 8220@LAPTOP-RC0037GI
25/12/10 23:45:53 INFO SecurityManager: Changing view acls to: ACER
25/12/10 23:45:53 INFO SecurityManager: Changing modify acls to: ACER
25/12/10 23:45:53 INFO SecurityManager: Changing view acls groups to:
25/12/10 23:45:53 INFO SecurityManager: Changing modify acls groups to:
25/12/10 23:45:53 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view perm
issions: ACER; groups with view permissions: EMPTY; users with modify permissions: ACER; groups with modify permissions:
EMPTY
25/12/10 23:45:53 INFO Utils: Successfully started service 'sparkWorker' on port 62931.
25/12/10 23:45:53 INFO Worker: Worker decommissioning not enabled.
25/12/10 23:45:53 INFO Worker: Starting Spark worker 26.11.67.195:62931 with 8 cores, 6.9 GiB RAM
25/12/10 23:45:53 INFO Worker: Running Spark version 3.5.7
25/12/10 23:45:53 INFO Worker: Spark home: C:\Spark
25/12/10 23:45:53 INFO ResourceUtils: =====
25/12/10 23:45:53 INFO ResourceUtils: No custom resources configured for spark.worker.
25/12/10 23:45:53 INFO ResourceUtils: =====
25/12/10 23:45:53 INFO JettyUtils: Start Jetty 0.0.0.0:8081 for WorkerUI
25/12/10 23:45:53 INFO Utils: Successfully started service 'WorkerUI' on port 8081.
25/12/10 23:45:53 INFO WorkerWebUI: Bound WorkerWebUI to 0.0.0.0, and started at http://LAPTOP-RC0037GI:8081
25/12/10 23:45:53 INFO Worker: Connecting to master 26.163.90.8:7077...
25/12/10 23:45:54 INFO TransportClientFactory: Successfully created connection to /26.163.90.8:7077 after 42 ms (0 ms sp
ent in bootstraps)
25/12/10 23:45:54 INFO Worker: Successfully registered with master spark://26.163.90.8:7077
```

Hình 25: Máy Worker (26.11.67.195) kết nối thành công đến Master

```

C:\Users\ADMIN>spark-class org.apache.spark.deploy.master.Master --host 26.163.90.8 --port 7077
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

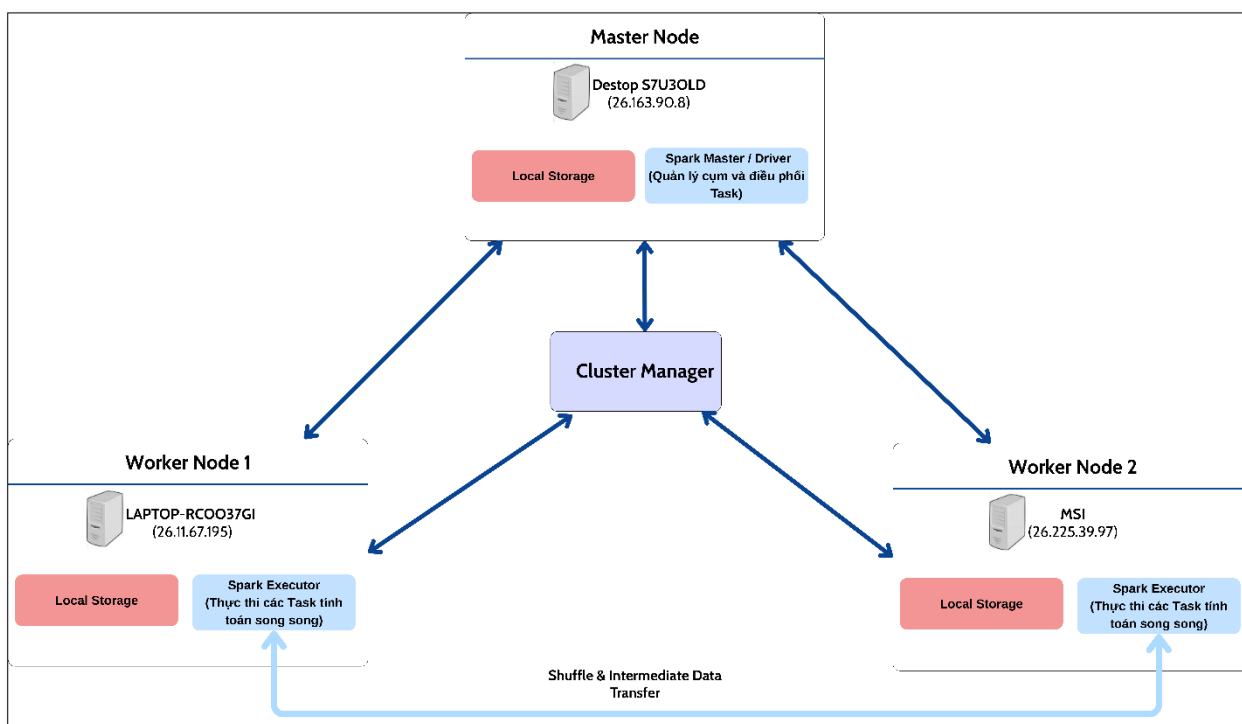
C:\Users\ADMIN>spark-class org.apache.spark.deploy.master.Master --host 26.163.90.8 --port 7077
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/12/10 23:11:31 INFO Master: Started daemon with process name: 8104@DESKTOP-S7U3OLD
25/12/10 23:11:31 INFO SecurityManager: Changing view acls to: ADMIN
25/12/10 23:11:31 INFO SecurityManager: Changing modify acls to: ADMIN
25/12/10 23:11:31 INFO SecurityManager: Changing view acls groups to:
25/12/10 23:11:31 INFO SecurityManager: Changing modify acls groups to:
25/12/10 23:11:31 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view perm
issions: ADMIN; groups with view permissions: EMPTY; users with modify permissions: ADMIN; groups with modify permission
s: EMPTY
25/12/10 23:11:31 INFO Utils: Successfully started service 'sparkMaster' on port 7077.
25/12/10 23:11:31 INFO Master: Starting Spark master at spark://26.163.90.8:7077
25/12/10 23:11:31 INFO Master: Running Spark version 3.5.7
25/12/10 23:11:31 INFO JettyUtils: Start Jetty 0.0.0.0:8080 for MasterUI
25/12/10 23:11:31 INFO Utils: Successfully started service 'MasterUI' on port 8080.
25/12/10 23:11:31 INFO MasterWebUI: Bound MasterWebUI to 0.0.0.0, and started at http://DESKTOP-S7U3OLD:8080
25/12/10 23:11:31 INFO Master: I have been elected leader! New state: ALIVE
25/12/10 23:45:53 INFO Master: Registering worker 26.11.67.195:62931 with 8 cores, 6.9 GiB RAM

```

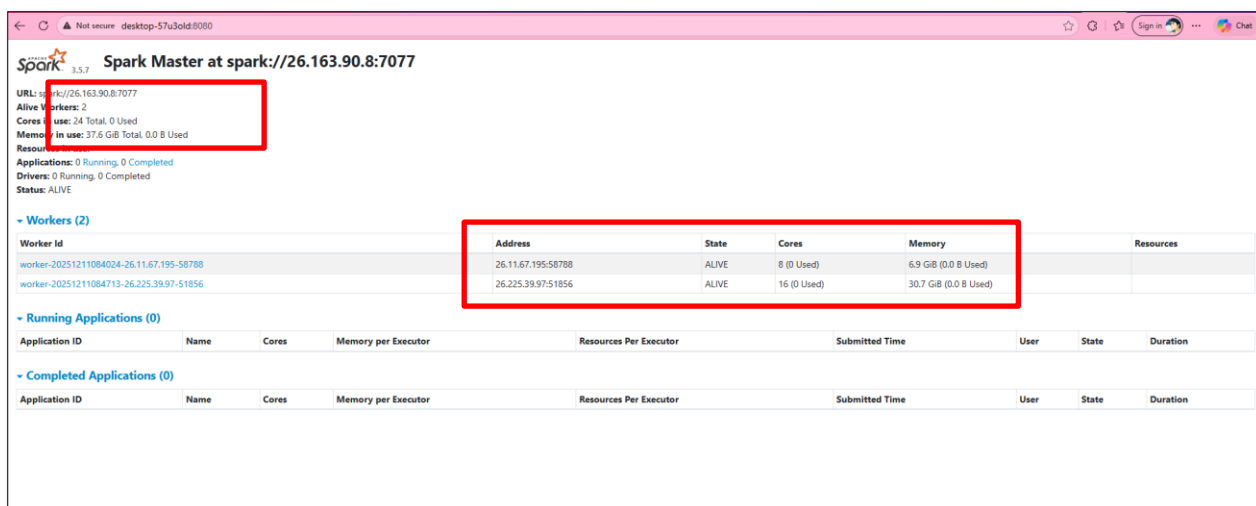
Hình 26: Máy Master xuất thông báo có máy Worker kết nối thành công

2.2.4. Kiến trúc cụm máy Apache Spark:

Sau khi thực hiện kết nối các máy Worker đến máy Master, ta có được cụm kết nối (Cluster).



Hình 27: Sơ đồ cụm máy



Hình 28: Giao diện quản lý kết nối

Hình ảnh trên hiển thị giao diện quản trị (Dashboard) của Apache Spark Master sau khi khởi tạo thành công. Hệ thống hoạt động ở chế độ Standalone với các thông số vận hành như sau:

- Quy mô cụm (Scale): Hệ thống bao gồm 1 Master Node và 2 Worker Nodes (được định danh qua địa chỉ IP VPN 26.11.67.195 và 26.225.39.97).
- Tài nguyên tính toán (Computing Resources): Tổng năng lực xử lý của cụm đạt 24 Cores CPU và 37.6 GB bộ nhớ RAM. Đây là nguồn tài nguyên đáng kể, cho phép xử lý các tập dữ liệu lớn mà một máy tính cá nhân đơn lẻ khó có thể thực hiện trong thời gian ngắn.
- Trạng thái (Status): Cả Master và các Worker đều ở trạng thái ALIVE, xác nhận rằng cấu hình mạng Radmin VPN và các thiết lập biến môi trường (Hadoop/Java) trên Windows đã hoàn toàn chính xác. Hệ thống sẵn sàng cho việc phân tán tác vụ (Task Distribution).

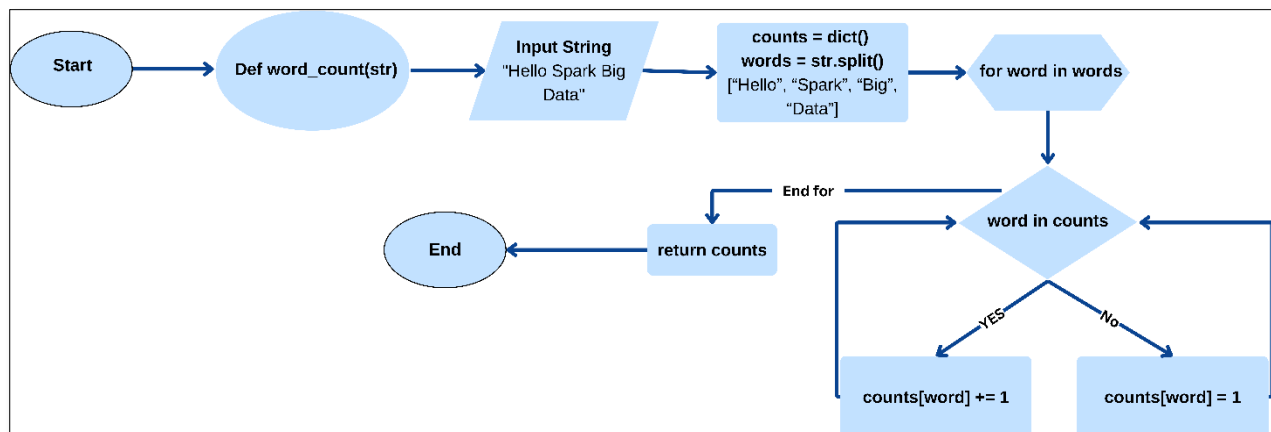
Node	IP	Cores	Memory
Master	26.163.90.8	-	-
Worker 1	26.11.67.195	8	6.9
Worker 2	26.225.39.97	16	30.7

Bảng 2: Danh sách tài nguyên của cụm máy

Với các kết quả về việc xây dựng hệ thống phân tán thành công, có thể sử dụng hệ thống để thực hiện những bài toán về triển khai tính toán song song và thực hiện đánh giá.

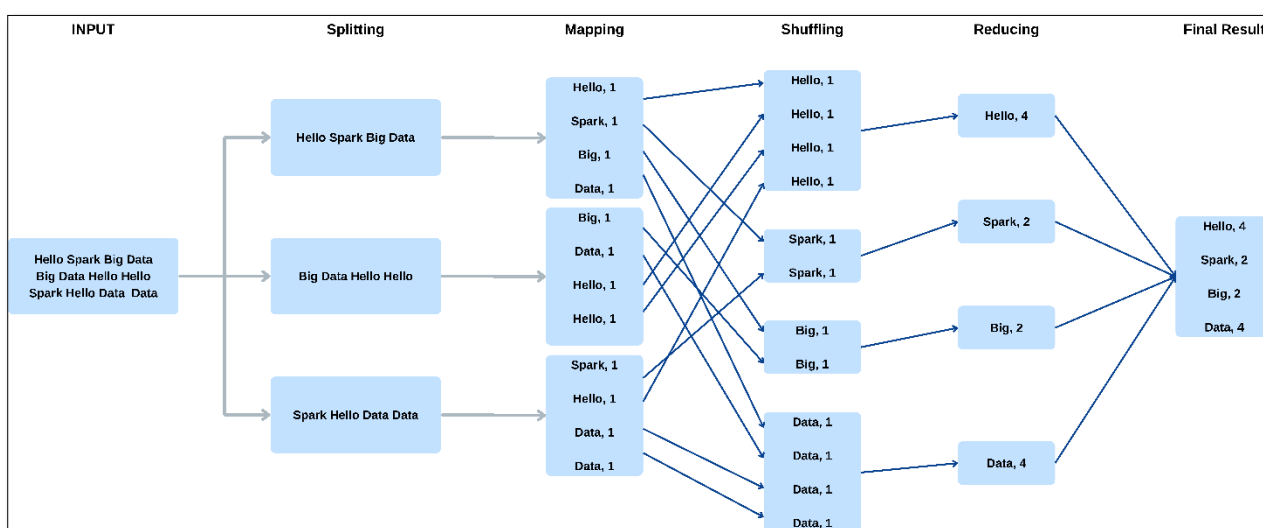
3. Đếm từ (Word count) với Apache Spark:

Bài toán Word Count (Đếm từ) trong Apache Spark là bài toán kinh điển, cơ bản nhất để minh họa khả năng xử lý dữ liệu lớn (Big Data) phân tán của Spark, giúp chúng ta hiểu cách Spark thực hiện các phép biến đổi trên RDD/DataFrame, phân chia công việc (Map) và tổng hợp kết quả (Reduce) để đếm tần suất xuất hiện của các từ trong một tập văn bản khổng lồ, vượt quá khả năng bộ nhớ của một máy đơn.



Hình 29: Word Count với mô hình tuần tự

Sơ đồ khối trên mô tả thuật toán đếm từ được cài đặt tuần tự bằng Python dưới dạng hàm `word_count(str)`. Từ điểm bắt đầu, chương trình nhận vào một chuỗi ký tự, ví dụ "Hello Spark Big Data", sau đó khởi tạo một từ điển rỗng `counts = dict()` và tách chuỗi thành danh sách các từ `words = str.split()`. Vòng lặp `for word in words` duyệt lần lượt từng phần tử trong danh sách này. Với mỗi từ, chương trình kiểm tra điều kiện nhánh: nếu từ đã tồn tại trong từ điển (`word in counts`), giá trị đếm tương ứng được tăng thêm 1; ngược lại, từ đó được thêm mới vào từ điển với giá trị khởi tạo bằng 1. Khi vòng lặp kết thúc, hàm trả về cấu trúc từ điển `counts` chứa số lần xuất hiện của từng từ và tiến tới điểm kết thúc.



Hình 30: Kiến trúc Word Count với Spark

Quy trình xử lý trong hình minh họa mô tả cách một bài toán đếm số lần xuất hiện của từ được thực hiện theo mô hình MapReduce (như trong Hadoop/Spark). Trước hết, dữ liệu văn bản đầu vào gồm nhiều dòng sẽ được chia nhỏ (Splitting) thành các phần độc lập, mỗi phần có thể được xử lý trên một nút khác nhau trong cụm máy. Trên từng phần dữ liệu đó, hệ thống thực hiện bước ánh xạ (Mapping): mỗi dòng được tách thành các từ, và mỗi từ được biến thành một cặp khóa–giá trị dạng (từ, 1) – ví dụ “Hello Spark Big Data” sẽ được ánh xạ thành các cặp (Hello,1), (Spark,1), (Big,1), (Data,1). Sau khi tất cả các mapper phát ra các cặp (từ,1), hệ thống bước vào giai đoạn xáo trộn và gom nhóm (Shuffling), trong đó các cặp có cùng khóa (cùng một từ) được chuyển và gom về cùng một nhóm trên một nút xử lý, chẳng hạn toàn bộ (Hello,1) nằm chung một nhóm, toàn bộ (Spark,1) nằm chung một nhóm,... Cuối cùng, ở bước giảm (Reducing), hàm reduce sẽ duyệt từng nhóm, cộng dồn các giá trị 1 tương ứng với mỗi từ để thu được tổng số lần xuất hiện của từ đó, tạo ra kết quả cuối cùng: Hello, 4, Spark, 2, Big, 2, Data, 4. Nhờ cơ chế “chia nhỏ – xử lý song song – gom nhóm – tổng hợp” này, hệ thống có thể mở rộng để xử lý các tệp văn bản rất lớn trên nhiều máy một cách hiệu quả.

So với mô hình Word Count trên Spark, mô hình đếm từ tuần tự có một số nhược điểm rõ rệt. Thứ nhất, toàn bộ quá trình xử lý chỉ diễn ra trên một tiến trình, một máy, nên hoàn toàn không tận dụng được khả năng xử lý song song; khi kích thước dữ liệu tăng lớn, thời gian thực thi sẽ tăng gần như tuyến tính và nhanh chóng trở nên quá tải. Thứ hai, chương trình bị giới hạn bởi tài nguyên phần cứng cục bộ (RAM, CPU, ổ đĩa) – nếu văn bản quá lớn để nạp vào bộ nhớ hoặc lưu trữ trên một máy, thuật toán này không còn khả thi, trong khi Spark cho phép phân tán dữ liệu và tính toán trên nhiều node. Thứ ba, mô hình tuần tự thiếu cơ chế chịu lỗi (fault tolerance): nếu quá trình thực thi bị lỗi giữa chừng, toàn bộ phép tính phải chạy lại từ đầu; trái lại, Spark lưu trữ lineage của RDD/DataFrame nên có thể tự động khôi phục các partition lỗi. Cuối cùng, mô hình Python thuần túy khó mở rộng sang các bài toán phức tạp hơn trên cụm máy (kết hợp với HDFS, streaming, machine learning,...), trong khi Word Count trên Spark chỉ là một ví dụ nhỏ nằm trong hệ sinh thái xử lý dữ liệu lớn phân tán, có thể tái sử dụng và mở rộng rất linh hoạt.

3.1. Word Count với Spark:

Sử dụng file text *wordcount.txt* trong file đính kèm để thực hiện đếm từ cũng như tần suất xuất hiện của các từ trong file.

```
C:\Users\ADMIN\PycharmProjects\pythonProject\venv\Scripts\python.exe "C:\Users\ADMIN\PycharmProjects\pythonProject\lapTrinhSongSongVaPhanTan\Word_Count_voi_Worker.py"
Đang khởi tạo Spark (Java 11)...
Picked up _JAVA_OPTIONS: --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED
Picked up _JAVA_OPTIONS: --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/11 09:51:25 WARN SparkConf: Note that spark.local.dir will be overridden by the value set by the cluster manager (via SPARK_LOCAL_DIRS in mesos/standalone/kubernetes and LOCAL_DIRS in YARN).
>>> SparkContext OK!
--- ĐANG ĐỌC FILE LOCAL: D:\Tài liệu đại học\Năm 4\Lập trình song song và phân tán\Dự án cuối kỳ\wordcount.txt ---
-> Đã đọc 44 dòng vào RAM Master.
--- ĐANG PHÂN TÁN DỮ LIỆU SANG CLUSTER ---
--- ĐANG XỬ LÝ MAP - REDUCE ---

=====
KẾT QUẢ THỐNG KÊ
=====
✅ TỔNG SỐ TỪ KHÁC NHAU (UNIQUE WORDS): 381
🕒 THỜI GIAN THỰC THI TOÀN BỘ: 8.3968 giây

-----
Chi tiết tần suất xuất hiện:
- 'the': 38 lần
- 'a': 28 lần
- 'of': 25 lần
- 'word': 24 lần
- 'and': 23 lần
- 'words': 21 lần
- 'is': 19 lần
- 'to': 18 lần
- 'count': 11 lần
```

Hình 31: Bảng kết quả trả về wordcount trong PyCharm

Kết quả trên màn hình cho thấy chương trình Word Count chạy bằng Spark đã thực thi đúng quy trình và trả về thống kê khá rõ ràng. Hệ thống đã xử lý toàn bộ file văn bản và tìm được 381 từ khác nhau (unique words), với thời gian thực thi toàn bộ chỉ khoảng 8.2275 giây, phù hợp với một bài toán đếm từ trên tập dữ liệu không quá lớn nhưng có đầy đủ overhead của Spark (khởi tạo context, phân tán dữ liệu, shuffle, reduce,...). Bảng “Chi tiết tần suất xuất hiện” cho thấy các từ có tần suất cao nhất đều là các hư từ tiếng Anh thường gặp như 'the' (38 lần), 'a' (28 lần), 'of' (25 lần), 'and' (23 lần), 'to', 'in',... – điều này phản ánh đúng đặc trưng của văn bản tiếng Anh nói chung, nơi các từ chức năng (stopwords) chiếm tần suất rất lớn. Một số từ gắn với nội dung bài toán như 'word' (24 lần), 'words' (21 lần), 'count' (11 lần) cho thấy nội dung file đầu vào có liên quan tới chủ đề “word count”. Nhìn chung, kết quả chứng minh kiến trúc Map → Shuffle → Reduce của Spark hoạt động chính xác, đồng thời thời gian xử lý ngắn thể hiện khả năng phù hợp của Spark cho các bài toán xử lý văn bản song song và có thể mở rộng lên dữ liệu lớn hơn.

3.2. So sánh Word Count khi sử dụng Spark với không sử dụng:

Thời gian chạy tuần tự	Thời gian chạy song song (1 worker node – 8 cores)	Thời gian chạy song song (2 workers node – 24 cores)
0.0000	8.2484	8.0992

Bảng 3: So sánh thời gian chạy giữa các yếu tố (giây)

The image displays three terminal windows side-by-side, each showing the output of a Word Count program. The first window, titled 'PHẦN 1: PYTHON LOCAL (CHỈ CHẠY TRÊN MÁY MASTER)', shows a Python-based execution with a time of 0.0000 seconds. The second window, titled 'PHẦN 2: SPARK CLUSTER (PHÂN TÁN TRÊN 2 MÁY WORKER)', shows a Spark-based execution with a time of 8.2484 seconds. The third window, titled 'PHẦN 2: SPARK CLUSTER (PHÂN TÁN TRÊN 1 MÁY WORKER)', shows a Spark-based execution with a time of 8.0992 seconds. All three windows display the same list of top 10 words and their frequencies: 'the': 38, 'a': 28, 'of': 25, 'word': 24, 'and': 23, 'words': 21, 'is': 19, 'to': 18, 'count': 11, 'in': 11. The Spark results also include a confirmation of 381 unique words.

Hình 32: Các kết quả chạy

Kết quả so sánh cho thấy với tập dữ liệu Word Count đang dùng, thời gian chạy tuần tự bằng Python gần như bằng 0 (0.0000 giây), trong khi khi chuyển sang chạy song song bằng Spark trên cụm 1 worker (8 cores) và 2 workers (24 cores) thì thời gian thực thi lần lượt vào khoảng xấp xỉ 8.2484 giây và 8.0992 giây. Điều này chứng tỏ chi phí khởi tạo SparkContext, phân tán dữ liệu, thực hiện các bước shuffle–reduce giữa các node và thu gom kết quả đã chiếm phần lớn thời gian xử lý, lớn hơn rất nhiều so với chi phí tính toán bản thân bài toán đếm từ trên một file văn bản nhỏ. Sự chênh lệch thời gian giữa cấu hình 1 worker và 2 workers cũng không đáng kể, cho thấy việc tăng số node và số lõi không mang lại lợi ích rõ rệt khi khối lượng dữ liệu quá nhỏ. Như vậy, thí nghiệm này minh họa khá rõ ràng với bài

toán và quy mô dữ liệu hiện tại, chạy tuần tự trên một máy đơn giản là hiệu quả hơn, còn ưu thế của Spark và mô hình lập trình song song – phân tán chỉ thực sự phát huy khi làm việc với các bộ dữ liệu lớn, nơi chi phí khởi tạo và truyền thông được “chia đều” trên một khối lượng tính toán đủ lớn.

3.3. So sánh Word Count với độ lớn dữ liệu tăng dần:

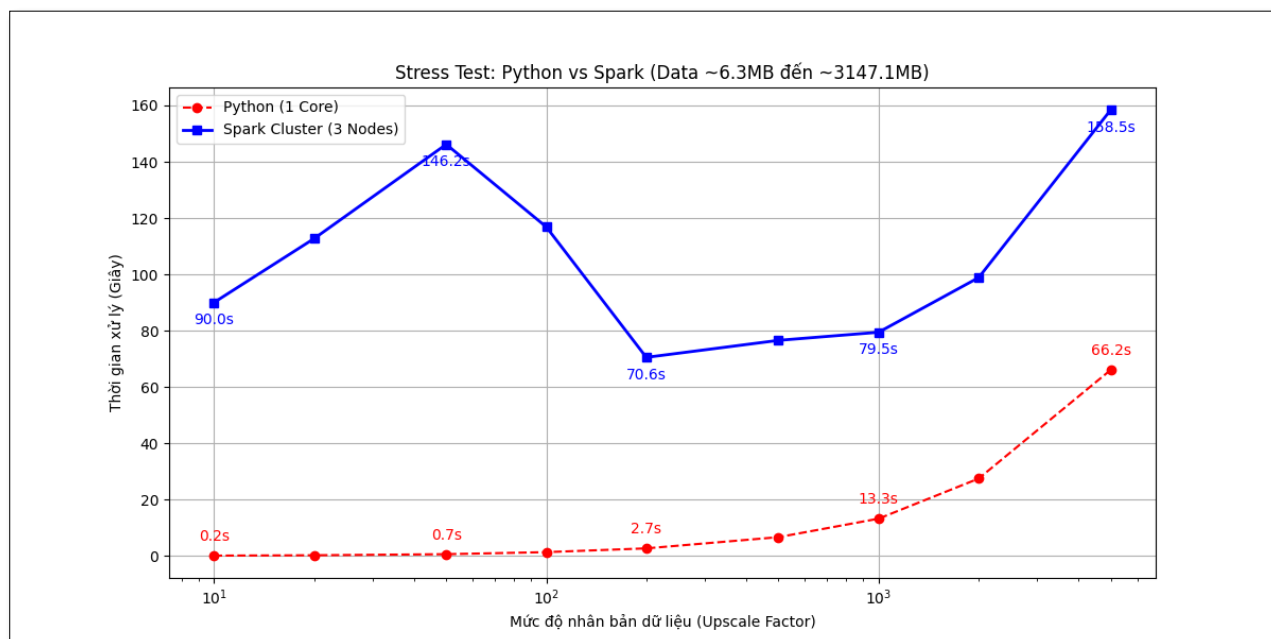
Tiến hành tạo một file dữ liệu text mẫu từ đoạn ký tự sau “*Hello Spark Big Data Hadoop AI Machine Learning Python Java Scala*”. Sau đó nhân lần lượt đoạn ký tự với các mức độ lớn để tạo thành các mức độ lớn của file văn bản để tiến hành đánh giá.

```
base_file = "base_content.txt"
# Tạo nội dung gốc khoảng 600KB
base_content = "Hello Spark Big Data Hadoop AI Machine Learning Python Java Scala " * 10000
# Các mức nhân bản (Upscale)
MULTIPLIERS = [10, 20, 50, 100, 200, 500, 1000, 2000, 5000]
```

File (× độ lớn)	Độ lớn (MB)	Thời gian chạy tuần tự - giây	Thời gian chạy song song với Spark (cụm 2 workers node) - giây
×10	~6.29	0.1517	90.0357
×20	~12.59	0.2842	112.7071
×50	~31.47	0.6718	146.1961
×100	~62.94	1.3943	116.8776
×200	~125.89	2.7306	70.5549
×500	~314.71	6.7203	76.5968
×1000	~629.43	13.2805	79.4712
×2000	~1258.85	27.5084	98.8666

×5000	~3147.13	66.1504	158.4862
-------	----------	---------	----------

Bảng 4: Bảng so sánh thời gian chạy với mỗi độ lớn khác nhau



Hình 33: So sánh thời gian chạy với các mức độ lớn của file

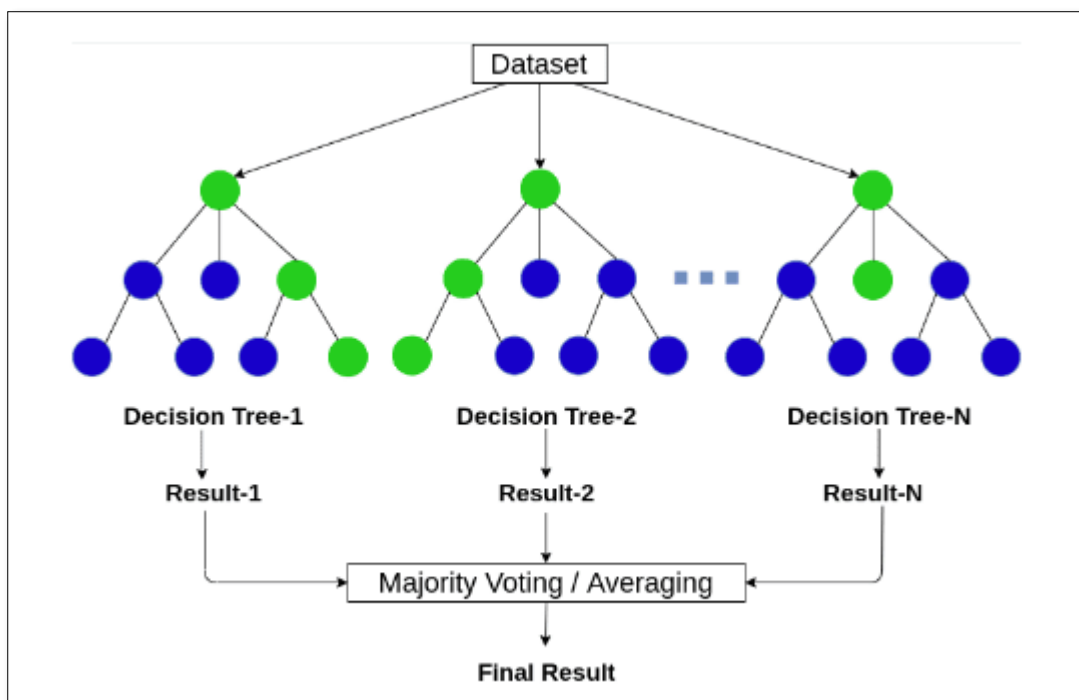
Kết quả thực nghiệm trên tập dữ liệu từ 6.3MB đến hơn 3GB cho thấy sự ưu việt về tốc độ xử lý của kiến trúc tuần tự (Python) so với kiến trúc phân tán (Spark) ở quy mô dữ liệu vừa và nhỏ. Cụ thể, tại mức dữ liệu lớn nhất (~3.1GB), Python chỉ mất 66.2 giây để hoàn thành tác vụ, nhanh gấp 2.4 lần so với Spark Cluster (158.5 giây). Nguyên nhân chính nằm ở chi phí khởi tạo (overhead) và độ trễ giao tiếp mạng (network I/O) của Spark là quá lớn so với thời gian tính toán thực tế. Trong khi Python tận dụng tối đa tốc độ truy xuất trực tiếp của RAM trên đơn máy, Spark lại tiêu tốn tài nguyên cho việc quản lý Executors và quá trình Shuffle dữ liệu giữa các node. Điều này khẳng định rằng công nghệ phân tán chỉ thực sự trở nên cần thiết khi dữ liệu vượt quá giới hạn phân cứng của một máy đơn lẻ (Big Data), còn với dữ liệu nằm trong khả năng của RAM, giải pháp tuần tự vẫn là tối ưu nhất.

4. Ứng dụng thuật toán Random Forest trong Apache Spark:

4.1. Giới thiệu thuật toán phân loại Random Forest và khả năng tính toán song song:

4.1.1. Thuật toán Random Forest:

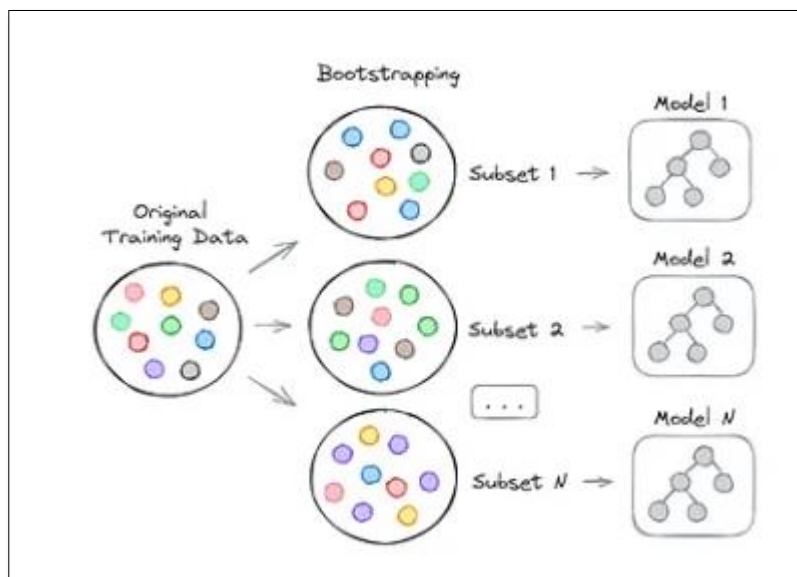
Random Forest là một thuật toán học máy thuộc nhóm học có giám sát (supervised learning) và được sử dụng phổ biến trong các bài toán phân loại (classification) và hồi quy (regression). Thuật toán này là một dạng của tập hợp học (ensemble learning), nơi mà nhiều mô hình yếu (weak learners), cụ thể là các cây quyết định (decision trees), được kết hợp lại để tạo thành một mô hình mạnh mẽ hơn.



Hình 34: Minh họa thuật toán Random Forest

Thuật ngữ “Random” trong Random Forest xuất phát từ hai yếu tố chính:

- Ngẫu nhiên trong chọn mẫu: thay vì sử dụng toàn bộ dữ liệu huấn luyện để xây dựng từng cây quyết định, thuật toán Random Forest chọn một mẫu ngẫu nhiên từ tập dữ liệu (có hoàn lại) để xây dựng mỗi cây. Kỹ thuật này được gọi là Bagging (Bootstrap Aggregating). Bagging giúp giảm thiểu phương sai của mô hình, cải thiện độ chính xác tổng thể.

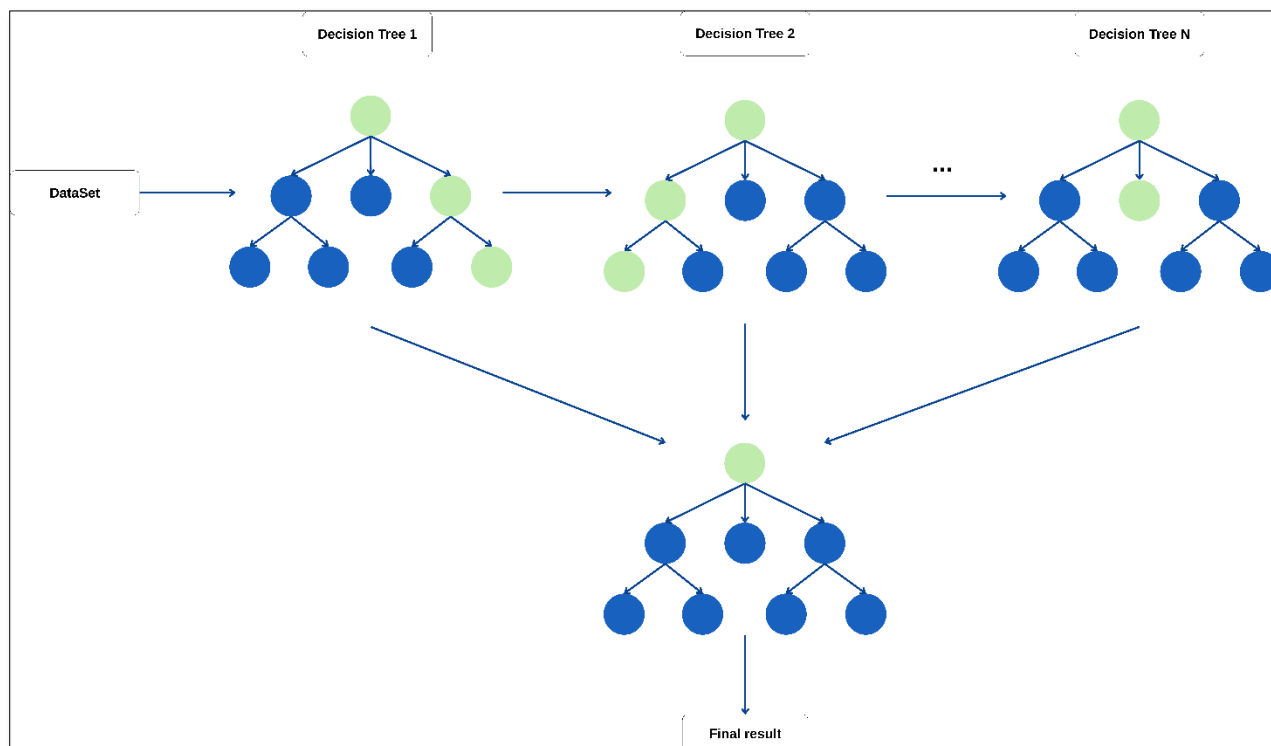


Hình 35: Kỹ thuật Bootstrap Aggregating

- Ngẫu nhiên trong chọn đặc trưng: khi tạo các nút trong mỗi cây, chỉ một tập con ngẫu nhiên của tất cả các đặc trưng được xem xét để chọn đặc trưng tốt nhất tại mỗi bước. Điều này giúp các cây quyết định đa dạng hơn, giảm thiểu hiện tượng

overfitting và đảm bảo rằng các cây không bị phụ thuộc quá mức vào một đặc trưng cụ thể nào đó.

Cơ chế hoạt động:



Hình 36: Kiến trúc tuần tự của Random Forest

Random Forest bao gồm nhiều cây quyết định (Decision Trees). Mỗi cây là một mô hình dự đoán độc lập và đưa ra một dự đoán. Đối với bài toán phân loại, Random Forest sẽ lấy kết quả dự đoán của từng cây và chọn kết quả nào xuất hiện nhiều nhất (majority vote). Đối với bài toán hồi quy, kết quả cuối cùng là giá trị trung bình của các dự đoán từ tất cả các cây.

Ví dụ, giả sử chúng ta có 100 cây quyết định. Đối với một mẫu mới, nếu 60 cây dự đoán rằng mẫu đó thuộc lớp A, và 40 cây dự đoán rằng mẫu đó thuộc lớp B, thì Random Forest sẽ dự đoán rằng mẫu đó thuộc lớp A (vì nó nhận được số phiếu cao hơn).

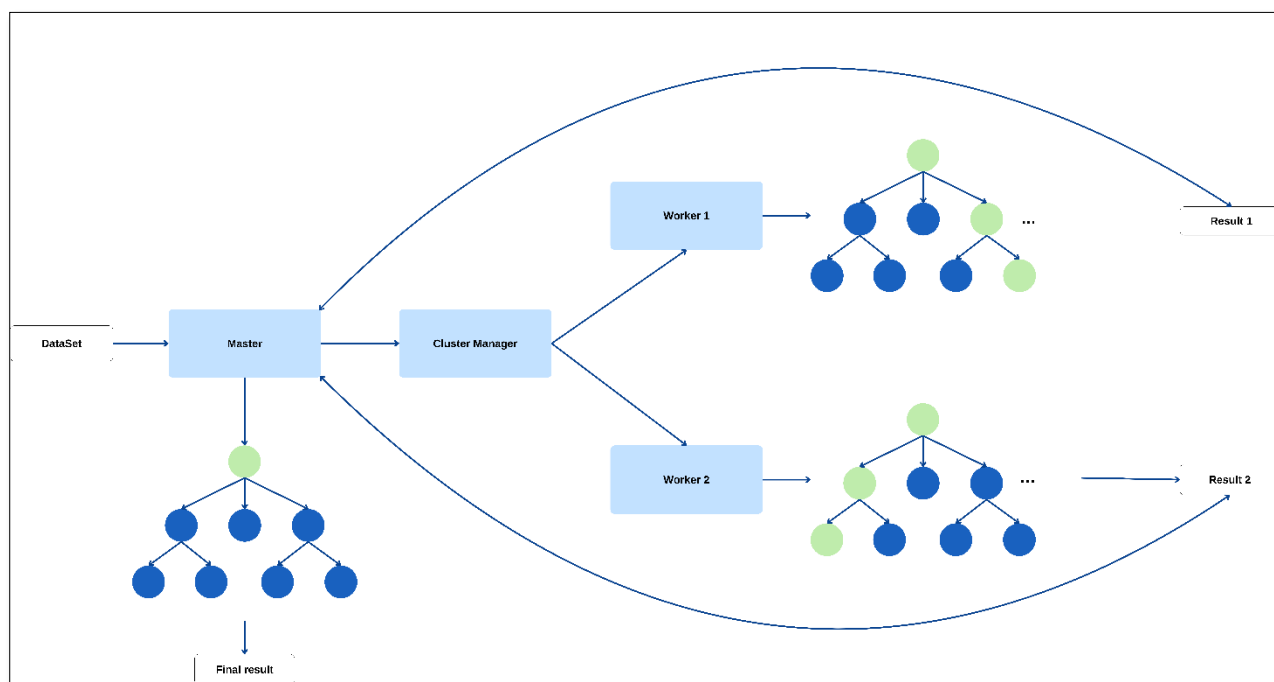
$$\hat{y} = \text{majority_vote}(\hat{y}^{(1)}, \hat{y}^{(2)}, \hat{y}^{(3)}, \dots, \hat{y}^{(n)})$$

Một cây quyết định trong Random Forest thực hiện phân loại hoặc hồi quy bằng cách chia nhỏ không gian đặc trưng thành các vùng con. Các phân vùng này được xác định dựa trên các điều kiện phân tách tại mỗi nút trong cây. Giả sử có một đặc trưng X và một ngưỡng phân t , việc phân tách tại một nút có thể được biểu diễn bằng cách chọn một hàm chỉ thị I , trong đó:

$$I(X \leq t) \text{ và } I(X > t)$$

Nếu đặc trưng X tại mẫu đó nhỏ hơn hoặc bằng t , mẫu sẽ được chuyển đến nhánh trái; ngược lại, nó sẽ được chuyển đến nhánh phải. Quá trình này tiếp tục cho đến khi đạt đến một nút lá, nơi giá trị đầu ra của nút đó được sử dụng làm dự đoán.

4.1.2. Kiến trúc thuật toán phân loại Random Forest song song:



Hình 37: Kiến trúc song song của Random Forest

Mô hình Random Forest khi triển khai trên Spark thực chất là sự kết hợp giữa thuật toán rừng cây quyết định và cơ chế xử lý song song – phân tán của Spark MLlib. Về bản chất, Random Forest là một tập hợp (ensemble) nhiều cây quyết định, mỗi cây được huấn luyện trên một mẫu con bootstrap của dữ liệu và một tập con ngẫu nhiên các thuộc tính tại mỗi nút chia. Kết quả dự báo cuối cùng được lấy theo nguyên tắc đa số phiếu. Khi đưa lên Spark, toàn bộ dữ liệu huấn luyện trước hết được lưu trữ dưới dạng RDD/DataFrame và phân tán thành nhiều partition trên các worker node trong cluster. Spark song song hóa Random Forest ở hai tầng:

(1) song song giữa các cây – các cây trong rừng là gần như độc lập nên có thể được phân chia thành nhiều tác vụ và huấn luyện đồng thời trên các executor khác nhau.

(2) song song bên trong từng cây – ở mỗi tầng của cây, việc tính toán các thống kê cần thiết để chọn điểm chia (số mẫu thuộc từng lớp, các tổng/độ sai lệch theo từng thuộc tính, từng ngưỡng) được thực hiện cục bộ trên từng partition rồi được Spark gom và cộng dồn lại theo cơ chế Map–Reduce để tìm split tối ưu cho từng nút.

Nhờ cách tổ chức này, Spark cho phép Random Forest tận dụng tối đa tài nguyên tính toán của nhiều lõi CPU và nhiều máy, xử lý hiệu quả các tập dữ liệu có kích thước lớn vượt quá khả năng bộ nhớ của một máy đơn lẻ, đồng thời thừa hưởng các ưu điểm của nền tảng như khả năng chịu lỗi, cache dữ liệu trên bộ nhớ và tích hợp dễ dàng với các bước tiền xử lý trong Pipeline. Tuy nhiên, tương tự các mô hình song song khác, chi phí khởi tạo phiên làm việc, phân tán và trao đổi dữ liệu giữa các node chỉ thực sự “được bù lại” khi kích thước dữ liệu đủ lớn. Với các tập dữ liệu nhỏ, Random Forest chạy cục bộ trên một máy đôi khi vẫn cho thời gian thực thi ngắn hơn so với khi triển khai qua Spark.

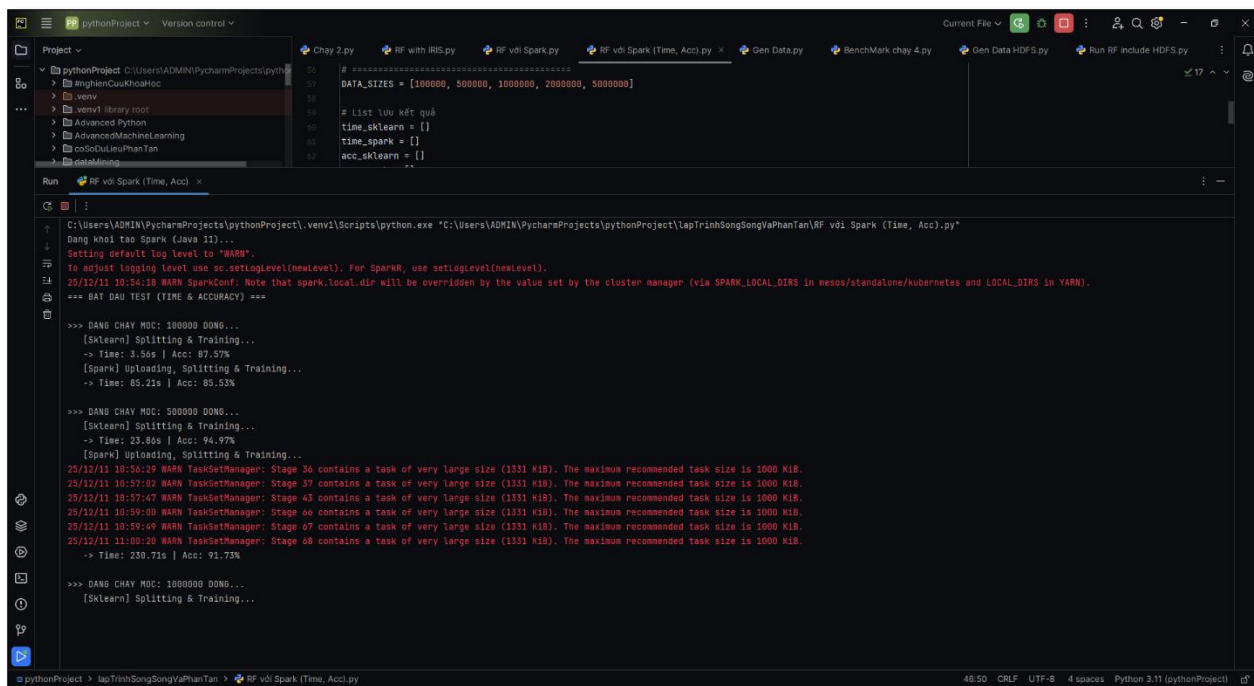
4.2. Triển khai Random Forest trong Spark và so sánh hiệu năng:

Tiến hành tạo một bộ dữ liệu mô phỏng gồm 4 đặc trưng và 1 thuộc tính phân loại. Thực hiện đồng thời thuật toán Random Forest tuần tự dựa trên thư viện Scikit-Learn và thuật toán Random Forest trên thư viện MLlib của Apache Spark. Bộ dữ liệu mô phỏng có dạng.

ID	F1	F2	F3	F4	Label
0	-0.405342	1.057510	-0.636091	0.457455	1
1	0.253490	2.547238	0.574340	-0.611829	1
2	-0.976033	1.727682	-2.004783	2.200316	0
3	-2.348048	-1.187137	-1.897594	-0.899520	0
4	0.847096	1.285691	-0.171944	-0.479354	1

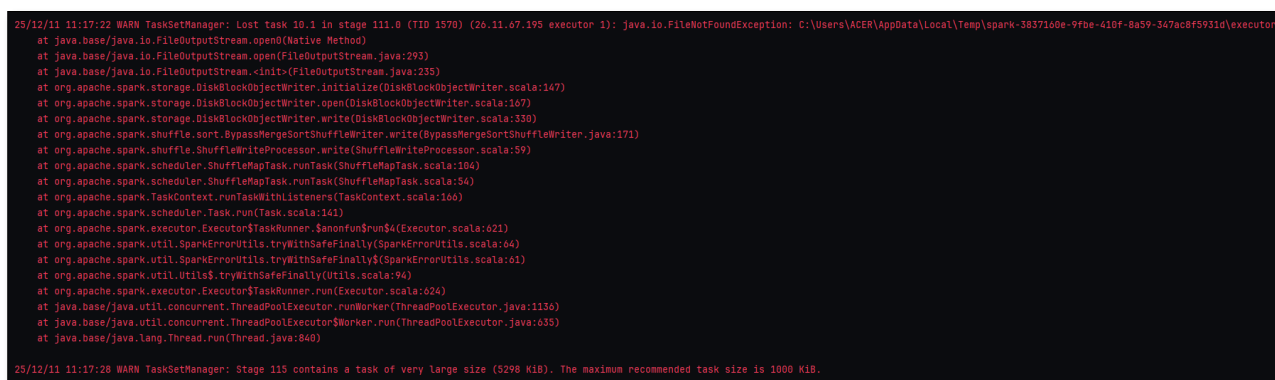
Bảng 5: Dữ liệu mô phỏng

Tiến hành huấn luyện dữ liệu với các nút Worker. Trong đánh giá mô hình và nền tảng xử lý như Spark, việc xem xét đồng thời cả độ chính xác và thời gian thực thi là yêu cầu bắt buộc, bởi hai chỉ số này trả lời cho hai nhóm câu hỏi khác nhau. Các chỉ số về độ chính xác (accuracy, F1-score, AUC...) cho biết mô hình dự đoán đúng đến mức nào, tức là phản ánh chất lượng chuyên môn của kết quả. Một mô hình chạy rất nhanh nhưng dự đoán sai nhiều thì gần như không có giá trị ứng dụng. Ngược lại, thời gian huấn luyện và thời gian dự đoán phản ánh hiệu năng hệ thống và khả năng đáp ứng yêu cầu thực tế. Một mô hình có độ chính xác cao nhưng huấn luyện quá chậm, tiêu tốn nhiều tài nguyên hoặc không mở rộng được khi dữ liệu tăng thì cũng khó triển khai trong môi trường sản xuất. Bởi vậy, trong các thí nghiệm so sánh giữa chạy cục bộ và chạy song song trên Spark, việc báo cáo đồng thời thời gian và độ chính xác giúp làm rõ “cái được” và “cái mất” của từng lựa chọn, từ đó rút ra kết luận hợp lý rằng với quy mô dữ liệu nhỏ có thể ưu tiên mô hình chạy local cho nhanh gọn, còn với dữ liệu lớn hoặc yêu cầu mở rộng thì chấp nhận chi phí thời gian và tài nguyên để đổi lấy khả năng xử lý phân tán.



Hình 38: Log hiển thị quá trình chạy Random Forest

Trong quá trình huấn luyện mô hình Random Forest với tập dữ liệu được nhân bản lên mức 5 triệu dòng, hệ thống Spark đã ghi nhận nhiều cảnh báo từ TaskSetManager về việc kích thước tác vụ (task size) đạt mức 1331 KiB, vượt quá ngưỡng khuyến nghị tiêu chuẩn là 1000 KiB. Hiện tượng này xuất phát từ cơ chế hoạt động của Spark MLlib khi phải đóng gói và tuần tự hóa (serialize) một lượng lớn metadata của các cây quyết định cùng thông tin phân vùng dữ liệu để gửi từ Driver xuống các Executor. Mặc dù các cảnh báo này không gây gián đoạn quá trình thực thi (non-fatal), nhưng chúng phản ánh chi phí quản lý (overhead) đáng kể về mặt mạng và tài nguyên CPU mà hệ thống phân tán phải gánh chịu khi xử lý các thuật toán học máy phức tạp trên tập dữ liệu lớn, góp phần giải thích cho độ trễ khởi tạo được quan sát thấy trong biểu đồ thời gian.



Hình 39: Quá trình cảnh báo dữ liệu bị lác mắt quá trình truyền tải đến nút worker

Dữ liệu log thực thi cung cấp bằng chứng rõ ràng về điểm chịu tải của hệ thống. Cụ thể, tại các giai đoạn (Stage) xử lý mốc dữ liệu cực đại, Spark liên tục phát ra cảnh báo về các tác vụ có kích thước lớn (~1.3MB). Điều này cho thấy khi dữ liệu đầu vào tăng lên, khối lượng thông tin điều phối mà Node Master phải truyền tải cho các Node Worker cũng tăng theo cấp số nhân, tạo ra các nút thắt cổ chai (bottleneck) tạm thời trong giao tiếp mạng. Việc hệ thống vẫn hoàn thành tác vụ dù xuất hiện cảnh báo chứng tỏ khả năng chịu lỗi (fault tolerance) tốt của Spark, nhưng đồng thời cũng xác nhận rằng kiến trúc phân tán đang phải đánh đổi thời gian giao tiếp để xử lý khối lượng tính toán mà một máy đơn lẻ không thể đảm nhận hiệu quả.

Spark Master at spark://26.163.90.8:7077

URL: spark://26.163.90.8:7077
 Alive Workers: 2
 Cores in use: 24 Total, 24 Used
 Memory in use: 37.6 GiB Total, 8.0 GiB Used
 Resources in use:
 Applications: 1 Running, 13 Completed
 Drivers: 0 Running, 0 Completed
 Status: ALIVE

Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20251211084024-26.11.67.195-58788	26.11.67.195:58788	ALIVE	8 (8 Used)	6.9 GiB (4.0 GiB Used)	
worker-20251211100917-26.225.39.97-54565	26.225.39.97:54565	ALIVE	16 (16 Used)	30.7 GiB (4.0 GiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-2025121111948-0013	(kill) Benchmark_Time_Vis_Accuracy	24	4.0 GiB		2025/12/11 11:19:48	ADMIN	RUNNING	30 min

Completed Applications (13)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20251211105418-0012	Benchmark_Time_Vis_Accuracy	24	4.0 GiB		2025/12/11 10:54:18	ADMIN	FINISHED	24 min
app-20251211104551-0011	Benchmark_Iris_Upscaled	24	4.0 GiB		2025/12/11 10:45:51	ADMIN	FINISHED	8.4 min
app-20251211104411-0010	Benchmark_Time_Vis_Accuracy	24	4.0 GiB		2025/12/11 10:44:11	ADMIN	FINISHED	1.3 min
app-20251211100938-0009	Benchmark_WordCount_Output	24	4.0 GiB		2025/12/11 10:09:38	ADMIN	FINISHED	9 s
app-20251211100746-0008	Benchmark_WordCount_Output	8	4.0 GiB		2025/12/11 10:07:46	ADMIN	FINISHED	8 s
app-20251211100544-0007	Benchmark_WordCount_Output	24	4.0 GiB		2025/12/11 10:05:44	ADMIN	FINISHED	8 s
app-20251211100508-0006	Benchmark_WordCount_Output	24	4.0 GiB		2025/12/11 10:05:08	ADMIN	FINISHED	9 s
app-20251211100001-0005	WordCount_Advanced	24	4.0 GiB		2025/12/11 10:00:01	ADMIN	FINISHED	9 s
app-20251211095745-0004	Benchmark_WordCount_Output	24	4.0 GiB		2025/12/11 09:57:45	ADMIN	FINISHED	8 s
app-20251211095126-0003	WordCount_Advanced	24	4.0 GiB		2025/12/11 09:51:26	ADMIN	FINISHED	9 s
app-20251211091208-0002	StressTest_Upscale	24	4.0 GiB		2025/12/11 09:12:08	ADMIN	FINISHED	24 min
app-20251211085041-0001	Benchmark_WordCount_Output	24	4.0 GiB		2025/12/11 08:50:41	ADMIN	FINISHED	8 s

Hình 40: Giao diện quản lý tài nguyên

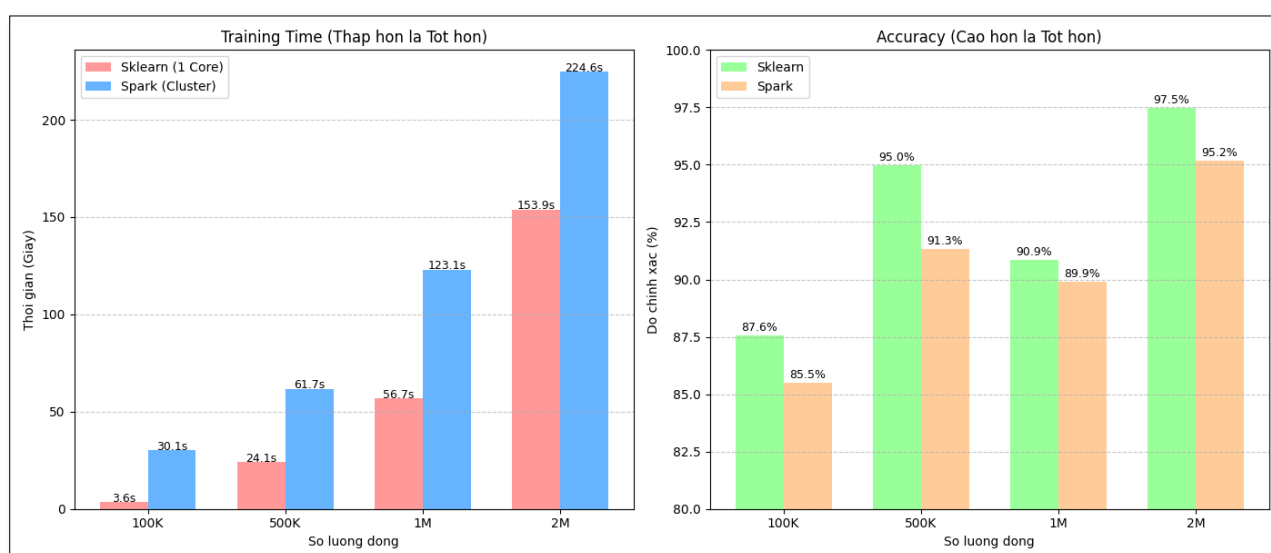
Độ lớn (dòng)	Thời gian chạy tuần tự lần 1	Thời gian chạy tuần tự lần 2	Thời gian chạy song song với Spark (cụm 1 worker node)	Thời gian chạy song song với Spark (cụm 2 workers node)
100000	3.6	3.71	30.1	89.74
500000	24.1	24.33	61.7	233.53
1000000	56.7	62.18	123.1	533.01
2000000	153.9	150.46	224.6	954.40

Bảng 6: So sánh các kết quả chạy (giây)

Độ lớn (dòng)	Thời gian chạy tuần tự lần 1	Thời gian chạy tuần tự lần 2	Thời gian chạy song song với Spark (cụm 1 worker node)	Thời gian chạy song song với Spark (cụm 2 workers node)
100000	87.6	87.57	85.5	85.52
500000	95.0	94.97	91.3	91.43
1000000	90.9	90.86	89.9	89.97
2000000	97.5	97.49	95.2	94.51

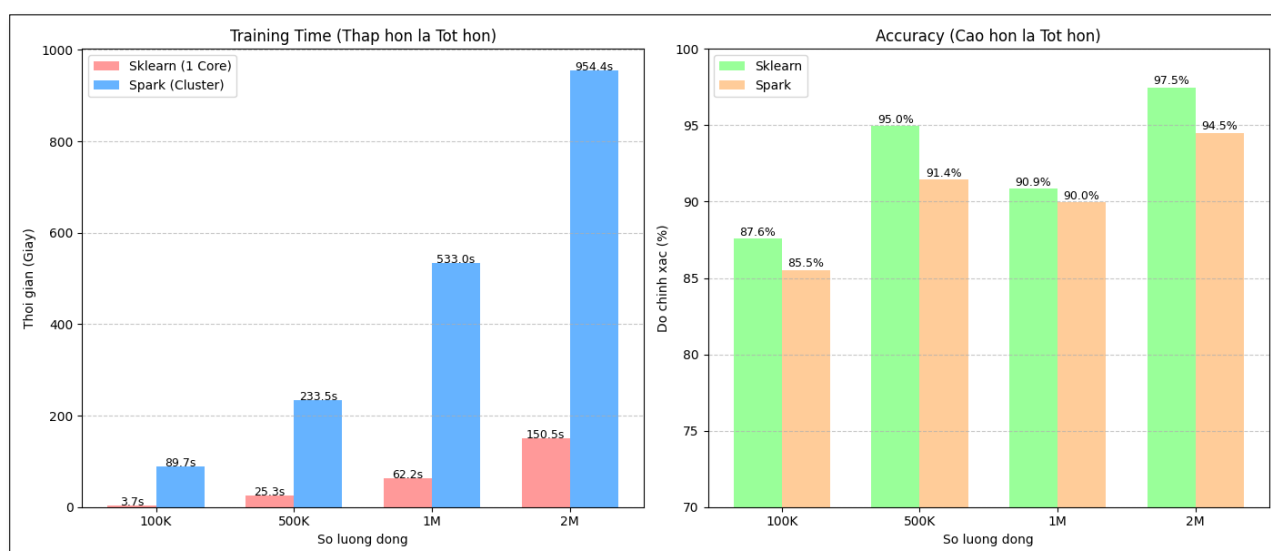
Bảng 7: So sánh độ chính xác (%)

Khi thực hiện tính toán, giao diện quản lý tài nguyên cho thấy các cores được dùng hết cho việc tính toán. Tuy nhiên, số liệu từ Bảng 5 minh chứng rõ rệt cho sự đánh đổi giữa chi phí quản lý và tốc độ tính toán đối với các tập dữ liệu kích thước trung bình. Trong mọi mức kiểm thử từ 100.000 đến 2 triệu dòng, giải pháp xử lý tuần tự luôn chiếm ưu thế tuyệt đối về thời gian hoàn thành so với Spark Cluster. Điểm đáng chú ý nhất là hiện tượng 'mở rộng ngược' (negative scaling): khi tăng tài nguyên từ 1 worker lên 2 workers tại mức 2 triệu dòng, thời gian xử lý không những không giảm mà còn tăng vọt từ 224,6 giây lên 845,14 giây. Điều này khẳng định rằng với lượng dữ liệu chưa vượt quá giới hạn bộ nhớ của một máy đơn, chi phí giao tiếp mạng (network overhead) để điều phối và xáo trộn dữ liệu (shuffling) giữa các node trong mô hình phân tán đã trở thành nút thắt cổ chai, khiến hiệu năng tổng thể thấp hơn nhiều so với việc xử lý cục bộ.



Hình 41: So sánh thời gian và độ chính xác (1 worker node)

Bảng 6 cho thấy độ chính xác của mô hình Random Forest ở bốn quy mô dữ liệu khác nhau khi chạy tuần tự và khi chạy song song trên Spark. Có thể thấy, với mỗi độ lớn dữ liệu, hai lần chạy tuần tự cho kết quả rất ổn định (chênh lệch chỉ 0,01 đến 0,03%), chứng tỏ quá trình huấn luyện cục bộ không bị dao động nhiều do yếu tố ngẫu nhiên. Khi chuyển sang Spark (cụm 1 worker node và cụm 2 workers node), độ chính xác nhìn chung giảm nhẹ so với chạy tuần tự, mức chênh lệch khoảng 2 đến 4% tùy kích thước dữ liệu, nhưng các giá trị vẫn giữ được cùng “mặt bằng” và không xuất hiện sai lệch lớn. Điều này cho thấy việc song song hóa trên Spark không làm thay đổi bản chất của mô hình, chỉ tạo ra dao động nhỏ do cơ chế chia dữ liệu và ngẫu nhiên hóa trong quá trình huấn luyện phân tán. Đáng chú ý, giữa cấu hình 1 worker và 2 workers, độ chính xác gần như tương đương, chứng tỏ việc tăng thêm node chủ yếu tác động đến thời gian xử lý hơn là cải thiện chất lượng mô hình. Từ các số liệu này có thể kết luận: về mặt độ chính xác, Random Forest trên Spark vẫn bảo toàn được chất lượng dự báo so với mô hình chạy tuần tự, do đó có thể chấp nhận sử dụng khi cần mở rộng quy mô xử lý dữ liệu lớn.



Hình 42: So sánh thời gian và độ chính xác (2 workers node)

5. Phương pháp cải thiện tốc độ xử lý:

Trong các hệ thống xử lý song song và phân tán (như HDFS + Spark), data mirroring là kỹ thuật nhân bản/“phản chiếu” dữ liệu: mỗi phân vùng (block/partition) không chỉ được lưu trên một nút mà có nhiều bản sao trên các nút khác nhau. Khi đó, bộ lập lịch (scheduler) sẽ ưu tiên bố trí task chạy tại nút chứa bản sao dữ liệu “gần nhất” (data locality – local, rack-local), thay vì phải luôn luôn tải dữ liệu từ một máy trung tâm qua mạng. Nhờ vậy, phần lớn luồng truy cập dữ liệu được thực hiện ngay trên ổ đĩa nội bộ hoặc trong cùng rack, lưu lượng truyền qua switch/routing giảm xuống, độ trễ truy xuất nhỏ hơn và băng thông mạng dành cho các tác vụ shuffle, giao tiếp điều khiển sẽ “dễ thở” hơn.

Để kỹ thuật data mirroring thực sự giúp giảm đường tải qua mạng, hệ thống phải đáp ứng một số yêu cầu sau:

- Hệ số nhân bản hợp lý (replication factor): thường là 2 hoặc 3, đủ để mỗi block có nhiều bản sao trên các nút khác nhau, nhưng không quá lớn để tránh lãng phí dung lượng lưu trữ.

- Phân bố bản sao có chủ đích: các bản sao cần được trải trên nhiều nút và nhiều rack khác nhau (rack-aware placement) để vừa đảm bảo khả năng chịu lỗi, vừa tạo nhiều cơ hội lập lịch task “gần dữ liệu”.
- Scheduler hỗ trợ data locality: bộ lập lịch của Spark/YARN phải biết vị trí các bản sao và ưu tiên gán task lên node có dữ liệu local (hoặc rack-local), chỉ rơi xuống “ANY” khi không còn tài nguyên phù hợp.
- Cơ chế đồng bộ và quản lý nhất quán: khi dữ liệu được cập nhật (trong các hệ thống có ghi), phải có cơ chế đảm bảo các bản sao không bị lệch nhau, hoặc chấp nhận mô hình “write once, read many” để đơn giản hóa.
- Hạ tầng lưu trữ đáp ứng được chi phí nhân bản: vì có thêm bản sao nên dung lượng đĩa sử dụng tăng, hệ thống phải tính toán trước về chi phí lưu trữ, backup, giám sát.

Ưu điểm của data mirroring là vừa giảm lưu lượng mạng, vừa tăng khả năng chịu lỗi (một node hỏng vẫn còn bản sao trên node khác), đồng thời cải thiện độ ổn định của các job Spark dài hạn. Tuy nhiên, kỹ thuật này cũng có đánh đổi: tốn thêm dung lượng lưu trữ, thời gian ban đầu để phân phối/nhân bản dữ liệu, và độ phức tạp trong quản lý vị trí – trạng thái bản sao. Vì vậy khi thiết kế hệ thống, cần cân nhắc đồng thời ba yếu tố: mức độ nhân bản, cấu trúc mạng (số rack, băng thông giữa rack) và đặc thù bài toán (đọc nhiều, ghi ít hay ngược lại) để lựa chọn cấu hình data mirroring tối ưu.

5.1. Data mirroring thủ công:

Mirroring thủ công (Manual Mirroring) thực chất là cách gọi kỹ thuật của việc con người tự tay sao chép dữ liệu từ nơi này sang nơi khác để tạo ra một bản sao. Tuy nhiên để chạy Apache Spark trên chế độ cụm (Cluster Mode) mà sử dụng phương pháp Mirroring thủ công (tức là không dùng HDFS, mà dùng file cục bộ file:///), bắt buộc phải tuân thủ 3 Nguyên tắc Vàng sau đây để hệ thống không bị báo lỗi "File Not Found". Nếu vi phạm bất kỳ nguyên tắc nào, Worker sẽ không tìm thấy dữ liệu mà Driver yêu cầu.

1. Đồng bộ Đường dẫn Tuyệt đối (Same Absolute Path): Đây là yêu cầu quan trọng nhất.

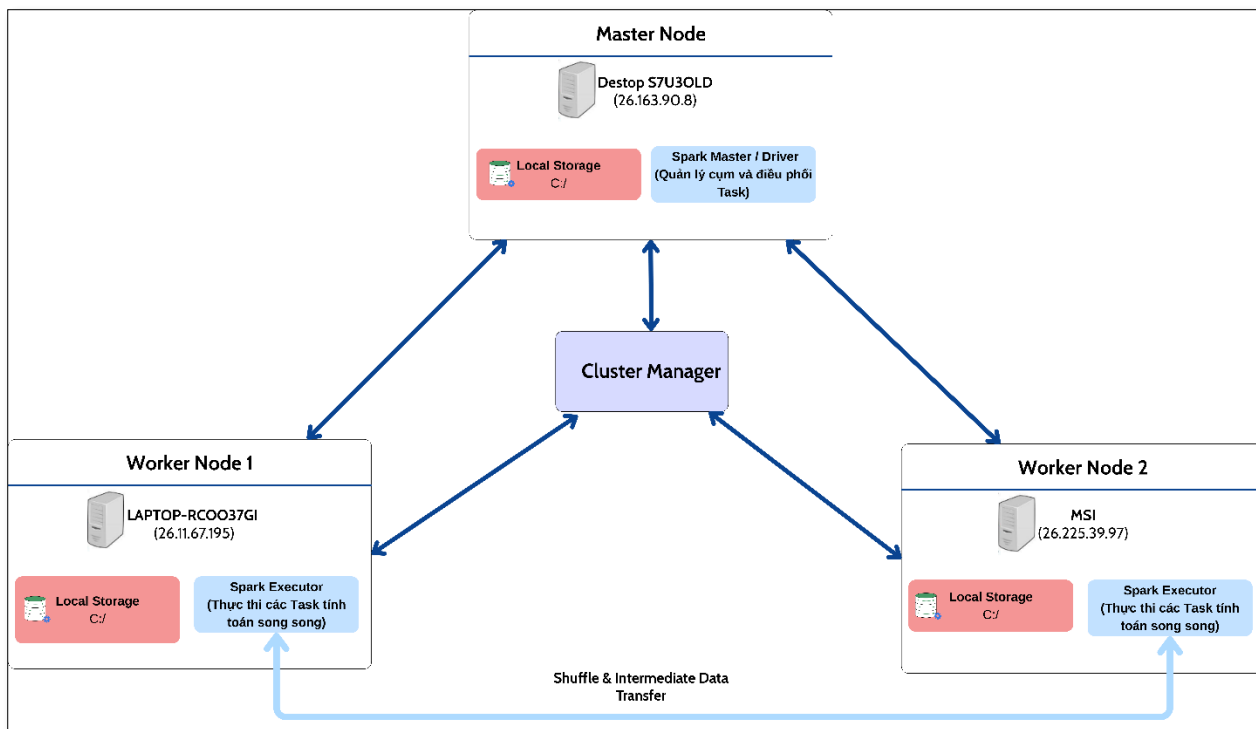
Nếu trên máy Master, dữ liệu nằm ở C:\DataTest\data.csv, thì trên tất cả các máy Worker, dữ liệu cũng phải nằm chính xác ở C:\DataTest\data.csv. Vì khi Spark Driver gửi lệnh cho Worker, nó chỉ gửi đường dẫn chuỗi ký tự (String path). Nếu Worker tìm không thấy thư mục đó, nó sẽ báo lỗi ngay lập tức. Lỗi thường gặp là khi máy Master để ở ổ D:\Data, nhưng máy Worker lại để ở C:\Data hệ thống sẽ báo lỗi.

2. Đồng bộ Nội dung (Content Consistency)

File dữ liệu ở tất cả các máy phải giống hệt nhau về kích thước và nội dung. Nếu cập nhật file trên Master (ví dụ thêm 100 dòng) mà quên copy thủ công sang các Worker, Spark có thể đọc sai offset (vị trí byte). Kết quả tính toán sẽ bị sai hoặc crash chương trình.

3. Đồng bộ Môi trường (Environment Consistency)

Không chỉ dữ liệu, mà code và thư viện cũng phải giống nhau. Tất cả các máy phải cài cùng phiên bản Python (Ví dụ cùng là 3.11). Nếu code của bạn dùng pandas, numpy, sklearn, thì tất cả các máy Worker đều phải được cài các thư viện này (pip install ...). Nếu Master có mà Worker không có sẽ xảy ra lỗi ModuleNotFoundError.



Hình 43: Các file dữ liệu được đặt sẵn ở từng máy

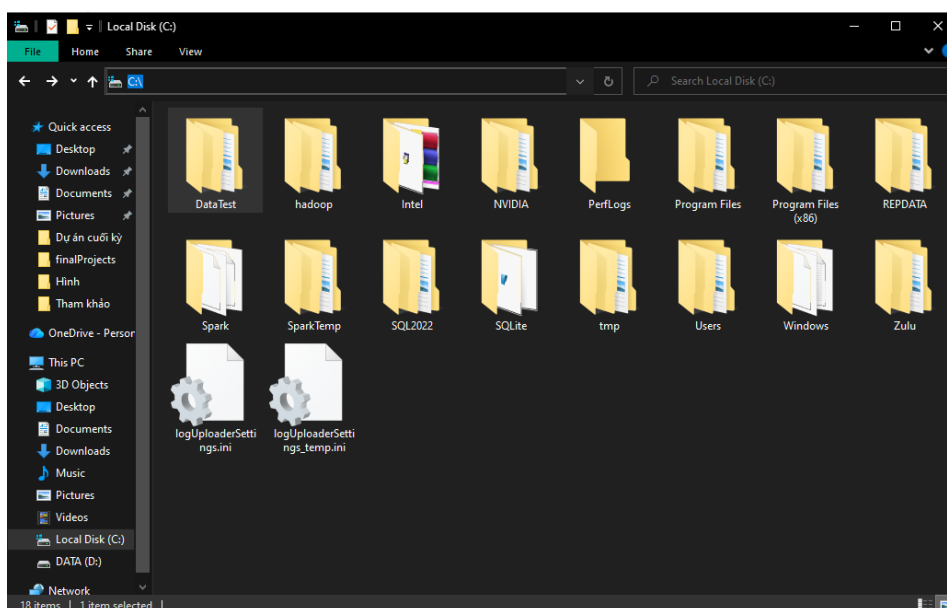
Thực hiện sinh dữ liệu ngẫu nhiên tương tự ở trên.

```
Dang khoi tao Spark...
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).

=== DANG TAO DU LIEU VA0: C:/DataTest ===

...Dang xu ly m6c 100000 dòng...
25/12/11 13:56:01 WARN TaskSetManager: Stage 0 contains a task of very large size (4015 KiB). The maximum recommended task size is 1000 KiB.
-> Da lưu xong: C:/DataTest/data_100000
```

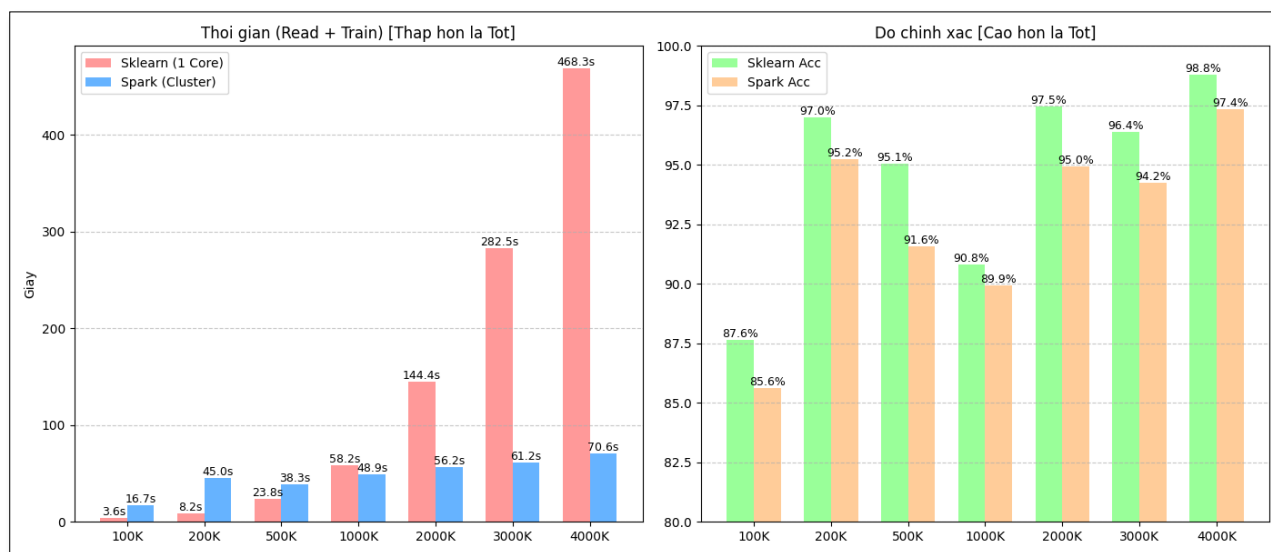
Hình 44: Log trạng thái sinh dữ liệu mẫu



Hình 45: Dữ liệu sinh ra được đặt cố định ở C

Độ lớn (dòng)	Thời gian chạy tuần tự (giây)	Thời gian chạy song song (24 cores) (giây)	Độ chính xác chạy tuần tự (%)	Độ chính xác chạy song song (24 cores) (%)
100000	3.60	16.69	87.64	85.60
200000	8.24	45.04	96.98	95.25
500000	23.84	38.34	95.05	91.56
1 triệu	58.19	48.87	90.83	89.93
2 triệu	144.43	56.23	97.46	94.95
3 triệu	282.47	61.19	96.38	94.23
4 triệu	468.32	70.63	98.80	97.36

Bảng 8: So sánh tốc độ chạy và độ chính xác



Hình 46: Kết quả chạy Random Forest với Data Mirroring

Kết quả trên cho thấy bức tranh tương đối đầy đủ về sự đánh đổi giữa tốc độ xử lý và độ chính xác khi huấn luyện mô hình Random Forest theo hai cách: chạy tuần tự và chạy song song trên Spark (24 cores). Với các tập dữ liệu nhỏ đến trung bình (100.000–500.000 dòng), thời gian chạy tuần tự luôn ngắn hơn đáng kể so với chạy song song. Ở mức 100.000 dòng, chạy tuần tự chỉ mất 3,60 giây trong khi Spark mất tới 16,69 giây. Ở mức 200.000 và 500.000 dòng, thời gian trên Spark lần lượt gấp khoảng 5 lần và 1,6 lần so với chạy tuần tự. Điều này phản ánh rõ chi phí “đầu vào” khi sử dụng Spark (khởi tạo phiên làm việc, phân tán dữ liệu, shuffle giữa các node) lớn hơn rất nhiều so với chi phí tính toán bản thân mô hình, nên ở quy mô nhỏ, xử lý trên một máy đơn là lựa chọn hiệu quả hơn. Tuy nhiên, từ ngưỡng khoảng 1 triệu dòng trở lên, xu hướng đảo chiều: thời gian chạy song song bắt đầu nhỏ hơn thời gian tuần tự và chênh lệch tăng dần khi dữ liệu lớn lên. Ở mức 4 triệu dòng, chạy tuần tự mất

468,32 giây trong khi Spark chỉ còn 70,63 giây, tức là nhanh hơn khoảng 6–7 lần, cho thấy ưu thế mở rộng (scalability) rất rõ rệt của mô hình phân tán khi khối lượng dữ liệu đủ lớn.

Về độ chính xác, kết quả cho thấy mô hình chạy tuần tự luôn nhanh hơn so với mô hình chạy song song từ khoảng 1–3%, nhưng cả hai phương án đều duy trì mức chính xác tương đối cao (đa số trên 90%). Sự khác biệt này chủ yếu đến từ các yếu tố ngẫu nhiên trong quá trình lấy mẫu bootstrap và chia dữ liệu trong môi trường phân tán, chứ không phải do Spark làm thay đổi bản chất thuật toán. Vì vậy có thể coi mức suy giảm độ chính xác là chấp nhận được trong bối cảnh thu được lợi ích rất lớn về thời gian ở các bộ dữ liệu hàng triệu bản ghi. Từ các số liệu trong bảng, có thể rút ra kết luận: với các tập dữ liệu nhỏ và vừa, nên ưu tiên chạy tuần tự để tránh overhead của Spark; còn với dữ liệu lớn (từ khoảng 1–2 triệu dòng trở lên), việc triển khai Random Forest trên Spark giúp rút ngắn đáng kể thời gian xử lý, trong khi vẫn bảo toàn được chất lượng mô hình ở mức tốt.

Ưu điểm:

- Giảm tải đọc dữ liệu từ máy trung tâm: Mỗi worker có một bản dữ liệu cục bộ, nên khi chạy job, phần dữ liệu thô (file CSV, Parquet, ảnh...) có thể được đọc trực tiếp từ đĩa local, hạn chế phải kéo toàn bộ dữ liệu qua mạng tại thời điểm thực thi.
- Triển khai đơn giản cho cụm nhỏ: Trong môi trường ít máy, dữ liệu ít thay đổi, việc copy file bằng scp, rsync, USB, share folder... có thể làm rất nhanh mà không cần triển khai cả HDFS hay hệ thống lưu trữ phân tán phức tạp.
- Không phụ thuộc hạ tầng Hadoop/HDFS: Phù hợp trong bối cảnh chỉ có Spark standalone hoặc cụm máy “thô”, chưa cài đặt đầy đủ stack Hadoop nhưng vẫn muốn tận dụng một phần lợi ích của “data locality” (đọc local thay vì remote).

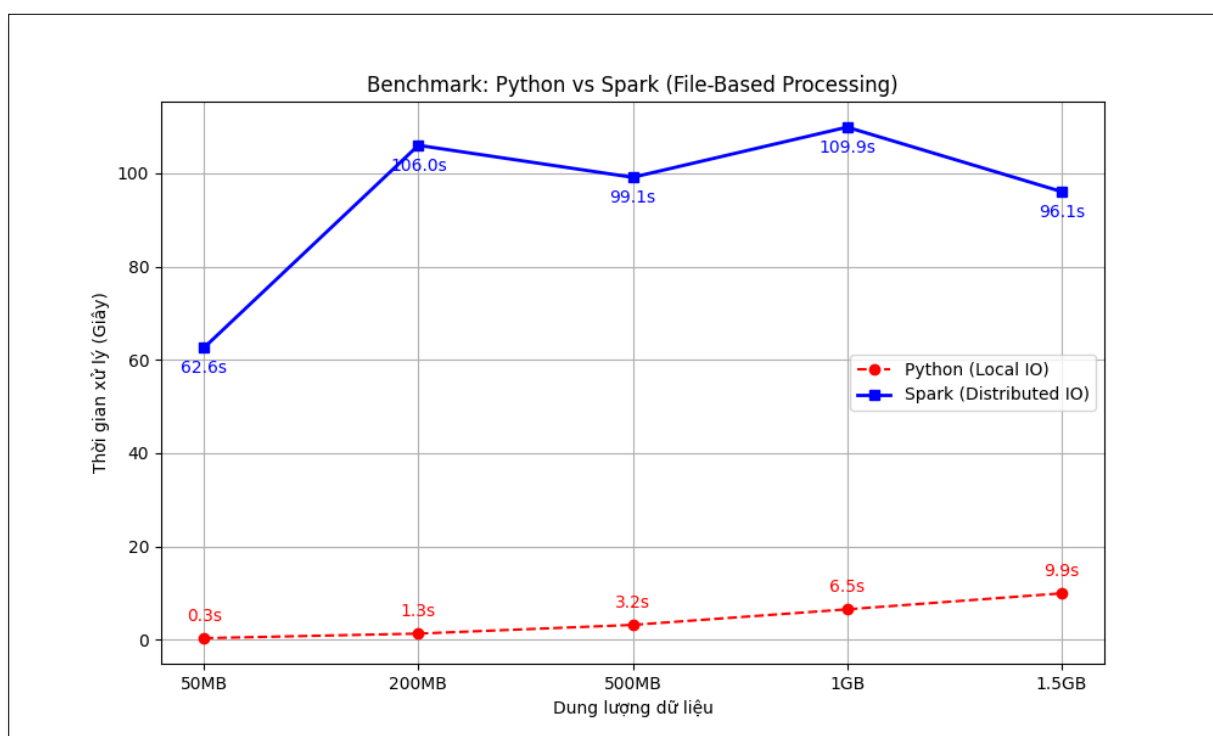
Nhược điểm:

- Quản lý khó khăn, dễ lỗi: Mỗi lần cập nhật dữ liệu, phải tự copy lại lên tất cả node. Rất dễ xảy ra tình trạng mỗi máy một phiên bản, thiếu đồng bộ, khó kiểm soát được toàn bộ cụm đang dùng bản dữ liệu nào.
- Không có cơ chế chịu lỗi tự động: Nếu một bản local bị hỏng, bị xóa nhầm hoặc đĩa lỗi, hệ thống không tự động chuyển sang bản sao khác như HDFS; việc khôi phục hoàn toàn phụ thuộc vào thao tác thủ công.
- Không tối ưu hóa phân bố dữ liệu: Mirroring thủ công thường là “copy đều đều”, không có thuật toán phân bố theo rack/node. Vì vậy, Spark/YARN cũng khó tận dụng triệt để cơ chế lập lịch theo locality như khi dùng HDFS.
- Khó mở rộng khi cụm lớn dần: Số node càng nhiều thì việc sao chép thủ công càng cực, tốn thời gian, dễ sai sót. Khi mở rộng cluster, bạn lại phải tổ chức copy lại toàn bộ dữ liệu cho các node mới.
- Không tích hợp chặt chẽ với công cụ quản lý cụm: Các hệ thống như HDFS cung cấp sẵn metadata, replication factor, giám sát block, báo lỗi node, trong khi mirroring thủ công hoàn toàn thiếu các cơ chế quản trị này.

Phương pháp mirroring thủ công chỉ phù hợp với các cụm nhỏ, dữ liệu ít thay đổi, yêu cầu đơn giản; còn với các hệ thống xử lý dữ liệu lớn, đòi hỏi khả năng chịu lỗi và mở rộng, việc sử dụng HDFS hoặc các giải pháp lưu trữ phân tán có cơ chế replication tự động là lựa chọn hiệu quả hơn.

Thử nghiệm lại Word count với Data Mirroring thủ công:

Độ lớn	Thời gian chạy tuần tự - giây	Thời gian chạy song song với Spark (cụm 2 workers node) - giây
50MB	0.3107	62.5923
200MB	1.2837	105.9914
500MB	3.1544	99.1294
1GB	6.4872	109.8846
1.5GB	9.9149	96.0728

Bảng 9: So sánh thời gian đếm sau khi đặt dữ liệu sẵn ở các máy*Hình 47: So sánh thời gian chạy*

Sau khi áp dụng data mirroring thủ công (đặt sẵn dữ liệu trên các máy worker), thời gian Spark giảm xuống mức ổn định hơn, dao động quanh 62 đến 110 giây cho các kích thước từ 50 MB đến 1,5 GB. Điều này chứng tỏ việc loại bỏ bước truyền dữ liệu từ máy Master trung tâm đã giúp giảm bớt chi phí I/O qua mạng. Tóm lại, kết quả thực nghiệm khẳng định rằng khi dữ liệu chưa đủ lớn, Word Count chạy tuần tự vẫn hiệu quả nhất. Data mirroring giúp cải thiện thời gian Spark so với truyền qua mạng thuần túy, nhưng không thể thay thế một hệ thống lưu trữ phân tán chuẩn. Và cuối cùng là việc lựa chọn giữa triển khai phân tán hay tuần tự phải luôn gắn với quy mô dữ liệu, cấu hình cụm và mục tiêu sử dụng thực tế.

5.2. Quản lý dữ liệu với HDFS:

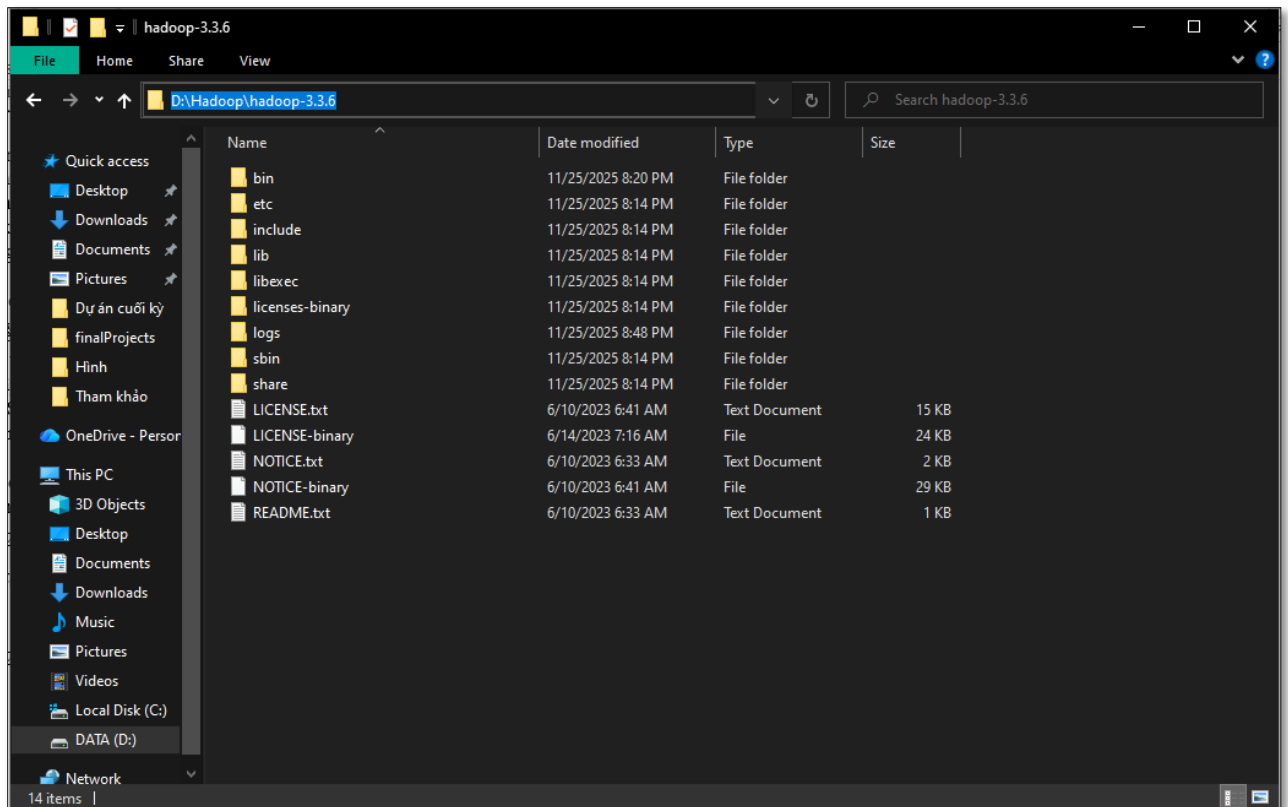
Hadoop Distributed File System (HDFS) là hệ thống tệp phân tán được thiết kế chuyên biệt cho việc lưu trữ và quản lý các tập dữ liệu rất lớn trong môi trường cụm máy (cluster). Thay vì lưu toàn bộ dữ liệu trên một máy đơn lẻ, HDFS chia file thành các khối (block) kích thước lớn (ví dụ 64MB, 128MB) và phân tán các block này lên nhiều DataNode trong cụm; đồng thời, mỗi block được nhân bản với một hệ số sao chép (replication factor) để đảm bảo an toàn dữ liệu và khả năng chịu lỗi: khi một node hỏng, hệ thống vẫn còn các bản sao trên node khác. Việc điều phối, quản lý siêu dữ liệu (metadata) như tên file, vị trí các block, quyền truy cập... được tập trung tại NameNode, còn DataNode chỉ chịu trách nhiệm lưu trữ và phục vụ dữ liệu. Nhờ cơ chế này, HDFS có thể mở rộng tuyến tính theo số lượng máy, hỗ trợ throughput đọc/ghi rất cao và cho phép các hệ thống tính toán như MapReduce, Spark đưa xử lý tới gần nơi dữ liệu được lưu (data locality), từ đó giảm tải truyền dữ liệu qua mạng. Trong các bài toán lập trình song song và phân tán, HDFS đóng vai trò là tầng lưu trữ nền tảng, giúp quản lý thống nhất dữ liệu đầu vào/đầu ra, hỗ trợ nhân bản, chịu lỗi và tạo tiền đề để các job Spark có thể xử lý dữ liệu lớn một cách hiệu quả và ổn định.

Để khắc phục những hạn chế về sự thiếu đồng bộ và rủi ro quản lý của phương pháp sao chép thủ công (Manual Mirroring), hệ thống chuyển sang sử dụng HDFS làm giải pháp lưu trữ tập trung. Khác với việc phải duy trì các bản sao vật lý riêng lẻ trên từng máy Worker, HDFS ảo hóa tài nguyên lưu trữ của toàn bộ cụm thành một không gian thống nhất, cho phép tự động phân chia và phân phối dữ liệu tới các node tính toán. Cơ chế này không chỉ loại bỏ hoàn toàn lỗi 'File Not Found' do sự bất nhất về đường dẫn, mà còn tối ưu hóa hiệu năng xử lý nhờ tận dụng tính cục bộ của dữ liệu (Data Locality), nơi Spark có thể ưu tiên xử lý dữ liệu ngay tại node đang lưu trữ nó, giảm thiểu đáng kể độ trễ truyền tải qua mạng.

5.2.1. Cài đặt và cấu hình HDFS:

Tiến hành tải và giải nén Hadoop 3.3.6 (Link: [Apache Hadoop](#)).

Vì Hadoop cũng yêu cầu các ứng dụng nền như Java 8 hoặc Java 11, cũng như các cấu hình biến môi trường tương tự với Spark. Do đó, các cấu hình khi cài đặt Hadoop cũng được giữ tương tự như Spark đã cài đặt ở phần trên.

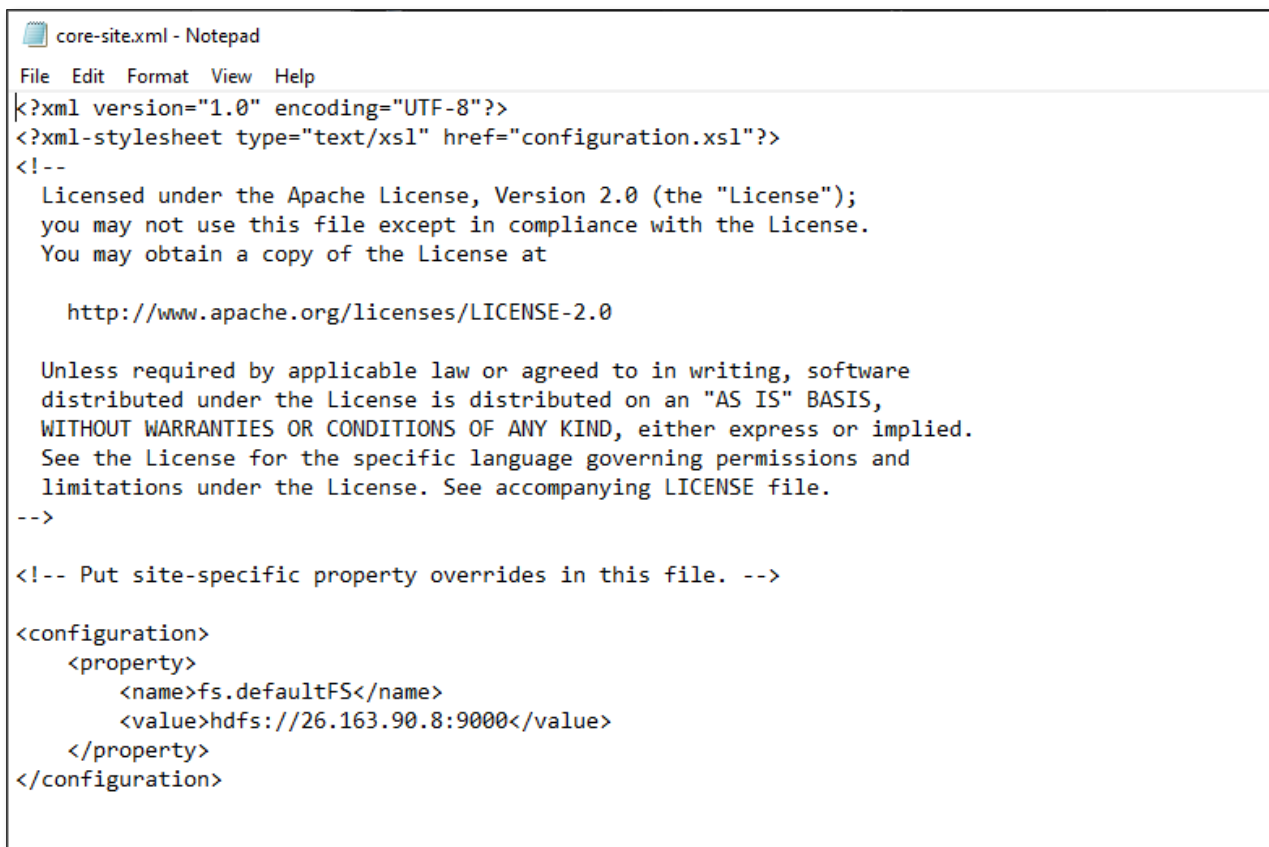


Hình 48: Thư mục Hadoop sau khi giải nén

Vào thư mục `D:\Hadoop\hadoop-3.3.6\etc\hadoop`, cần chú ý sửa 4 file quan trọng nhất:

1. File `core-site.xml` (Chỉ định Master node): File này báo cho toàn hệ thống biết địa chỉ của NameNode (Master). Do đó thay bằng IP máy Master muốn làm NameNode.

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://26.163.90.8:9000</value>
  </property>
</configuration>
```



```

core-site.xml - Notepad
File Edit Format View Help
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://26.163.90.8:9000</value>
  </property>
</configuration>

```

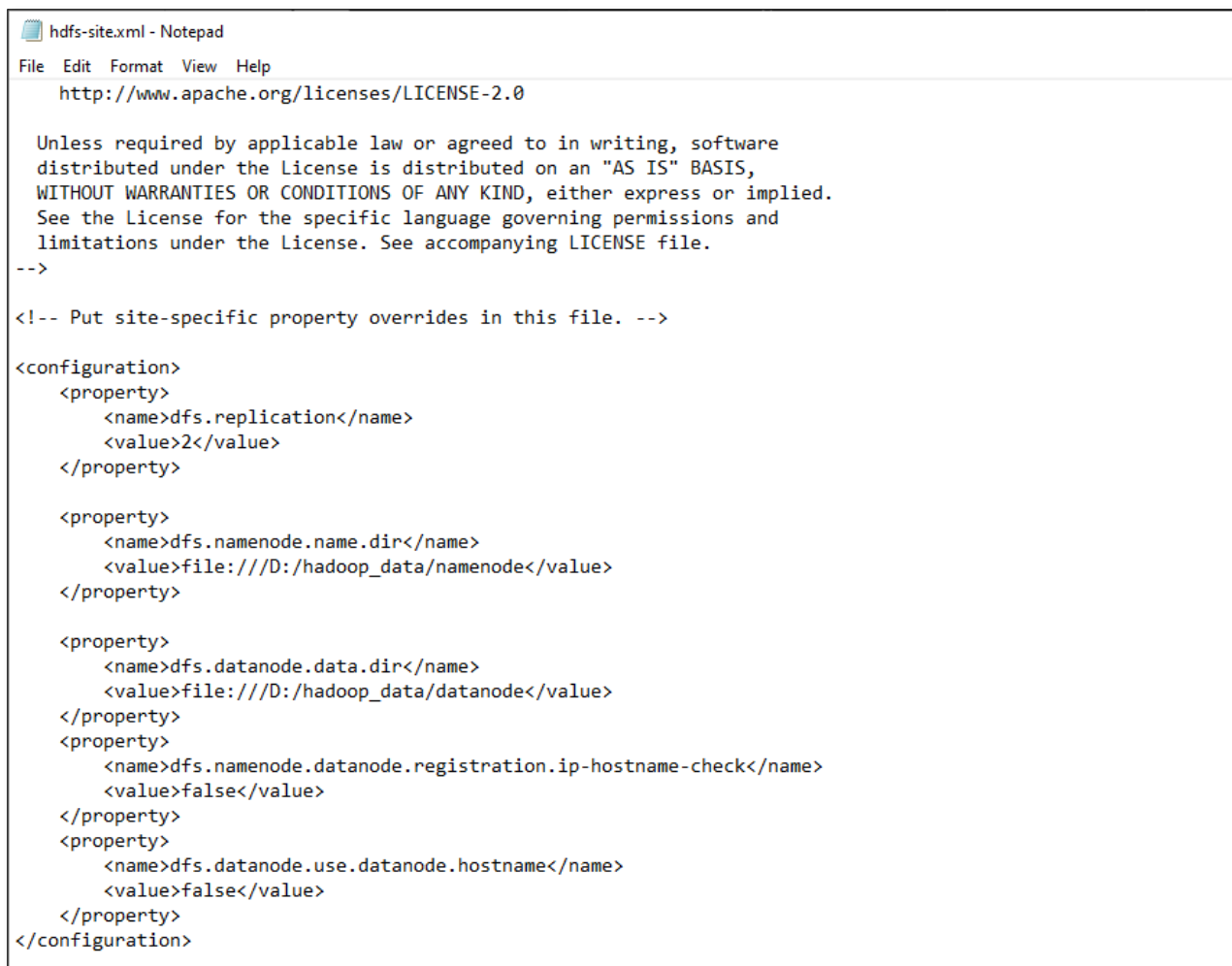
Hình 49: Kết quả điền file core-site.xml

2. File *hdfs-site.xml* (Cấu hình lưu trữ): Quy định nơi lưu dữ liệu và số bản copy (Replication).

```

<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value> </property>    #quy định replication là 2
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/D:/hadoop_data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/D:/hadoop_data/datanode</value>
  </property>
</configuration>

```



```

hdfs-site.xml - Notepad
File Edit Format View Help
http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>

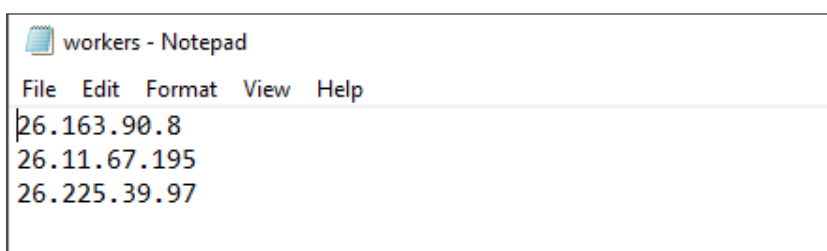
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///D:/hadoop_data/namenode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///D:/hadoop_data/datanode</value>
  </property>
  <property>
    <name>dfs.namenode.datanode.registration.ip-hostname-check</name>
    <value>false</value>
  </property>
  <property>
    <name>dfs.datanode.use.datanode.hostname</name>
    <value>false</value>
  </property>
</configuration>

```

Hình 50: Kết quả điền file hdfs-site.xml

3. File *workers* (Danh sách DataNode): Mở file *workers* (có thể không có đuôi .txt), xóa hết nội dung cũ và điền IP của 2 máy Worker vào.



```

workers - Notepad
File Edit Format View Help
26.163.90.8
26.11.67.195
26.225.39.97

```

Hình 51: kết quả điền file workers

4. File *hadoop-env.cmd* (Cấu hình Java): Tìm dòng set *JAVA_HOME*=... và sửa lại đường dẫn tới thư mục Java tại máy.

```
hadoop-env.cmd - Notepad
File Edit Format View Help
@rem optional. When running a distributed configuration it is best to
@rem set JAVA_HOME in this file, so that it is correctly defined on
@rem remote nodes.

@rem The java implementation to use. Required.
set JAVA_HOME=D:\Java

@rem The javac implementation to use. Javac is required to run secure datanodes.
@rem set JAVAC_HOME=%JAVAC_HOME%

@rem set HADOOP_CONF_DIR=
```

Hình 52: Cấu hình đường dẫn Java

Sau khi cấu hình trong các file quan trọng, tiến hành đồng bộ hóa ở tất cả các nút Worker muốn tham gia vào hệ thống lưu trữ HDFS. Sau khi đồng bộ tất cả tiến hành gõ lệnh: `hdfs namenode -format`. Nếu thấy thông báo *"Storage directory... has been successfully formatted"* là thành công.

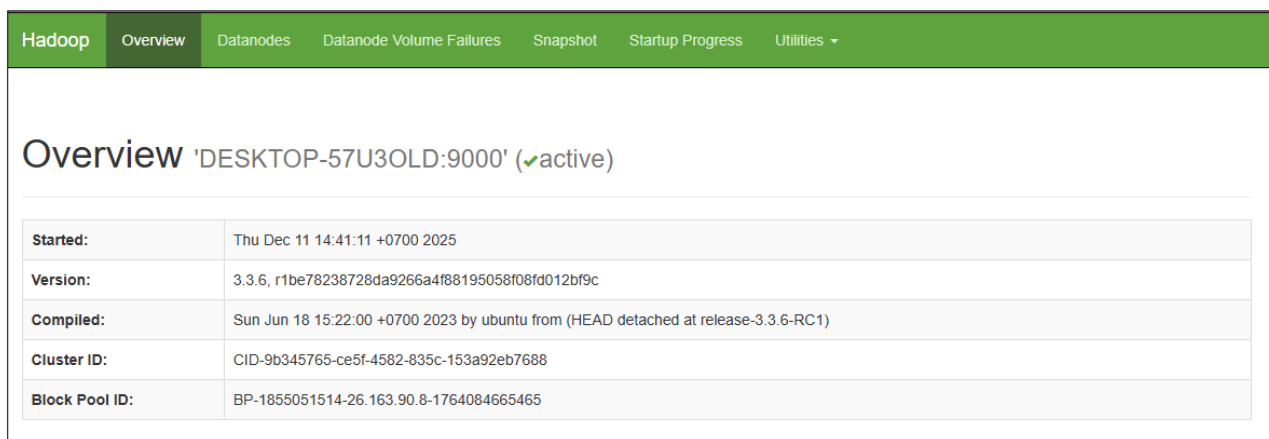
Kích hoạt hệ thống lưu trữ HDFS với dòng lệnh cho máy Master – Namenode:

```
start hdfs namenode
```

Kích hoạt hệ thống lưu trữ HDFS với dòng lệnh cho máy Worker – Datanode:

```
start hdfs datanode
```

Giao diện quản lý HDFS ở link: <http://localhost:9870>



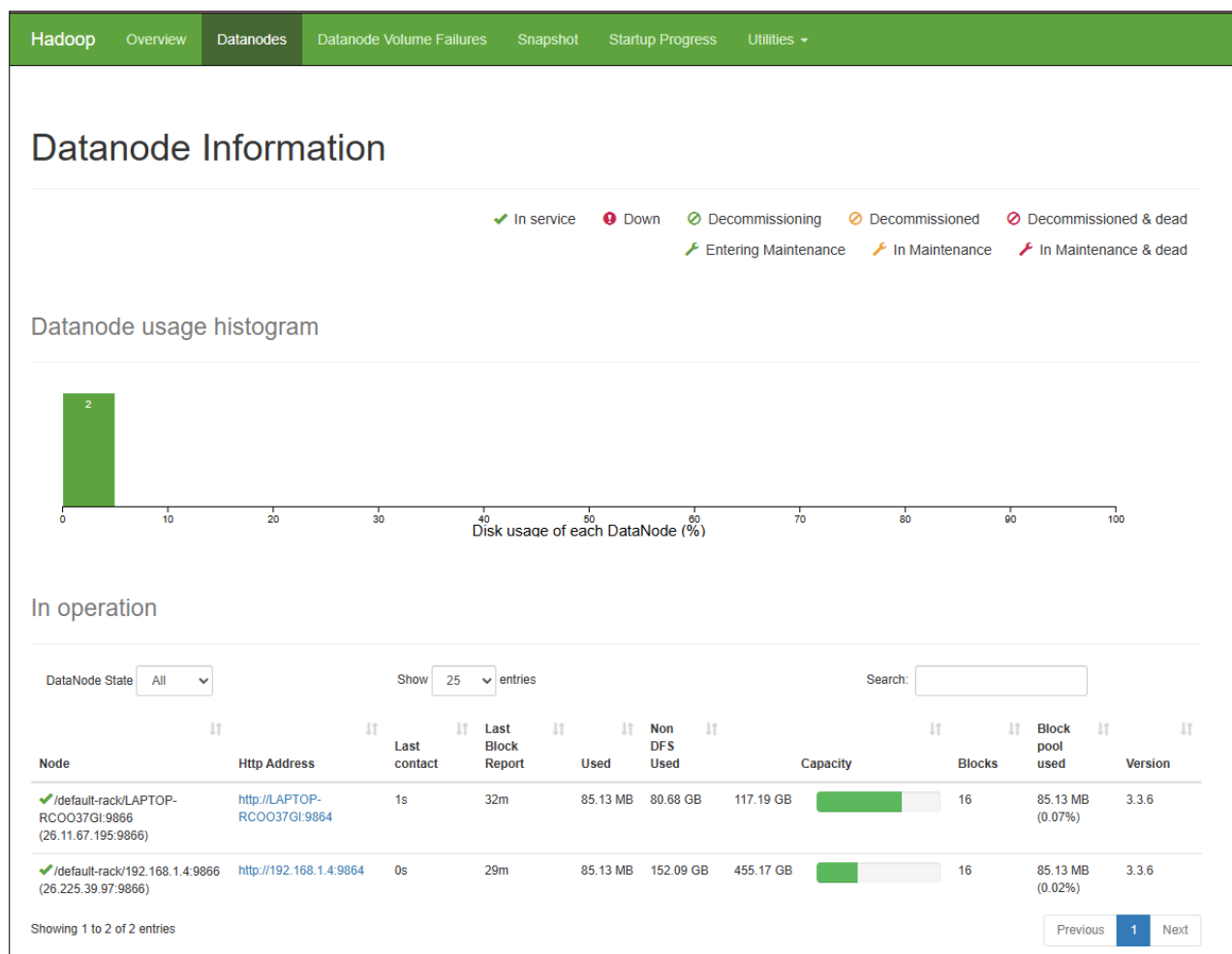
Hình 53: Overview của hệ thống HDFS

HDFS đang hoạt động trên cụm có địa chỉ `DESKTOP-57U3OLD:9000` (trạng thái active). Các thông tin bên dưới cho biết dịch vụ NameNode đã được khởi động từ 14:41:11, ngày 11/12/2025, đang sử dụng phiên bản Hadoop 3.3.6 (build bởi Ubuntu, biên dịch ngày 18/6/2023). Trên cùng một giao diện, hệ thống cũng hiển thị Cluster ID và Block Pool ID, là các mã định danh duy nhất dùng để nhận diện cụm HDFS và không gian lưu trữ các block dữ liệu. Nhìn chung, cụm HDFS đã được cài đặt, cấu hình thành công và đang ở trạng thái ổn định, sẵn sàng làm tầng lưu trữ phân tán cho các job xử lý dữ liệu lớn (ví dụ như Spark, MapReduce) trong các thí nghiệm của đề tài.

Summary	
Security is off.	
Safemode is off.	
26 files and directories, 16 blocks (16 replicated blocks, 0 erasure coded block groups) = 42 total filesystem object(s).	
Heap Memory used 170.51 MB of 333 MB Heap Memory. Max Heap Memory is 889 MB.	
Non Heap Memory used 56.32 MB of 57.61 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.	
Configured Capacity:	572.35 GB
Configured Remote Capacity:	0 B
DFS Used:	170.26 MB (0.03%)
Non DFS Used:	232.77 GB
DFS Remaining:	339.42 GB (59.3%)
Block Pool Used:	170.26 MB (0.03%)
DataNodes usages% (Min/Median/Max/stdDev):	0.02% / 0.07% / 0.07% / 0.03%
Live Nodes	2 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	16
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Thu Dec 11 14:41:11 +0700 2025
Last Checkpoint Time	Thu Dec 11 14:38:27 +0700 2025
Enabled Erasure Coding Policies	RS-6-3-1024k

Hình 54: Thống kê của hệ thống HDFS

Giao diện Summary của NameNode cho thấy cụm HDFS đang hoạt động ổn định và còn rất nhiều dư địa lưu trữ. Trước hết, trạng thái Security is off và Safemode is off cho thấy hệ thống đang ở chế độ vận hành bình thường, không bị khóa ghi. Tổng dung lượng cấu hình của cụm đạt khoảng 572,35 GB, trong đó phần không gian dành cho HDFS mới sử dụng 170,26 MB (0,03%), phần Non-DFS (hệ điều hành, ứng dụng, log...) chiếm khoảng 232,77 GB, và dung lượng còn lại dành cho HDFS vẫn còn khoảng 339,42 GB (59,3%). Hệ thống hiện quản lý 26 file/thư mục với 16 block, tất cả đều đã được nhân bản đầy đủ (không có under-replicated blocks), số block chờ xóa là 0, tức là không có vấn đề tồn đọng về nhất quán dữ liệu. Ở phía hạ tầng, cụm có 2 Live Nodes, không có node chết hoặc đang decommission/maintenance, DataNodes usages dao động quanh 0,02–0,07% cho thấy mức tải đĩa còn rất thấp. Các thông tin về thời điểm checkpoint và block deletion start time đều gần với thời gian khởi chạy NameNode, chứng tỏ cơ chế quản lý metadata và dọn dẹp block đang hoạt động bình thường. Nhìn chung, bảng Summary xác nhận cụm HDFS của đề tài đang ở trạng thái “khỏe mạnh”, tài nguyên lưu trữ dồi dào và đủ tin cậy để làm tầng dữ liệu nền cho các thí nghiệm Spark/MapReduce.

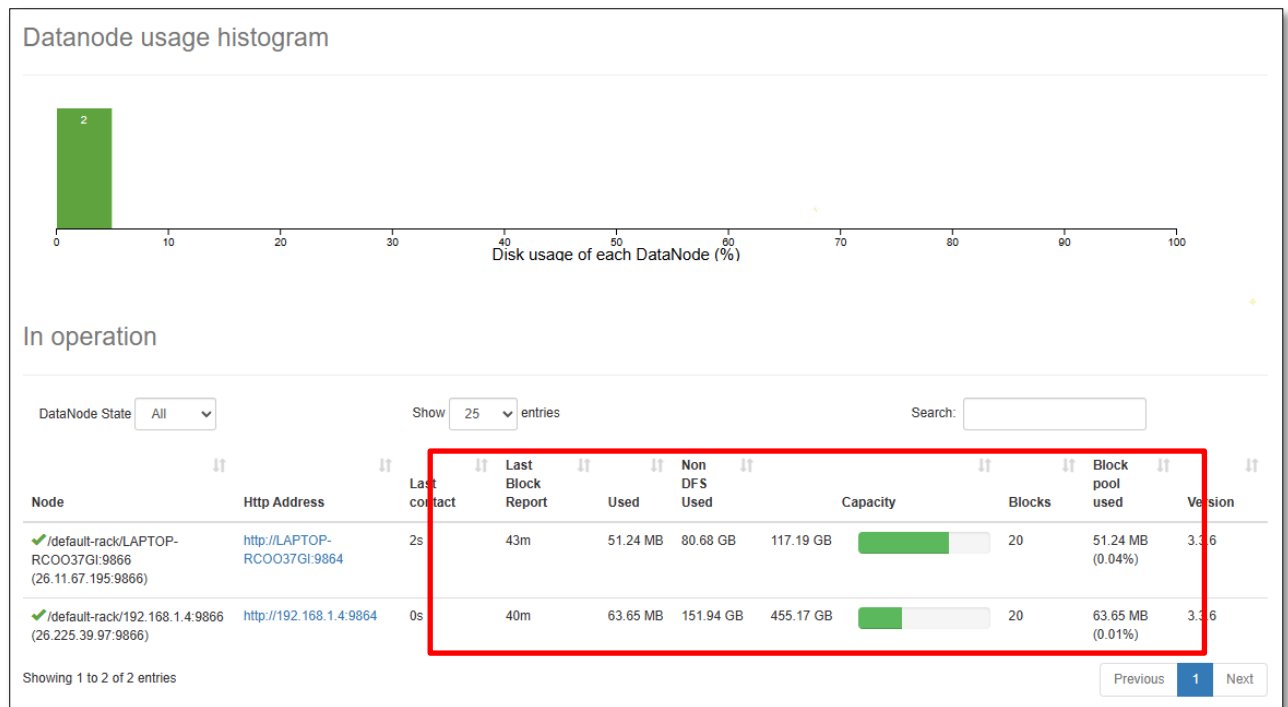


Hình 55: Thông tin về lưu trữ của các Datanode

Hình Ảnh “Datanode Information” cho thấy cụm HDFS hiện đang vận hành với 2 **DataNode** đều ở trạng thái *In service*, không có node nào Down, Decommissioning hay In Maintenance. Biểu đồ histogram phía trên thể hiện cả hai DataNode đều có mức sử dụng đĩa rất thấp (nằm trong khoảng 0–10%), phù hợp với kết luận ở phần Summary rằng hệ thống mới chỉ sử dụng một phần rất nhỏ dung lượng lưu trữ. Bảng “In operation” bên dưới cung cấp thông tin chi tiết: mỗi DataNode đều gửi tín hiệu “Last contact” về NameNode trong vòng 0–1 giây gần nhất, “Last Block Report” trong khoảng 29–32 phút, chứng tỏ đường truyền và cơ chế báo cáo block vẫn hoạt động ổn định. Về dung lượng, DataNode thứ nhất có tổng Capacity khoảng 117,19 GB, DataNode thứ hai khoảng 455,17 GB; lượng dữ liệu HDFS thực sự sử dụng trên mỗi node chỉ khoảng 85,13 MB (0,07% và 0,02%), trong khi phần Non-DFS Used chủ yếu dành cho hệ điều hành và các ứng dụng khác. Cả hai node đều đang lưu 16 block, phản ánh việc HDFS đã nhân bản dữ liệu một cách cân bằng giữa các DataNode. Nhìn chung, giao diện này khẳng định tầng lưu trữ phân tán đang ở trạng thái ổn định.

5.2.2. Tạo dữ liệu và chuyển sang các máy với HDFS:

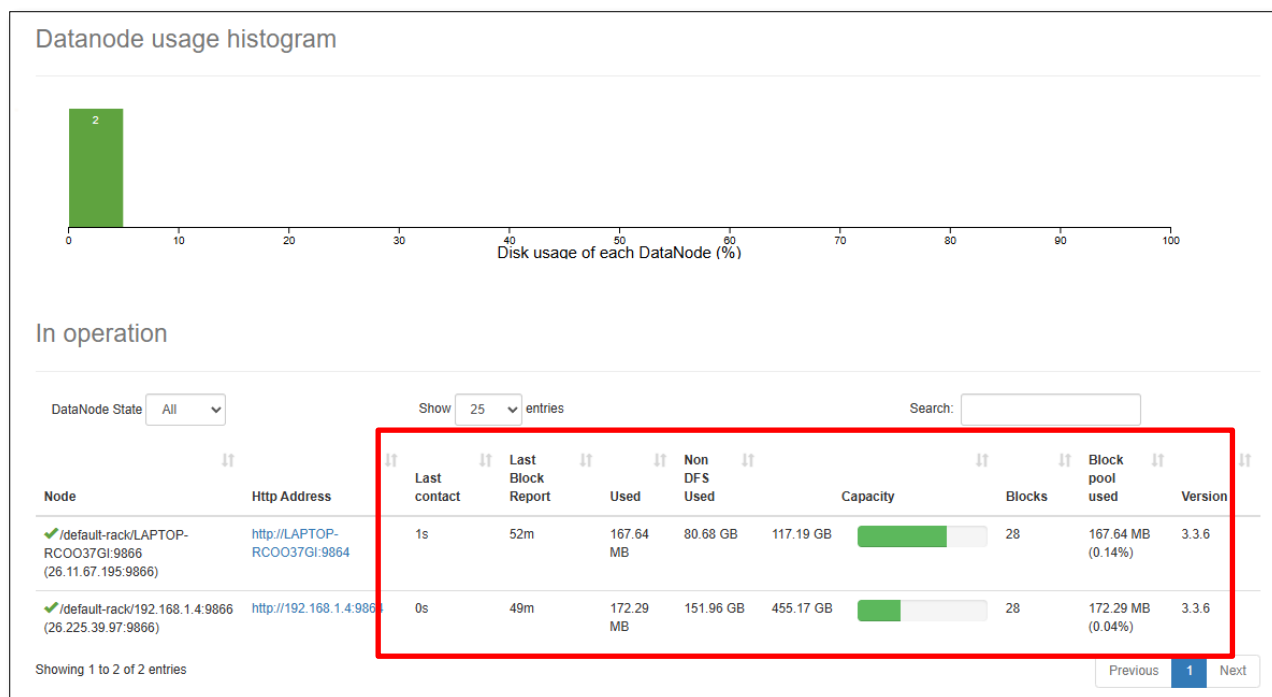
Tiến hành tạo dữ liệu giả lập tương tự như các bước trước. Sau đó để cho HDFS phân tán lên các node trong hệ thống.



Hình 56: Hoạt động phân chia dữ liệu của HDFS (16 tăng lên 20 blocks)

```
=====
SIZE (Dòng)      | SINH DATA (s)    | UPLOAD HDFS (s)
-----
1000000          | 0.02              | 19.23
2000000          | 0.05              | 33.91
5000000          | 0.10              | 35.02
10000000         | 0.23              | 58.78
20000000         | 0.45              | 71.53
30000000         | 0.67              | 121.92
40000000         | 0.88              | 172.24
=====
```

Hình 57: Kết quả trả về sau khi sinh dữ liệu thành công



Hình 58: Kết quả cuối sau khi HDFS phân tán thành công

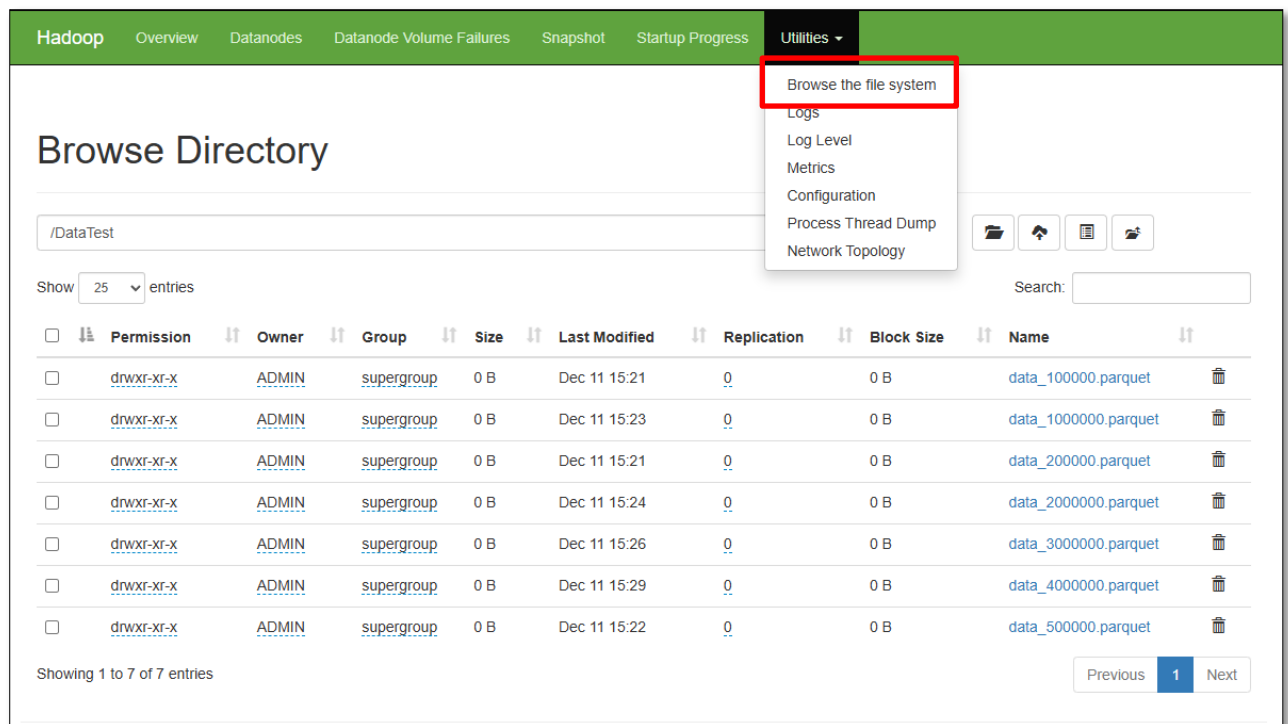
Sau khi phân tán dữ liệu cho thấy cụm HDFS đã lưu trữ dữ liệu thành công và tương đối cân bằng trên hai DataNode. Biểu đồ phía trên thể hiện cả hai node đều đang hoạt động (In service) và mức sử dụng đĩa vẫn ở ngưỡng rất thấp, dưới 1% tổng dung lượng. Bảng “In operation” cho biết DataNode thứ nhất có dung lượng khả dụng khoảng 117,19 GB, đang dùng 167,64 MB cho HDFS (chiếm 0,14%) với 28 block, đã tăng lên so với 16 block – 85.13MB so với trước khi phân tán. DataNode thứ hai có dung lượng 455,17 GB, đang dùng 172,29 MB (0,04%) với 28 block, đã tăng lên so với 16 block – 85.13MB so với trước khi phân tán.

Để kiểm tra xem dữ liệu đã thực sự được tạo ra chưa, ta có thể dùng `hdfs dfs -ls /DataTest` trong CMD.

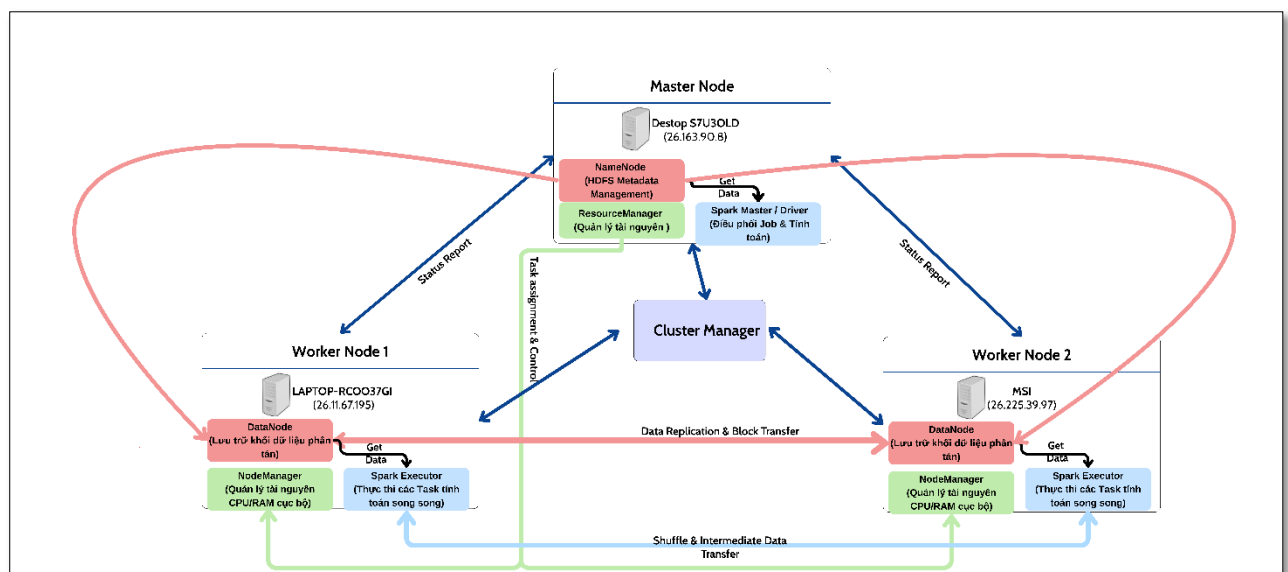
```
C:\Users\ADMIN>hdfs dfs -ls /DataTest
Found 7 items
drwxr-xr-x - ADMIN supergroup 0 2025-12-11 15:21 /DataTest/data_100000.parquet
drwxr-xr-x - ADMIN supergroup 0 2025-12-11 15:23 /DataTest/data_1000000.parquet
drwxr-xr-x - ADMIN supergroup 0 2025-12-11 15:21 /DataTest/data_200000.parquet
drwxr-xr-x - ADMIN supergroup 0 2025-12-11 15:24 /DataTest/data_2000000.parquet
drwxr-xr-x - ADMIN supergroup 0 2025-12-11 15:26 /DataTest/data_3000000.parquet
drwxr-xr-x - ADMIN supergroup 0 2025-12-11 15:29 /DataTest/data_4000000.parquet
drwxr-xr-x - ADMIN supergroup 0 2025-12-11 15:22 /DataTest/data_500000.parquet
```

Hình 59: Kết quả kiểm tra các file dữ liệu được HDFS phân tán

Hoặc kiểm tra trực tiếp trong giao diện quản lý của HDFS.



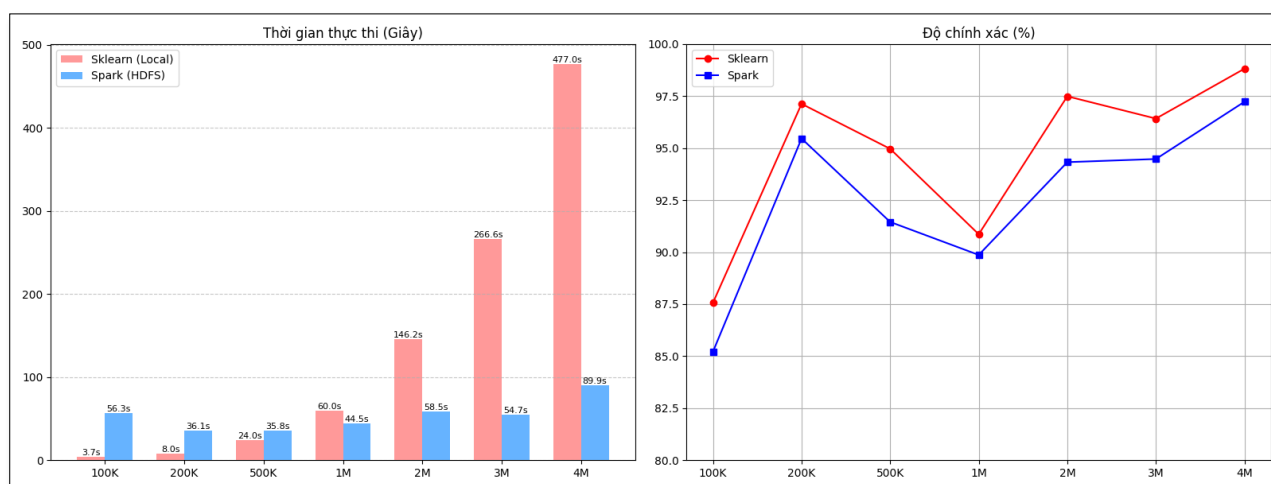
Hình 60: Kiểm tra dữ liệu thông qua giao diện HDFS
Cả hai kết quả đều cho thấy, dữ liệu đã được đẩy lên hệ thống đầy đủ.
5.2.3. Đánh giá kết quả chạy thuật toán Random Forest với HDFS:



Hình 61: Kiến trúc phân tán 2 Worker với HDFS

Độ lớn (dòng)	Thời gian chạy tuần tự (giây)	Thời gian chạy song song (24 cores) (giây)	Độ chính xác chạy tuần tự (%)	Độ chính xác chạy song song (24 cores) (%)
100000	3.70	56.28	87.57	85.22
200000	8.02	36.09	97.13	95.46
500000	23.98	35.80	94.97	91.45
1 triệu	60.03	44.52	90.86	89.87
2 triệu	146.171	58.45	97.49	94.33
3 triệu	266.64	54.71	96.42	94.48
4 triệu	476.98	89.90	98.82	97.23

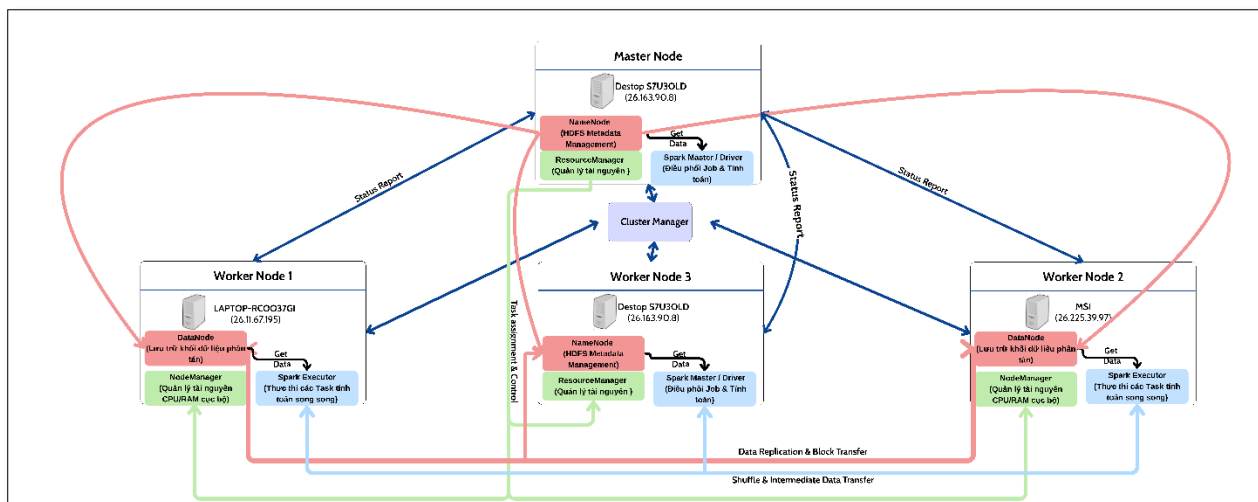
Bảng 10: So sánh kết quả chạy thuật toán và độ chính xác



Hình 62: So sánh kết quả chạy thuật toán và độ chính xác (HDFS)

Kết quả cho thấy rõ sự khác biệt về tốc độ và độ chính xác giữa mô hình Random Forest chạy tuần tự và chạy song song trên Spark (HDFS, 24 cores) theo các mức độ lớn dữ liệu khác nhau. Với các tập dữ liệu nhỏ đến trung bình (100.000 đến 500.000 dòng), thời gian thực thi tuần tự vẫn ngắn hơn chạy song song do chi phí khởi tạo phiên Spark, phân tán và đọc dữ liệu từ HDFS khá lớn (ví dụ ở mức 100.000 dòng: 3,70 giây so với 56,28 giây). Tuy nhiên, khi dữ liệu tăng từ 1 đến 4 triệu dòng, tốc độ của Spark bắt đầu thể hiện ưu thế: thời gian chạy song song chỉ tăng rất ít (từ 44,52 lên 89,90 giây), trong khi thời gian chạy tuần tự tăng mạnh từ 60,03 lên 476,98 giây, chênh lệch ở mức 4 triệu dòng lên tới hơn 5 lần. Về độ chính xác, mô hình chạy tuần tự nhìn chung nhỉnh hơn khoảng 1 cho đến 3% so với Spark ở hầu hết các mức dữ liệu nhưng cả hai cách triển khai đều duy trì mức chính xác cao (trên 90%). Như vậy, kết quả cho thấy: với dữ liệu nhỏ, triển khai tuần tự là phù hợp hơn. Còn khi kích thước dữ liệu tăng lên hàng triệu bản ghi, việc sử dụng Spark trên HDFS giúp rút ngắn đáng kể thời gian xử lý mà vẫn bảo đảm chất lượng mô hình ở mức chấp nhận được.

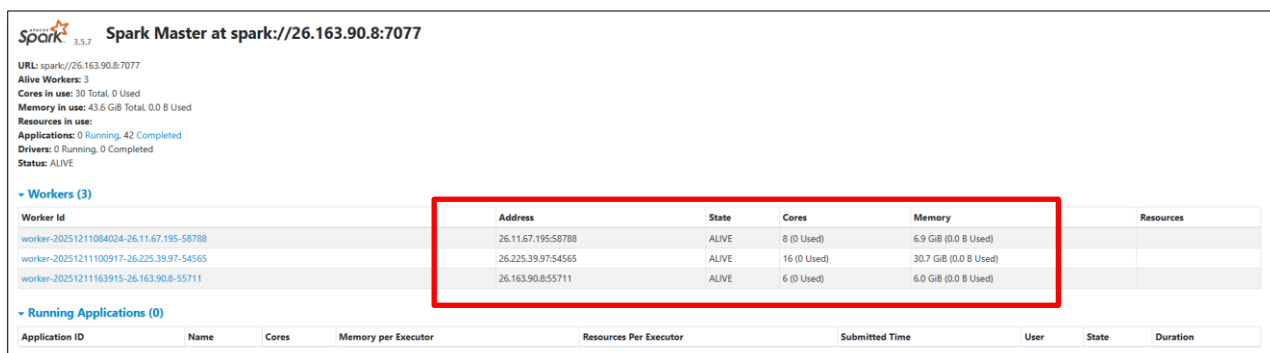
Để kiểm tra xem khi số nhân tính toán tăng lên, thì mô hình có học nhanh hơn không. Ta tiếp tục kết nối thêm máy Worker thứ 3 để tăng số nhân lên 30. Trong thực tế, không thường sử dụng máy Master để tham gia vào quá trình tính toán, tuy nhiên trong phạm vi môn học, sử dụng thêm máy để có thể so sánh hiệu năng phần cứng.



Hình 63: Kiến trúc phân tán với 3 Workers

Node	IP	Cores	Memory
Master	26.163.90.8	-	-
Worker 1	26.11.67.195	8	6.9
Worker 2	26.225.39.97	16	30.7
Worker 3 (giới hạn cấp phát tài nguyên)	26.163.90.8	6	6

Bảng 11: Danh sách tài nguyên cụm máy (3 workers)

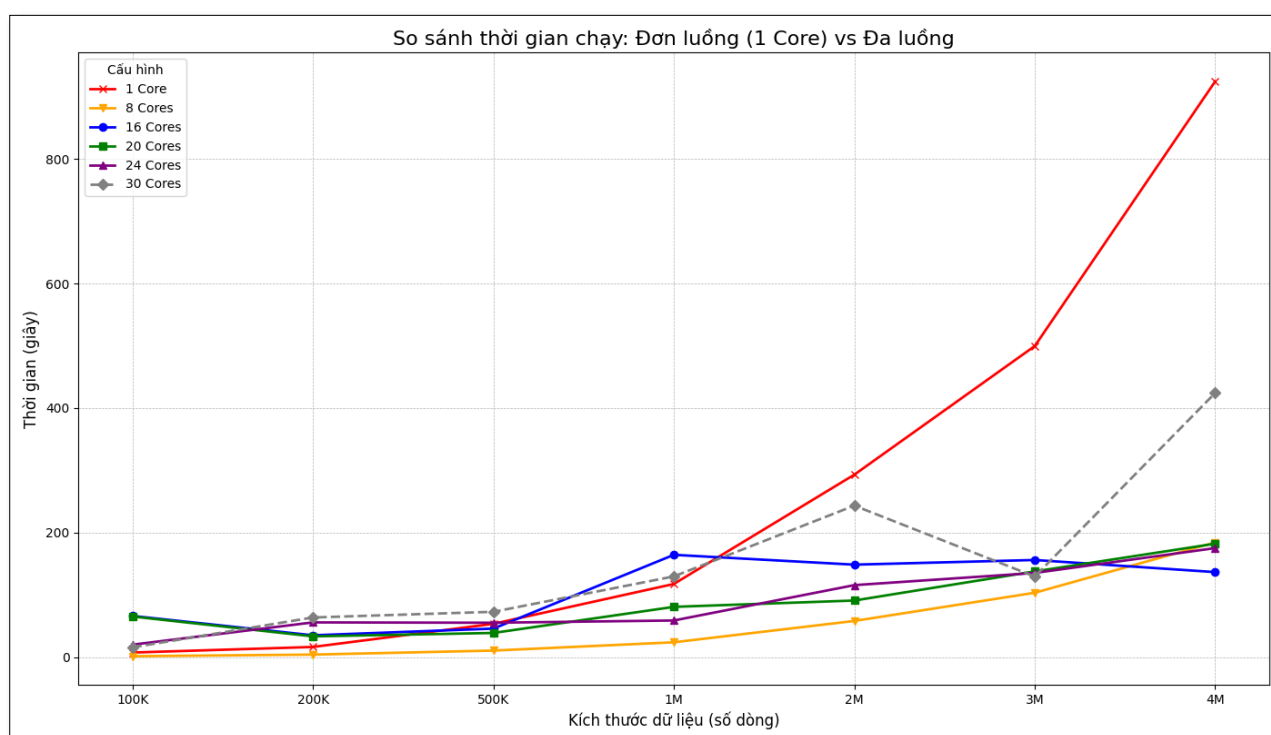


Hình 64: Giao diện quản lý cụm gồm 3 workers

Độ lớn (dòng)	Thời gian chạy song song (giây)					
	1 cores	8 cores	16 cores	20 cores	24 cores	30 cores

100000	7.50	1.61	66.30	65.37	20.14	15.85
200000	16.48	4.19	35.19	33.49	55.90	63.82
500000	53.62	10.68	45.95	38.96	55.36	72.93
1 triệu	117.62	23.96	164.40	80.86	58.98	129.74
2 triệu	293.08	58.22	148.61	91.01	115.73	243.40
3 triệu	499.60	103.50	156.10	137.76	135.30	129.58
4 triệu	924.55	184.4	136.66	182.26	175.21	424.30

Bảng 12: So sánh thời gian chạy với số nhân



Hình 65: So sánh thời gian chạy theo số nhân (Cores)

Hình trên cho thấy rất rõ tác động của số nhân (cores) tới thời gian xử lý của Random Forest trên Spark. Khi chỉ dùng 1 core, thời gian thực thi tăng cực nhanh theo kích thước dữ liệu và vượt xa các cấu hình đa luồng, chứng tỏ lợi ích rõ rệt của việc khai thác song song trên nhiều nhân. Tuy nhiên, giữa các cấu hình đa nhân, hiệu năng không tăng tuyến tính theo số core: 8 đến 16 cores cho xu hướng tăng thời gian khá ổn định, trong khi mức 20 đến 24 cores thường nằm trong nhóm tốt nhất ở các mức dữ liệu lớn. Riêng cấu hình 30 cores nhiều lần cho thời gian cao bất thường, đặc biệt ở mức 2 triệu đến 4 triệu dòng, phản ánh chi phí overhead của việc chia nhỏ quá nhiều task, đồng bộ và truyền thông trong cụm. Nhìn chung, kết quả minh họa rõ ràng tăng thêm core chỉ có ý nghĩa đến một ngưỡng nhất định. Sau đó, lợi ích song song hóa bị bão hòa và thậm chí bị triệt tiêu bởi chi phí giao tiếp. Vì vậy, với

cấu hình cụm và bài toán trong thí nghiệm này, khoảng 16 đến 24 cores là vùng lựa chọn vừa đủ, cân bằng giữa tốc độ xử lý và overhead của hệ thống phân tán. Song song hóa là cần thiết nhưng không phải công cụ toàn diện, so với 1 core, các cấu hình đa nhân giúp thời gian giảm rất mạnh, nhưng sau một ngưỡng nhất định (khoảng 16–24 cores) lợi ích bắt đầu bão hòa. Tiếp đó, tăng số core luôn đi kèm overhead (chia nhỏ job, khởi tạo nhiều task, đồng bộ, shuffle dữ liệu), nên nếu “bơm” quá nhiều core như cấu hình 30 cores thì chi phí giao tiếp và tranh chấp tài nguyên có thể khiến thời gian chạy... tăng lên thay vì giảm. Thứ ba, hiệu năng của hệ thống phân tán không chỉ phụ thuộc vào số core danh nghĩa, mà còn phụ thuộc vào cách chia partition, số executor, kích thước dữ liệu và tình trạng cụm tại thời điểm chạy. Vì vậy, khi triển khai Spark thực tế, cần coi việc chọn cấu hình tài nguyên là một bài toán tối ưu: thử nghiệm, đo đạc và tìm ra vùng “vừa đủ” thay vì mặc định rằng càng nhiều core càng tốt.

Ưu điểm của HDFS (so với mirroring thủ công):

- Quản lý tập trung, nhất quán: NameNode quản lý toàn bộ metadata (tên file, vị trí block, số bản sao...), tránh tình trạng mỗi máy một phiên bản dữ liệu như khi copy thủ công.
- Replication tự động & chịu lỗi tốt: HDFS tự động nhân bản block theo replication factor, tự theo dõi node hỏng và sao chép bù sang node khác; với mirroring thủ công, mất file ở một node là phải tự copy lại.
- Tận dụng tốt data locality: NameNode biết chính xác block nằm ở đâu để scheduler (Spark/MapReduce) ưu tiên gán task “gần dữ liệu”; nếu chỉ copy thủ công, việc lập lịch theo locality thường kém hiệu quả hơn.
- Dễ mở rộng quy mô: Khi thêm node mới, chỉ cần cấu hình DataNode, HDFS tự động phân phối dữ liệu và cân bằng tải; còn với mirroring thủ công, phải tổ chức copy lại dữ liệu cho từng node mới.
- Công cụ giám sát, bảo trì đầy đủ: Giao diện web (NameNode UI) cho phép theo dõi live nodes, dung lượng, under-replicated blocks, lỗi volume..., giúp quản trị cụm dễ dàng hơn rất nhiều so với việc tự kiểm tra từng máy.

Nhược điểm:

- Cài đặt và cấu hình phức tạp hơn: Cần thiết lập NameNode, DataNode, SSH, quyền truy cập, cấu hình core-site, hdfs-site..., trong khi mirroring thủ công chỉ cần sao chép file.
- Tốn thêm tài nguyên cho dịch vụ nền: NameNode và DataNode luôn chạy, sử dụng RAM, CPU, dung lượng đĩa cho metadata và log, với cụm rất nhỏ, chi phí này đôi khi không “đáng” so với lợi ích.
- Không phù hợp cho quá nhiều file nhỏ: Kiến trúc block lớn (64–128 MB) khiến HDFS kém hiệu quả nếu lưu hàng triệu file rất nhỏ; trong một số trường hợp, mirroring thủ công trên filesystem thường (ext4, NTFS) lại nhẹ nhàng hơn.
- Độ phức tạp vận hành: Cần backup NameNode, checkpoint, theo dõi trạng thái cluster; nếu thiếu kinh nghiệm quản trị, hệ thống có thể gặp lỗi khó xử lý hơn so với việc chỉ dùng ổ đĩa local.

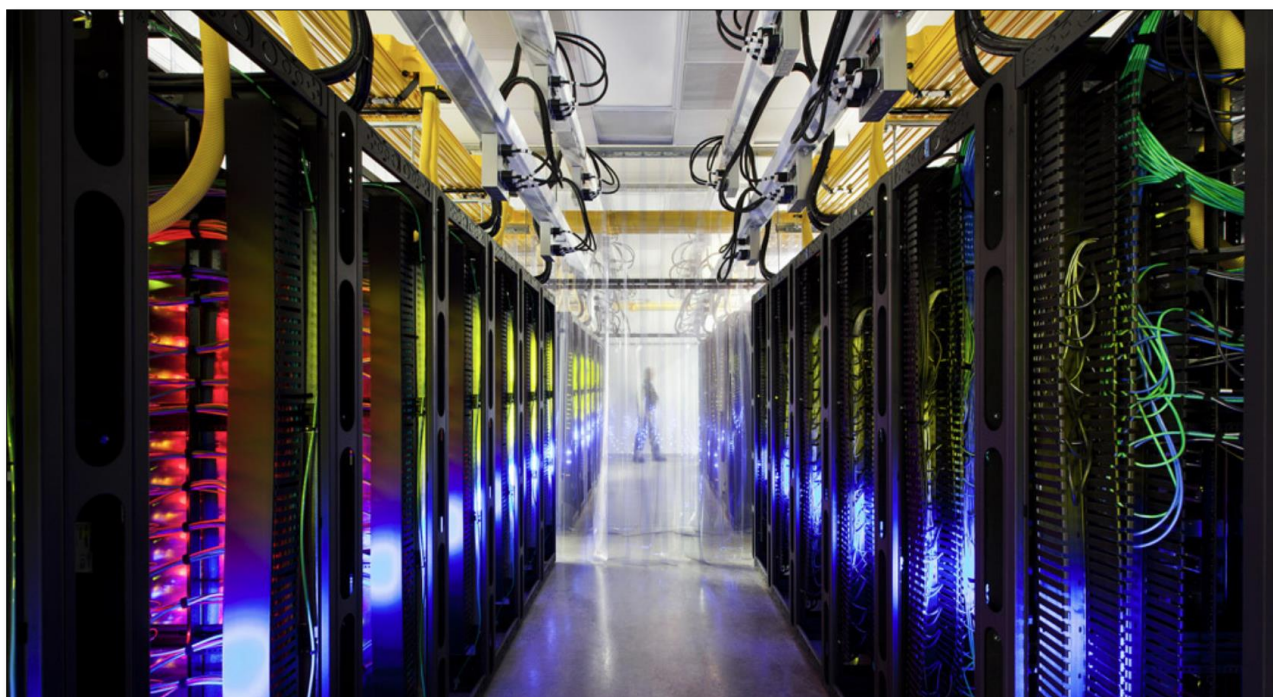
Với cụm nhỏ, ít node và dữ liệu ít thay đổi, mirroring thủ công có thể tạm chấp nhận. Tuy nhiên, khi hệ thống hướng tới xử lý dữ liệu lớn, nhiều node và yêu cầu chịu lỗi tốt, HDFS là lựa chọn vượt trội nhờ cơ chế replication tự động, quản lý tập trung và khả năng mở rộng.

6. Nhận xét tổng thể:

Nhìn tổng thể, các thí nghiệm cho thấy hệ thống phân tán được xây dựng (HDFS + Spark) đã vận hành ổn định, thực hiện đúng cơ chế nhân bản, phân tán và xử lý song song dữ liệu. Qua so sánh giữa chạy tuần tự và chạy trên cụm nhiều nhân, có thể thấy rõ ưu thế về khả năng mở rộng của mô hình phân tán khi dữ liệu tăng lên hàng triệu bản ghi, đồng thời độ chính xác của mô hình học máy vẫn được bảo toàn ở mức chấp nhận được. Từ đó có thể rút ra ba yếu tố quan trọng nhất đối với một hệ thống phân tán:

1. Phần mềm: Lựa chọn nền tảng (Hadoop, HDFS, Spark, hệ điều hành) và cấu hình đúng cơ chế replication, scheduler, cache, giúp bảo đảm tính tin cậy, khả năng chịu lỗi và thuận tiện cho lập trình.

2. Phần cứng: Số lượng node, số lõi CPU, dung lượng RAM, ổ đĩa và băng thông mạng quyết định trực tiếp đến tốc độ xử lý và ngưỡng dữ liệu mà hệ thống có thể mở rộng. Google sở hữu các “warehouse-scale computer” với hàng trăm nghìn máy chủ. Các hệ thống như MapReduce, BigTable, Spanner vận hành trên cụm khổng lồ, tận dụng số lượng CPU core và dung lượng RAM cực lớn để xử lý tìm kiếm, quảng cáo, YouTube... gần như theo thời gian thực. Amazon (AWS): Cung cấp các cụm EMR, Redshift, Aurora, EC2 với cấu hình rất đa dạng (nhiều vCPU, RAM, SSD NVMe, mạng 25–100 Gbps). Đây là ví dụ rõ nhất về việc dùng “sức mạnh phần cứng” để mở rộng khả năng xử lý dữ liệu cho khách hàng doanh nghiệp. Nhờ đầu tư vào phần cứng (số node, core, RAM, ổ đĩa nhanh, mạng tốc độ cao), các công ty này có thể đào tạo mô hình, chạy truy vấn và phân tích dữ liệu khổng lồ trong thời gian ngắn.

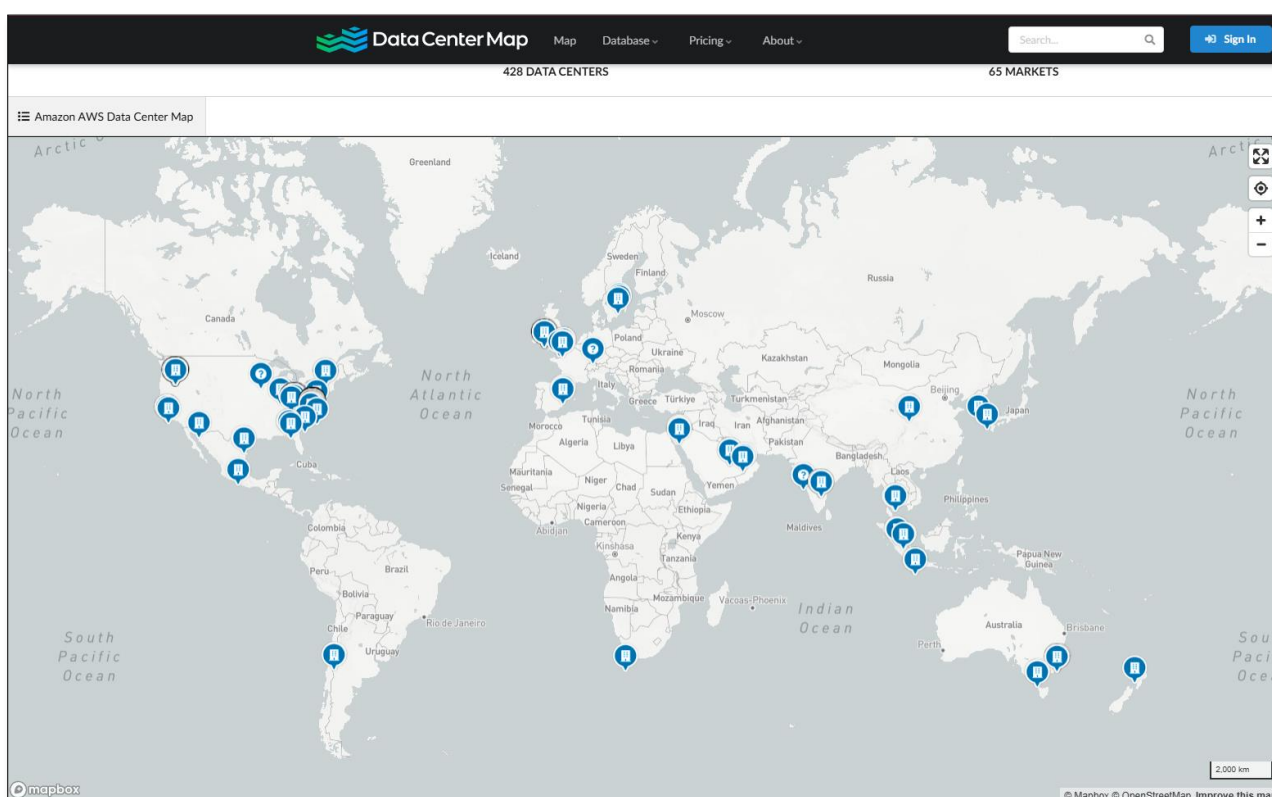


Hình 66: Warehouse-scale computer

3. Vị trí địa lý và kiến trúc mạng: Việc bố trí các nút trong cùng LAN, cùng rack hay phân tán theo nhiều khu vực ảnh hưởng rất lớn đến độ trễ truyền dữ liệu và chi phí giao tiếp, do đó cần thiết kế hợp lý để tận dụng tối đa tính “gần dữ liệu” (data locality) và tránh nghẽn mạng. Microsoft Azure / Google Cloud / AWS: Luôn bố trí data center ở nhiều châu lục (US,

EU, Asia-Pacific...). Mỗi khu vực lại chia nhỏ thành nhiều availability zone, kết nối bằng đường truyền riêng tốc độ rất cao. Các hệ thống phân tán bên trong (HDFS tương đương, GFS/Colossus, S3, GCS...) được thiết kế để vừa nhân bản dữ liệu giữa các rack trong cùng data center, vừa sao lưu sang khu vực địa lý khác để phòng thảm họa. Alibaba Cloud / Tencent Cloud: Tập trung mạnh ở Trung Quốc và Đông Nam Á, tối ưu kiến trúc mạng để giảm độ trễ giữa các thành phố lớn (Bắc Kinh, Thượng Hải, Hàng Châu, Singapore...). Khi triển khai hệ thống phân tán cho thương mại điện tử, họ luôn chọn data center “gần người dùng nhất” để đảm bảo tốc độ truy cập và giảm chi phí truyền dữ liệu xuyên biên giới. Đây là ví dụ cho thấy “vị trí địa lý và kiến trúc mạng” ảnh hưởng trực tiếp đến độ trễ, băng thông và chi phí vận hành; các hãng lớn luôn phải thiết kế hệ thống theo hướng tận dụng tối đa data locality và giảm tối đa lưu lượng phải đi xa.

Ba yếu tố này gắn bó chặt chẽ với nhau, tạo nên nền tảng cho một hệ thống phân tán hiệu quả, ổn định và có khả năng mở rộng trong thực tế.



Hình 67: Amazon AWS có 428 server ở khắp nơi. Khắc phục điểm yếu về địa lý.

7. Nhận xét về quá trình thực hiện tiểu luận cuối kỳ và hướng phát triển:

Sau khi hoàn thành tiểu luận cuối kỳ về *tìm hiểu Apache Spark và ứng dụng thuật toán Random Forest*, nhóm sinh viên nhận thấy đây là một quá trình học tập nghiêm túc và hiệu quả, giúp củng cố kiến thức lý thuyết cốt lõi của môn học, đồng thời phát triển kỹ năng phân tích, trình bày và tư duy dữ liệu một cách khoa học.

- Kiến thức nền tảng về hệ thống phân tán:** Thông qua việc cài đặt Hadoop, cấu hình HDFS và triển khai Spark cluster, sinh viên nắm chắc hơn các khái niệm cốt lõi như kiến trúc master-worker, NameNode-DataNode, replication factor, data locality, cơ chế fault tolerance, cũng như mô hình lập trình MapReduce và DAG thực thi của Spark. Việc trực tiếp quan sát log, giao diện giám sát và các chỉ số về block, node, dung lượng đã giúp làm rõ sự khác biệt giữa xử lý tuần tự trên một máy và xử lý phân tán trên nhiều node.
- Kỹ năng phân tích thực nghiệm và tổ chức nội dung:** Tiểu luận yêu cầu thiết kế chuỗi thí nghiệm có kiểm soát, thay đổi độ lớn dữ liệu từ 100.000 đến 4 triệu dòng, so sánh thời gian chạy và độ chính xác giữa mô hình chạy local và chạy trên Spark/HDFS. Quá trình này rèn luyện kỹ năng xây dựng kịch bản thử nghiệm, thu thập số liệu, biểu diễn kết quả bằng bảng và đồ thị, sau đó phân tích xu hướng, chỉ ra ngưỡng dữ liệu mà tại đó hệ thống phân tán bắt đầu phát huy ưu thế so với cách triển khai tuần tự.
- Kỹ năng triển khai và cấu hình hệ thống:** Việc thiết lập cụm Hadoop – Spark với 2 DataNode, cấu hình replication, quản lý tài nguyên CPU/RAM và phân tích trạng thái cluster trên giao diện web giúp sinh viên hiểu sâu hơn về cách một hệ thống phân tán được vận hành. Bên cạnh HDFS, nhóm cũng thử nghiệm phương án mirroring dữ liệu thủ công, từ đó rút ra nhận xét thực tế về ưu – nhược điểm của từng cách tiếp cận trong quản lý dữ liệu phân tán.
- Tư duy tối ưu và đánh đổi hiệu năng – độ chính xác:** Khi đo thời gian và độ chính xác của Random Forest trên các cấu hình khác nhau, sinh viên nhận thức rõ hơn về “cái giá” phải trả cho việc song song hóa: với dữ liệu nhỏ, overhead khởi tạo Spark và truyền thông mạng khiến chạy local nhanh hơn; ngược lại, với dữ liệu hàng triệu bản ghi, Spark giúp rút ngắn thời gian tính toán rất đáng kể trong khi độ chính xác chỉ giảm nhẹ và vẫn nằm trong mức chấp nhận được. Đây là trải nghiệm trực tiếp về bài toán tối ưu – đánh đổi (trade-off) vốn là trọng tâm của môn học.
- Hạn chế và hướng phát triển:** Hệ thống thử nghiệm hiện mới dừng ở quy mô nhỏ, số node hạn chế, dữ liệu vẫn còn tương đối nhỏ gọn và chủ yếu tập trung vào một thuật toán Random Forest chạy ở chế độ batch. Trong tương lai, nhóm mong muốn mở rộng sang các kịch bản phức tạp hơn như xử lý streaming, thử nghiệm thêm các mô hình khác (Gradient Boosting, k-means, ALS...), triển khai cluster trên môi trường cloud, đồng thời ứng dụng các kỹ thuật tối ưu nâng cao như dynamic allocation, tối ưu partition hoặc kết hợp với container (Docker, Kubernetes).
- Khả năng tự học và khai thác tài liệu:** Để hoàn thành tiểu luận, sinh viên phải thường xuyên tra cứu tài liệu chính thức của Apache Hadoop/Spark, các blog kỹ thuật và diễn đàn cộng đồng nhằm giải quyết các lỗi cài đặt, cấu hình và tối ưu thực thi. Quá trình này không chỉ củng cố kiến thức chuyên môn về lập trình song song và phân tán mà còn rèn luyện năng lực tự học, đọc hiểu tài liệu tiếng Anh kỹ thuật – một năng lực không thể thiếu đối với kỹ sư làm việc với hệ thống dữ liệu lớn.

Tổng thể, tiểu luận không chỉ là bài kiểm tra cuối kỳ mà còn là bước làm quen đầu tiên với một hệ thống phân tán thực tế. Những kiến thức và kỹ năng thu nhận được sẽ là nền tảng quan trọng để sinh viên tiếp cận các học phần nâng cao về Big Data, hệ phân tán và học máy, cũng như góp phần hình thành tư duy thiết kế và đánh giá hệ thống một cách nghiêm túc, khoa học trong các dự án dữ liệu lớn sau này.

8. Tài liệu tham khảo:

Trang web:

1. Companies that use Apache Spark (62,271) | TheirStack.com.
2. Spark là gì? – Giới thiệu về Apache Spark và Phân tích – AWS.
3. Apache Spark Architecture 101: How Spark Works (2025).
4. The Good and the Bad of Apache Spark.
5. Machine Learning in Apache Spark Quick Start Guide | Qubole
6. Data Center Map - Colocation, Cloud and Connectivity

Bài đọc:

1. Chen, J., Li, K., Tang, Z., Bilal, K., Yu, S., Weng, C., & Li, K. (2018). A Parallel Random Forest Algorithm for Big Data in a Spark Cloud Computing Environment. *IEEE Transactions on Parallel and Distributed Systems.* , ,
2. Nguyễn, H. V., Nguyễn, H. P., & Trần, L. T. (2024). *Implementation of Classification Algorithms for Predicting Customer Subscription to Term Deposits on Apache Spark* (Đồ án cuối kì). Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM. , , ,
3. St. Anns College for Women. (n.d.). *Hadoop and Spark: A Comparative Analysis of Data Processing Techniques* (Master's in Computer Application material). ,
4. Tôn, T. K. T., & Đinh, L. Q. N. (2025). *Khai thác dữ liệu sử dụng Apache Spark* (Đồ án tốt nghiệp). Trường Đại học Đà Lạt.

Dữ liệu:.

1. Tập dữ liệu wordcount.txt dành cho Spark test của tác giả Github - nivdul.